

Christian Igel

Concepts of Deep Reinforcement Learning

A Short Course

March 27, 2026

Common notation

A	set of actions
a, a_t	action, action at time step t
α_H	entropy regularization weight
D	input dimension
$\mathcal{D}$	data set or mini batch
$\mathbb{E}$	expectation
$\mathbb{E}^\pi$	expectation over trajectories when following policy π
η^π	not normalized state distribution
	$\eta^\pi(s) = \mathbb{E}_{s_0 \sim p_{\text{start}}} \left[\sum_{k=0}^{\infty} \Pr\{s_0 \xrightarrow{k} s \mid \pi\} \right]$
η_γ^π	discounted state distribution
	$\eta_\gamma^\pi(s) \mathbb{E}_{s_0 \sim p_{\text{start}}} \left[\sum_{k=0}^{\infty} \gamma^k \Pr\{s_0 \xrightarrow{k} s \mid \pi\} \right]$
γ	discount factor
$G_t^{(n_R)}(c)$	multi-step return
$H(p)$	entropy of distribution p
H_{target}	target entropy
$\mathcal{H}$	hypotheses space
K	output dimension
$\text{KL}[p \parallel q]$	KL divergence between two distributions p and q
LHS	left hand side
ℓ	pointwise loss function
λ	offspring population size
λ_H	learning rate for entropy regularization
M	experience replay buffer
μ	parent population size
μ^π	on-policy distribution
n_{batch}	batch size
$n_{\text{gradient-steps}}$	number of gradient steps
$P_{ss'}^a$	probability of state s' after taking action a in state s
$\Pr\{X\}$	probability of X
p_{start}	start state distribution

$\Pr\{s \xrightarrow{k} x \mid \pi\}$	probability of going from state s to state x in k steps under policy π , with $\Pr\{s \xrightarrow{0} s \mid \pi\} = 1$ and $\Pr\{s \xrightarrow{0} s' \mid \pi\} = 0$
π	policy
$\pi_{\boldsymbol{\theta}}$	policy parametrized by $\boldsymbol{\theta}$
$\pi_{\boldsymbol{\theta}_{\text{target}}}$	target policy parametrized by $\boldsymbol{\theta}_{\text{target}}$
π^*	optimal policy
Q^π	state-action value function of policy π , superscript is omitted if clear from context
$\hat{Q}_{\mathbf{w}}$	approximate value function with parameters $\mathbf{w}$
R_t	return, accumulated discounted rewards after time t
$R_{ss'}^a$	expected (immediate) reward after taking action a in state s and transitioning to state s'
R_s^a	expected (immediate) reward after taking action a in state s
RHS	right hand side
r_t	immediate reward at time step t
$\mathcal{R}_p$	risk
$\mathcal{R}_{\mathcal{D}}$	empirical risk
ρ	smoothing coefficient
S	set of states
s, s_t	state, state at time step t
T	length of a finite episode
T_{update}	update delay
$T_{\text{target-update}}$	update delay for target actors or target critics
t	discrete time step
$\boldsymbol{\theta}, \boldsymbol{\theta}_{\text{target}}$	parameters of policies
V^π	state-value function of policy π , superscript is omitted if clear from context
V^*	value function of optimal policy
$\hat{V}_{\mathbf{w}}$	approximate value function with parameters $\mathbf{w}$
$\mathbf{w}, \mathbf{w}_{\text{target}}$	parameters of function approximators

List of Algorithms

1	Tabular SARSA algorithm	18
2	Tabular Q -learning	20
3	AdamW algorithm	32
4	Cross-entropy method	49
5	CMA-ES	55
6	Simple deep Q -learning with experience replay	66
7	Q -learning with gradient accumulation	68
8	Deep Q -learning (DQN)	69
9	Double deep Q -learning (DDQN)	72
10	REINFORCE	86
11	REINFORCE with baseline	90
12	Simple actor-critic with TD learning	91
13	Advantage Actor-Critic (A^2C)	96
14	Deep Deterministic Policy Gradient (DDPG)	97
15	Twin Delayed DDPG (TD3)	101
16	Soft Actor Critic (SAC)	107
17	Proximal Policy Optimization (PPO)	111

Contents

1	Introduction	3
2	Background	7
2.1	Reinforcement learning	7
2.1.1	Markov decision processes	8
2.1.2	Value functions	11
2.1.3	Exploration and exploitation	15
2.1.4	Temporal difference learning	15
2.1.5	Types of RL algorithms	21
2.2	Deep learning	23
2.2.1	Feedforward neural networks	23
2.2.2	Loss functions and encoding	26
2.2.3	Gradient-based optimization	31
2.2.4	Convolutional neural networks	33
2.2.5	Recurrent neural network models	37
2.3	Monte Carlo methods	42
2.3.1	Importance weighting	42
2.3.2	Control variates	43
2.3.3	Differentiating expectations	44
3	Direct policy search	47
3.1	Direct search	47
3.2	Cross-entropy method	48
3.3	Covariance matrix adaption evolution strategy	51
3.4	Direct search for reinforcement learning	57
4	Deep Q-learning	59
4.1	Function approximation	59
4.2	Gradient-based adaptation of value functions	62
4.3	Dealing with terminal states	64
4.4	Gradient accumulation and mini-batch updates	65

4.5	Experience replay	65
4.6	Delayed Q -update	67
4.7	Double Q -learning	70
4.8	Making the loss function more robust	73
4.9	Rainbow	74
5	Policy gradient methods	77
5.1	Gradient-based policy optimization	77
5.2	Differentiable policies	78
5.3	Policy gradient theorems	80
5.4	REINFORCE algorithm	86
5.5	Adding a baseline	87
5.6	Policy gradients in the wild	87
6	Actor-critic methods	89
6.1	Basic actor-critic methods	89
6.1.1	REINFORCE with value function baseline	90
6.1.2	Simple actor-critic with TD learning	91
6.1.3	Actor-critic neural networks	92
6.2	Advantage Actor-Critic	93
6.2.1	Advantage estimation	93
6.2.2	Advantage Actor-Critic	95
6.3	Deep Deterministic Policy Gradient	96
6.4	Twin Delayed Deep Deterministic Policy Gradients (TD3)	99
6.5	Regularization of stochastic actors	100
6.5.1	Entropy bonus	100
6.5.2	KL regularization	104
6.6	Soft Actor Critic	105
6.7	Proximal Policy Optimization	108
6.7.1	CPI loss function	108
6.7.2	PPO clipping	109
6.7.3	PPO bells and whistles	112
6.7.4	Group Relative Policy Optimization	114
7	Summary	115
	Index	121
	Bibliography	132

Chapter 1

Introduction

Deep reinforcement learning combines deep neural networks with reinforcement learning (RL) to solve complex decision-making tasks. In RL, an agent perceives the state it is in and interacts with an environment trying to maximize the rewards it receives as consequences from its actions. The RL framework is very general. The agent can be a robot learning to perform a physical task as efficiently as possible, a game AI learning to improve its scores, a large language model optimizing its answers, or software optimizing the layout of a computer chip.

Reinforcement learning agents build statistical models based on their experience. These models can be a critic which evaluates a state or an action in a state. Or, they can be an actor mapping a state to a probability distribution over actions that define the agent’s behavior. Neural networks can be used as actor, critic, or both – and then we talk of deep RL.

Short history of deep RL

In their seminal textbook, Turing Award laureates Barto and Sutton review the history of RL [Sutton and Barto, 2018]. The combination of RL and models of neural computation can be traced back to Minsky [1952], who proposed the Stochastic Neural Analog Reinforcement Calculator (SNARC); and the most influential early paper may arguably be “Neuronlike adaptive elements that can solve difficult learning control problems” by Barto et al. [1983]. The first big success of deep RL was TD-Gammon, a system that learned to play backgammon, a board game with approximately 1020 states, at the level of a human player [Tesauro, 1991, Tesauro et al., 1995]. This was achieved by self-play, that is, the agent learned by playing against itself. Around that time, Bertsekas and Tsitsiklis [1996] coined the term neurodynamic programming for approximate dynamic programming using neural networks. As RL can be viewed as approximate dynamic programming, neurodynamic programming is basically an early term for deep RL. Despite successful examples, using neural networks as models in RL was considered as rather difficult be-

cause theoretical convergence guaranties were (and still are) missing and the training of the models was difficult to stabilize. But with the success of deep learning in the 2010's [LeCun et al., 2015], due to algorithmic improvements but nonetheless due to the increase in available training data and compute resources, deep RL also started to take off.

A breakthrough in deep RL was the work of Mnih et al. [2015], who showed that deep RL can reach human-level performance in playing classic Atari video games (from the Arcade Learning Environment, ALE, Bellemare et al., 2013). The next milestone was AlphaGo [Silver et al., 2016]. AlphaGo became the first program to defeat a world champion in the game of go. The game allows for about 2×10^{170} legal positions, and mastering it had been considered a moonshot problem for AI. AlphaGo used a combination of supervised learning from human games and RL from self-play, employing deep neural networks for policy and value estimation. AlphaZero [Silver et al., 2017] generalized the AlphaGo approach to playing chess, go, and shogi without human data; it learned solely through self-play using deep RL. Then MuZero [Schrittwieser et al., 2020] extended AlphaZero by learning to play games from raw observations without knowing the rules. It used a model-based RL approach, predicting future states and rewards directly from pixel inputs. This can be regarded as an example of achieving (super)human performance from sensory data without a priori special knowledge about the domain.

In video-game playing, the deep RL agent AlphaStar achieved the grand-master level in StarCraft II, a real-time strategy game with imperfect information and multi-agent dynamics [Vinyals et al., 2019]. And OpenAI Five, a team of five RL agents, defeated professional human players in Dota 2, a multiplayer online battle arena game. OpenAI Five as described by Berner et al. [2019] uses policy gradient methods and self-play, with a network having approximately 158 million parameters.

But deep RL is not just play, it has also achieved impressive results in other domains, solving real-world tasks. A common application area of RL is robotics, and real-world successes are nicely summarized in the review article by Tang et al. [2025]. Deep RL has been successfully applied to the design of the physical layout of a computer chip [Mirhoseini et al., 2021]. AlphaCode is a deep learning system that can generate and refine code. It has been reported to achieve performance comparable to that of human competitors in coding competitions [Li et al., 2022]. AlphaCode uses an RL-like approach to guide learning toward the most promising solutions to a problem.

Large language models (LLMs) are at the time of writing arguably the biggest topic in artificial intelligence (AI) – and deep RL is used in state-of-the-art LLMs to fine tune their answers [Ouyang et al., 2022, Shao et al., 2024].

Why do we need deep neural networks in RL?

Deep neural networks can be used to model actors and critics in RL. But why are they a popular choice? First, most serious applications of RL require models that can compress information and generalize. The state spaces in RL tasks can be huge, for example, if the states are related to observation of continuous quantities, but also in games, where the state space is discrete but suffers from combinatorial explosion. Thus, it is infeasible to store the best action for each state (or, even worse, the expected rewards for each state-action pair) in a look-up table, we need models such as neural networks that can compress the gathered information. However, even if storing a huge state space is no problem, the RL agent does not have the resources to visit all relevant states, not to mention to test all reasonable actions in all relevant states. Thus, we need agents that learn from limited data and generalize, that is, learn about situations not observed during training (i.e., not experienced from interacting with the environment). If the action space corresponds to continuous signals or is very large for other reasons, the agent's behavior is best represented by a function mapping the observed state to the action. For example, if an RL agent experienced what to do if, say, the temperature is 20°C and what to do if it is 20.2°C, then a function approximator such as a neural network can interpolate what to do at 20.1°C. Highly expressive models such as deep neural networks allow us to learn non-linear patterns, to represent multi-modal functions, and even to approximate discontinuities (e.g., a phase transition occurring at a certain temperature). Agents often perceive the environment through images, audio, text, etc., and modern deep learning architectures have proven to be able to learn good task-dependent representations of such inputs that support generalization. Deep RL can learn both the representations of complex input signals and the decision making on top of these representations. For example, a deep RL agent can learn which frames observed in a video game require similar actions.

Neural networks allow us to incorporate prior knowledge about the RL task in their architecture. For example, the structure of convolutional neural networks supports generalization when a state space is spatially structured or is of sequential nature. Recurrent neural network models give the agent memory. They allow the agent to use important information from the past to disambiguate the current state and to coordinate behavior over time.

Finally, for RL to work on real-world tasks, scalable function approximation is typically needed, and deep neural networks have proven to be applicable to large-scale learning problems, allowing efficient iterative gradient-based optimization and fast inference.

Overview of this text

This text is organized into five chapters. The first chapter reviews background knowledge in RL, deep learning, and the crude Monte Carlo used in the later

chapters. It cannot replace proper introductions to RL and deep learning (as provided by many existing excellent textbooks, e.g., [Bishop and Bishop, 2023](#), [Mannor et al., 2026](#)), it serves as a reference for concepts referred to in the chapters on deep RL. Consequently, the selection of topics is very specific. This chapter can be skipped by readers familiar with deep learning and basic RL.

Chapter 3 considers direct policy search methods, actor-only approaches in which gradient-free optimization methods are applied to RL. As the optimization algorithms are generic, they do not take advantage of the underlying structure of the RL task. The material in this chapter is not required for the core material in the chapters 4 to 6.

Chapter 4 introduces deep Q -learning algorithms, in which deep neural networks are used as critics. The chapter explains how critics can be trained via gradient-based optimization and what can be done to stabilize the learning process. Chapter 5 provides the theoretical foundations for estimating gradients to learn actors. Chapter 6 brings together the concepts from chapters 4 and 5. It presents actor-critic deep RL approaches, such as Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO), which mark the state of the art in applied RL.

Chapter 2

Background

Abstract For deep reinforcement learning (RL), you need to know the basics of RL and deep learning. These will be briefly summarized in this chapter with a focus on concepts that will be used later. The introductions are mainly meant to refresh the readers memory of the topics and to make the whole text more self-contained – they are no replacements for proper textbooks on RL and deep learning, respectively. Finally, the chapter discussed the crude Monte Carlo method for estimating expectations and gradients of expectations.

2.1 Reinforcement learning

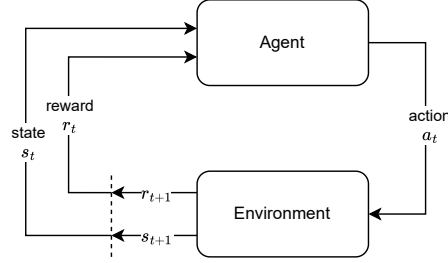
Reinforcement learning (RL) comprises biological and technical concepts to solve a class of problems that can be described as follows. An agent (e.g., an animal, a robot, or just a computer program) interacting with an environment is supposed to find an optimal behavioral strategy while perceiving only limited feedback from the environment. The agent receives (not necessarily complete) information on the current state of the environment, can take actions that can change the state of the environment, and receives reward or punishment signals, reflecting how appropriate the agent’s behavior has been in the past. This reward signal may be sparse, delayed, and noisy. The goal of RL is to find a behavioral strategy, a policy, that maximizes long-term reward. Compared to supervised learning, where training data provide information about the correct behavior in particular situations, the RL problem is more general, and thus more difficult, since learning typically has to be based on considerably less knowledge.

This text is not intended to be an introduction to RL; we refer to the seminal textbook by [Sutton and Barto \[2018\]](#) or the more concise and mathematical rigorous introduction by [Szepesvári \[2022\]](#). However, this section briefly introduces the basic concepts of RL needed in the remainder of this

text (partly based on [Heidrich-Meisner et al. \[2007\]](#) for nostalgic reasons), mainly to fix the notation. The formal description of basic RL tasks are Markov decision processes (MDPs), and most solution methods are rooted in Bellman’s optimality equation. In the following, we introduce these concepts and two basic RL algorithms, SARSA and Q -learning.

2.1.1 Markov decision processes

Fig. 2.1 Markov decision process: At time step t the agent perceives the environment to be in state s_t and takes action a_t . This leads to the reward r_{t+1} and a transition of the environment to state s_{t+1} .



Markov decision processes model time-discrete stochastic state-transition automata. An MDP $\mathcal{M} = \langle S, A, P, R \rangle$ consists of a set of states S , a set of actions A , transition probabilities P , and the expected (immediate) rewards R . The probability that in state s action a takes the agent to state s' is given by $P_{s,s'}^a$. The expected reward received during the transition from state s to state s' by executing the action a is denoted by $R_{s,s'}^a$. We assume discrete time, and at every point in time t , the MDP is in some state s_t . An agent chooses an action $a_t \in A$ that causes a transition from state s_t to some successor state s_{t+1} with probability $P_{s_t,s_{t+1}}^{a_t}$. The agent receives a scalar reward (or punishment) $r_{t+1} \in \mathbb{R}$ for choosing the action a_t when being in state s_t . Note that

$$P_{ss'}^a = \Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\} \quad (2.1)$$

is a probability and

$$R_{ss'}^a = \mathbb{E}[r_{t+1} \mid s_t = s, s_{t+1} = s', a_t = a] \quad (2.2)$$

is an expected value. Sometimes, we marginalize over the state transitions and use $R_s^a = \mathbb{E}[r_{t+1} \mid s_t = s, a_t = a] = \sum_{s' \in S} P_{ss'}^a R_{ss'}^a$.

The Markov property requires that the probabilities of arriving in a state s_{t+1} and receiving a reward r_{t+1} depend only on the state s_t and the action a_t .

They are conditionally independent of previous states, actions, and rewards (i.e., independent of $s_{t'}$, $a_{t'}$, $r_{t'+1}$ for $t' < t$) given a_t and s_t .

Reward time indexing

In our notation based on [Sutton and Barto](#), the agent receives the reward r_{t+1} after taking the action a_t at time step t , that is, the state-action-reward sequence looks like this:

$$s_0, a_0, r_1, s_1, a_1, r_2, \dots \quad (2.3)$$

However, the alternative notation that the reward after taking action a_t at time step t is denoted by r_t is also common, that is, the state-action-reward sequence can alternatively be written as:

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots \quad (2.4)$$

The latter notation is more common in literature rooted in control theory and in particular in online learning. However, I learned from RL from [Sutton and Barto \[2018\]](#) and thus stick to the notion in (2.3).

The agent that interacts with the MDP is modeled in terms of a policy. A deterministic policy $\pi : S \rightarrow A$ is a mapping from the set of states to the set of actions; applying π means always selecting the action $\pi(s)$ in state s . This is a special case of a stochastic policy, which specifies a probability distribution over A for each state s , where $\pi(a | s)$ denotes the probability to choose an action a in state s . We write $\Pr\{s \xrightarrow{k} x | \pi\}$ for the probability of going from state s to state x in k steps under policy π , and we define $\Pr\{s \xrightarrow{0} s | \pi\} = 1$ and $\Pr\{s \xrightarrow{0} s' | \pi\} = 0$ for $s \neq s'$. In the following, policies will often be parametrized; we write π_θ for a policy with parameters θ .

Initially, the agent starts in a start-state s_0 drawn randomly from a start-state distribution p_{start} over S . It is not uncommon (and reasonable) to consider p_{start} as an additional defining element of an MDP and write $\mathcal{M} = \langle S, A, P, R, p_{\text{start}} \rangle$. The start-state distribution can be an important property of an MDP, for example, it is typically sufficient to win a game from all possible starting positions and not necessary to win from all feasible positions.

So far we have implicitly assumed finite MDPs, that is, MDPs with finite state and action spaces. However, the considerations in this text can be generically extended to continuous state and action spaces. The expectations in discrete spaces and the corresponding sums become integrals over probability densities in continuous spaces. Of course, the dimensionality of state and action spaces influence the algorithm choice. But this is due to the size of these spaces and is related to whether there is a meaningful metric indi-

cating similarities between states and/or actions, not depending on whether the space is theoretically infinite. To keep the notation simple, we present the results for the case of discrete S and A .

✚ Infinity, measure theory, etc.

To quote [Hamming \[1970\]](#):

The floating point numbers in a computing machine form a discrete, finite set. As is true in so many applications of mathematics to practical problems, we shall approximate a discrete distribution by a continuous one of sufficient smoothness. [...] The fact that computers are finite and operate at a finite speed for a finite length of time spares us from taking seriously all the confusions that can arise in mathematics when dealing with the infinite.

In this spirit, we ignore any measure theoretical issues, to such an extent that we do not explicitly define the probability spaces underlying the random processes discussed in this text. When going from sums to integrals, we assume that all standard regularity conditions are met. For example, we assume that the conditions for the Leibniz rule for differentiation under the integral sign as well as for Fubini's theorem are fulfilled. That is, we assume that integrals and derivatives are interchangeable and that we can compute a double integral by using an iterated integral.

The learning goal

The goal of RL at some time t is to find a policy that maximizes the accumulated sum of rewards over time

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad , \quad (2.5)$$

where $\gamma \in [0, 1]$ is called the discount factor and determines how strongly immediate rewards are weighted compared to rewards in the future. A discount factor $\gamma < 1$ guarantees that R_t is always a finite number if the immediate reward is bounded. The quantity R_t is called the return or the long-term cumulative reward; if $\gamma < 1$, we refer to a discounted return and a discounted task, otherwise the return and the task are called undiscounted.

Since the state transition process is random, the actual observed accumulated (discounted) reward $\sum_{t=0}^{\infty} \gamma^t r_{t+1}$ might be different from the expected return $\mathbb{E}^{\pi}[R]$ that the agent gets on average when applying policy π starting from some initial state $s_0 \sim p_{\text{start}}$.

Partially observable Markov decision processes

Partially observable Markov decision processes (POMDP) are a generalization of MDPs [[Kaelbling et al., 1998](#)]. In a POMDP, the environment is

determined by an MDP, but the agent cannot directly observe the state of the MDP. Formally, a POMDP can be described by a tuple $\langle S, A, P, R, \Omega, O \rangle$. The first four elements define the underlying MDP, Ω is the set of observations an agent can perceive, and the observation function O maps a state and the action that resulted in this state to a probability distribution over observations (i.e., $O(s', a)(o)$ is the probability of observing $o \in \Omega$ given that the agent took action a and landed in state s').

2.1.2 Value functions

The return (2.5) has the recursive property

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} = r_{t+1} + \gamma \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k+1} = r_{t+1} + \gamma R_{t+1} \quad (2.6)$$

Its expectation conditioned on the current state s and the policy π is called the value V^π of state s :

$$V^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right] \quad (2.7)$$

Plugging in (2.6) and using

$$\mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_{t+1} \mid s_1 = s' \right] = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s' \right] = V^\pi(s') \quad (2.8)$$

gives the the fundamental property

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} P_{s,s'}^a (R_{ss'}^a + \gamma V^\pi(s')) \quad (2.9)$$

for all $s \in S$. This fixed point equation for V^π is known as Bellman equation. In addition to this definition of value functions by infinite sums of expected future rewards, it is also possible to define value functions based on the average future reward as we will discuss later.

For finite S and A , equation (2.9) defines a system of $|S|$ linear equations, which can be solved to determine V^π given π : Let $S = \{1, \dots, n\}$ and $\mathbf{v}^\pi \in \mathbb{R}^n$ have the components $[\mathbf{v}^\pi]_i = V^\pi(i)$. Let $\mathbf{P}^\pi \in \mathbb{R}^{n \times n}$ be the matrix with components $[\mathbf{P}^\pi]_{i,j} = \sum_{a \in A} P_{ij}^a \pi(a \mid i)$. That is, $[\mathbf{P}^\pi]_{i,j}$ is the probability to go from state i to state j when following policy π . Similarly, let $\mathbf{r}^\pi \in \mathbb{R}^n$ with components $[\mathbf{r}^\pi]_i = \sum_{a \in A} \sum_{j \in S} P_{ij}^a \pi(a \mid i) R_{ij}^a$ be the expected one-step reward (i.e., the expected immediate reward) when being in state i and following policy π . With these definitions and assuming $\gamma < 1$, we can write (2.9) as

$$\mathbf{v}^\pi = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}^\pi \quad (2.10)$$

and solve

$$\mathbf{v}^\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi. \quad (2.11)$$

The matrix $(\mathbf{I} - \gamma \mathbf{P}^\pi)$ for $0 \leq \gamma < 1$ is invertible, because $\mathbf{P}^\pi$ is a stochastic matrix, which implies that all its eigenvalues λ_i have a magnitude upper bounded by one. Thus, the eigenvalues of $(\mathbf{I} - \gamma \mathbf{P}^\pi)$, which are given by $1 - \gamma \lambda_i$, are all strictly larger than zero, which means that the matrix is non-singular (see also [Mannor et al. \[2026, Lemma 6.2\]](#)).

Similar to the value function V^π that corresponds to the value (the expected return) of a state when following π , we define the state-value function Q^π :

$$Q^\pi(s, a) = \mathbb{E}^\pi \{r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a\} \quad (2.12)$$

For a given policy π , the Q -function models the expected accumulated reward if we play action a in state s and follow π afterward. The counterpart of equation reads

$$Q^\pi(s, a) = \sum_{s' \in S} P_{s,s'}^a (R_{ss'}^a + \gamma V^\pi(s')) \quad (2.13)$$

for all $s \in S$ and $a \in A$.

Expectations

The expression $\mathbb{E}^\pi[R]$ needs further discussion; it is typical for the (common) notation used in the following. What does the expectation refer to? R is a random quantity. What are the random quantities on which it depends? First, it depends on the start-state, which is distributed according to p_{start} . All actions in a state depend on the policy π , which could be a stochastic policy, in which case $a_t \sim \pi(\cdot \mid s_t)$. Then, the s_{t+1} for $t > 0$ are randomly distributed according to $P_{s_t s_{t+1}}^{a_t}$, and the rewards r_{t+1} can also be random variables depending on s_t , s_{t+1} and a_t . If not stated otherwise, the expectation is over all these random quantities. Now the question arises of why one should write $\mathbb{E}^\pi[R]$ instead of simply $\mathbb{E}[R_0]$. The reason is to stress the dependence on π , for example, to distinguish between expectations with respect to different policies. If we write $\mathbb{E}^\pi[R_0]$, this implies that the start state is drawn from some p_{start} . If we write $\mathbb{E}^\pi[R_0 \mid s_0 = s]$, this means that the agent follows policy π and the start state is fixed to s .

The value functions induce a partial order on the set of policies, namely $\pi_1 \geq \pi_2$ if $V^{\pi_1}(s) \geq V^{\pi_2}(s)$ for all states $s \in S$. Since we are interested in learning a policy that accumulates as much reward as possible, we are interested in finding a policy π^* that outperforms all other policies (i.e., $\pi^* \geq \pi$ for all policies π). For bounded rewards and finite S and A , it has

been shown that such an optimal policy always exists for episodic tasks and for infinite-horizon tasks when $\gamma < 1$ [Puterman, 2005], however, an optimal policy need not be unique. All optimal policies of an MDP share the same value function V^* and include at least one deterministic policy. For an optimal policy π^* and all $s \in S$, we have

$$V^{\pi^*}(s) = \max_a Q^{\pi^*}(s, a) \quad (2.14)$$

$$= \max_a \mathbb{E}^{\pi^*} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right] \quad (2.15)$$

$$= \max_a \mathbb{E}^{\pi^*} \left[r_1 + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r_{t+1} \mid s_0 = s, a_0 = a \right] \quad (2.16)$$

$$= \max_a \mathbb{E}^{\pi^*} \left[r_1 + \gamma V^{\pi^*}(s_{t+1}) \mid s_0 = s, a_0 = a \right] \quad (2.17)$$

$$= \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^{\pi^*}(s')] \quad (2.18)$$

the Bellman optimality equation for a state value function. The corresponding Bellman optimality equation for a state-value function is given in eq. (2.29).

Episodic and infinite-horizon tasks

Typically, the interaction between an agent and the environment can be split into finite subsequences, the so called episodes. Often, this split is natural, and each episode starts from a set of start states and ends after a finite number of interactions in a terminal state. In this case, the RL task and the corresponding MDP are called episodic. Terminal states are also called goal states. The start states and the terminal states S_{goal} are subsets of S . For example, in a game of chess, a terminal state would be a win, loss, or draw. The length T of the subsequence

$$s_0, a_0, r_1, s_1, a_1, r_2, \dots, a_{T-1}, r_T, s_T \quad (2.19)$$

can differ between episodes, in this case T is viewed as a random variable. Throughout this text, we assume that T is finite with probability one, a property also called almost-sure termination. Any action in a terminal state leads to zero reward and no change in state (i.e., for $t > T$: $r_t = 0$ and $s_t = s_T$). Thus, we have:

$$V^{\pi}(s) = \begin{cases} 0 & \text{if } s \in S_{\text{goal}} \\ \sum_{a \in A} \pi(a|s) \sum_{s' \in S \setminus S_{\text{goal}}} P_{s,s'}^a (R_{ss'}^a + \gamma V^{\pi}(s')) & \text{otherwise} \end{cases} \quad (2.20)$$

If there are no terminal states, the task and the corresponding MDP said to have an infinite horizon. In theory, infinite-horizon tasks never end. In

practice, this means that a potential end of the task is so far in the future that it is not modeled. As an example, consider a service robot that is continuously running. It may make a difference in the theoretical considerations whether a task is episodic or has an infinite horizon. However, as we will see later, in practice we will address infinite-horizon tasks or tasks with very long episodes by artificially breaking the interaction into subsequences, typically of the same a priori defined length T .

Policy improvement

Let us assume a deterministic policy π . For a given state s , when would it be better to take an action $a \neq \pi(s)$? It is only better to switch to action a for state s if $Q^\pi(s, a) > V^\pi(s)$, as formalized in the policy improvement theorem [Sutton and Barto, 2018]:

Theorem 2.1 (policy improvement theorem) *Given two policies $\pi, \pi' : S \rightarrow A$. If $\forall s \in S : Q^\pi(s, \pi'(s)) \geq V^\pi(s)$ and $\exists s \in S : Q^\pi(s, \pi'(s)) > V^\pi(s)$ then for $\forall s \in S : V^{\pi'}(s) \geq V^\pi(s)$ and $\exists s \in S : V^{\pi'}(s) > V^\pi(s)$.*

Proof The result follows from

$$\begin{aligned}
V^\pi(s) &\leq Q^\pi(s, \pi'(s)) \\
&= \mathbb{E} \{ r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = \pi'(s) \} \\
&= \mathbb{E}^{\pi'} \{ r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s \} \\
&\leq \mathbb{E}^{\pi'} \{ r_{t+1} + \gamma Q^\pi(s_{t+1}, \pi'(s_{t+1})) \mid s_t = s \} \\
&= \mathbb{E}^{\pi'} \{ r_{t+1} + \\
&\quad \gamma \mathbb{E} \{ r_{t+2} + \gamma V^\pi(s_{t+2}) \mid s_{t+1}, a_{t+1} = \pi'(s_{t+1}) \} \mid s_t = s \} \\
&= \mathbb{E}^{\pi'} \{ r_{t+1} + \gamma r_{t+2} + \gamma^2 V^\pi(s_{t+2}) \mid s_t = s \} \\
&\dots \\
&\leq \mathbb{E}^{\pi'} \{ r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 V^\pi(s_{t+3}) \mid s_t = s \} \\
&\dots \\
&\leq \mathbb{E}^{\pi'} \{ r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots \mid s_t = s \} \\
&= V^{\pi'}(s)
\end{aligned} \tag{2.21}$$

for all $s \in S$ [Sutton and Barto, 2018]. $\square$

This leads to the following RL strategy. Given V^π , we change the policy for all states to get a new policy π' that is greedy with respect to V^π :

$$\begin{aligned}
\pi'(s) &= \operatorname{argmax}_a Q^\pi(s, a) \\
&= \operatorname{argmax}_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')] \quad (2.22)
\end{aligned}$$

Then $\forall s \in S : V^{\pi'}(s) \geq V^\pi(s)$. From $V^{\pi'} = V^\pi$ we have $\forall s \in S : V^\pi(s) = \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$, and then the Bellman optimality equation (2.18) establishes $V^\pi = V^{\pi'} = V^*$. Thus, we can find an optimal policy iteratively by evaluating the current policy π_t (starting with some random policy π_0), that is, determine V^{π_t} (by solving the system of linear equations defined by (2.1.2), dynamic programming, etc., see [Sutton and Barto, 2018](#)), then improve the behavior by considering the greedy policy π_{t+1} with respect to V^{π_t} . The policy evaluation and policy improvement steps are repeated until $\pi_t = \pi_{t+1}$:

$$\begin{array}{ccccccc}
\pi_0 & \xrightarrow{\text{evaluate}} & V^{\pi_0} & \xrightarrow{\text{improve}} & \pi_1 & \xrightarrow{\text{evaluate}} & V^{\pi_1} \xrightarrow{\text{improve}} \dots \\
& & & & & & \dots \xrightarrow{\text{improve}} \pi^* \xrightarrow{\text{evaluate}} V^{\pi^*} \xrightarrow{\text{improve}} \pi^*
\end{array}$$

This process is called policy iteration.

2.1.3 Exploration and exploitation

In practice, the MDP is typically unknown and the only way to get information about it is by interacting with the environment and observing transitions and rewards. Hence, we need algorithms that estimate the value function and derive a good policy from a set of observable quantities, subsequences of episodes. These algorithms need to balance exploration and exploitation, where the former means trying new actions to discover their effects, and the latter refers to choosing the best-known action to maximize reward over time. The trade-off between the two is crucial in RL because too much exploration wastes resources on suboptimal actions, while too much exploitation can trap the agent in a poor strategy without ever finding better ones.

2.1.4 Temporal difference learning

In the following, we present two RL algorithms, SARSA and Q -learning, which learn better policies by iteratively improving a value function approximation. For now, we assume that the values are simply stored in look-up tables, therefore we refer to tabular SARSA and tabular Q -learning, respec-

tively. SARSA is an on-policy method and Q -learning an off-policy method, a difference that we will briefly discuss at the end of this section.

Temporal difference error

If we only have an approximation $\hat{V}$ of a value function V^π , then the Bellman equation (2.9) need not hold if we replace V^π by $\hat{V}$. The difference between the RHS and the LHS is called the Bellman error:

$$\delta_{\text{BE}}^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P_{s,s'}^a \left(R_{ss'}^a + \gamma \hat{V}(s') \right) - \hat{V}(s) \quad (2.23)$$

One way to evaluate a policy π is to minimize the squared Bellman error $(\delta_{\text{BE}}^\pi(s))^2$ for all $s \in S$ until the Bellman error vanishes and $\hat{V} = V^\pi$.

In the model-free setting, P and R are unknown. Thus, we minimize the squared Bellman error based on observations. We observe transitions in the form of quadruples (s, a, r, s') when the agent takes action a in state s that leads to a transition to s' and reward r . This allows us to compute the temporal difference (TD) error

$$\delta_{\text{TD}}^\pi(s, r, s') = r + \gamma \hat{V}(s') - \hat{V}(s) \quad , \quad (2.24)$$

which is a Monte Carlo approximation of (2.23). For sufficiently small learning rate $\alpha > 0$, the update

$$\hat{V}(s) \leftarrow \hat{V}(s) + \alpha \left(r + \gamma \hat{V}(s') - \hat{V}(s) \right) \quad (2.25)$$

$$= (1 - \alpha) \hat{V}(s) + \alpha \left(r + \gamma \hat{V}(s') \right) \quad (2.26)$$

moves $\hat{V}(s)$ closer to $r + \gamma \hat{V}(s')$ and it leads to a reduction of the squared temporal difference error (see section 4.2). Algorithms based on reduction of some form of TD error perform temporal difference learning [Sutton, 1988].

Generalized policy iteration

Now, we could implement policy iteration as introduced in the previous section as follows. Given a deterministic policy π_i , we sample a random state s , take the action $a = \pi_i(s)$, and observe the reward r and the new state s' . Then we update $\hat{V}(s)$ following (2.25). If we do this infinitely often and select every state infinitely often (and reduce α over time in an appropriate way), $\hat{V}$ will converge to V^{π_i} . Then we can improve the behavior by considering the greedy policy w.r.t. V^{π_i} and repeat the process until we get close enough to an optimal policy.

This approach is not practical for several reasons. First, of course, optimizing $\hat{V}$ to closely match the current policy before updating the behavior is extremely inefficient. Thus, in practice, we only update $\hat{V}$ a little to bet-

ter match the current policy and consider the new policy defined based on the updated $\hat{V}$ directly afterwards. This is called generalized policy iteration [Sutton and Barto, 2018]. In the extreme case, we perform an update step (2.25) for each sampled (s, a, r, s') and always consider a policy based on the most recent $\hat{V}$.

Second, starting the interaction from an arbitrary state is inefficient and may even not be possible. We would like to focus on relevant states, those which are actually visited by well-performing agents. When doing so, we gather experience from real episodes, starting according to p_{start} and performing the update (2.25) for the observed $(s_t, a_t, r_{t+1}, s_{t+1})$ tuples in each episode. Then we cannot simply consider fully greedy policies, because this would most likely lead to suboptimal behavior. We need to ensure that there is enough exploration, that is, that the agent also tries other actions than the one suggested by the current greedy policy. Exploration is needed to gather enough information about the environment. For example, it could happen that a very good state was evaluated very badly in the beginning of learning because the behavior after being in this state was not yet properly learned, or the agent received a low reward after that state simply because of randomness in the rewards and transitions. If the agent would only exploit this negative experience and never try the state again, it would never experience the high value of that state. Thus, without exploration, the agent will settle on actions based on limited experience that may not be optimal. Therefore, we consider randomized policies derived from $\hat{V}$ with the property $\pi(a|s) \geq \epsilon$ with $\epsilon > 0$ for all s and a , so called ϵ -soft policies. The most popular ϵ -soft policy is the ϵ -greedy policy. The ϵ -greedy follows the greedy policy but takes a random action with a probability of ϵ at every step. That is, the probability for exploitation (i.e., taking the greedy action) is $1 - \epsilon + \frac{\epsilon}{|A|}$ and the probability for exploration any other action is $\frac{\epsilon}{|A|}$. Exploration can be adjusted during learning, in particular by reducing ϵ over time (see example 4.3 in section 4.6). The reason for reducing exploration over time is that we expect the RL agent to become more and more confident about what it learned about the environment and would like the policy to converge to optimal behavior.

Third, defining a policy on top of a state-value function V still requires some knowledge of the underlying process, since for all states s the possible successor states s' must be known. This is not necessary if we estimate the Q -function. Thus, we consider an update for an approximated state-action value function

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right) \quad (2.27)$$

$$= (1 - \alpha) \hat{Q}(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}) \right) \quad (2.28)$$

based on the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$, where we added the action taken in the state s_{t+1} .

The TD update performs bootstrapping, that is, it updates value estimates based on other learned value estimates, rather than waiting for a full return. This allows the algorithms to learn online, that is, to change values function estimates and the policy during an episode.

The TD error is the difference between the estimation of a value of at time t and at time $t + 1$, in (2.30) between $\hat{Q}(s_t, a_t)$ and $r_{t+1} + \gamma\hat{Q}(s_{t+1}, a_{t+1})$, which explains the name temporal difference error [Sutton, 1988]. The TD update moves the estimated value closer to the estimate at time $t + 1$, because the latter incorporates additional feedback from the environment in the form of r_{t+1} and one may argue that the values function estimates can be expected to be more reliable for states closer to a terminal state.

SARSA

If we combine these three ideas, we arrive at the SARSA algorithm, where the name comes from the quintuple defining a single update. The pseudocode is shown in algorithm 1.

Algorithm 1: Tabular SARSA algorithm

Input: learning rate α , discount factor γ , exploration rate ϵ

```

1 Initialize  $\hat{Q}(s, a)$  arbitrarily
2 for each episode do
3   Initialize state  $s \sim p_{\text{start}}$ 
4   Choose action  $a$  using policy derived from  $\hat{Q}$  (e.g.,  $\epsilon$ -greedy)
5   while state  $s$  is not terminal do
6     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
7     Choose action  $a'$  using policy derived from  $\hat{Q}$  (e.g.,  $\epsilon$ -greedy)
8      $\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha[r + \gamma\hat{Q}(s', a') - \hat{Q}(s, a)]$ 
9      $s \leftarrow s'$ 
10     $a \leftarrow a'$ 

```

Q-learning

Arguably the most prominent model-free RL algorithm is *Q-learning* [Watkins and Dayan, 1992]. Let Q^* denote the Q -function of an optimal policy, then theorem 2.1 gives $V^*(s) = \max_{a \in A} Q^*(s, a)$ and we have the Bellman optimality equation

$$Q^*(s, a) = \sum_{s' \in S} P_{s,s'}^a \left(R_{ss'}^a + \gamma \max_{a' \in A} Q^*(s', a') \right) \quad (2.29)$$

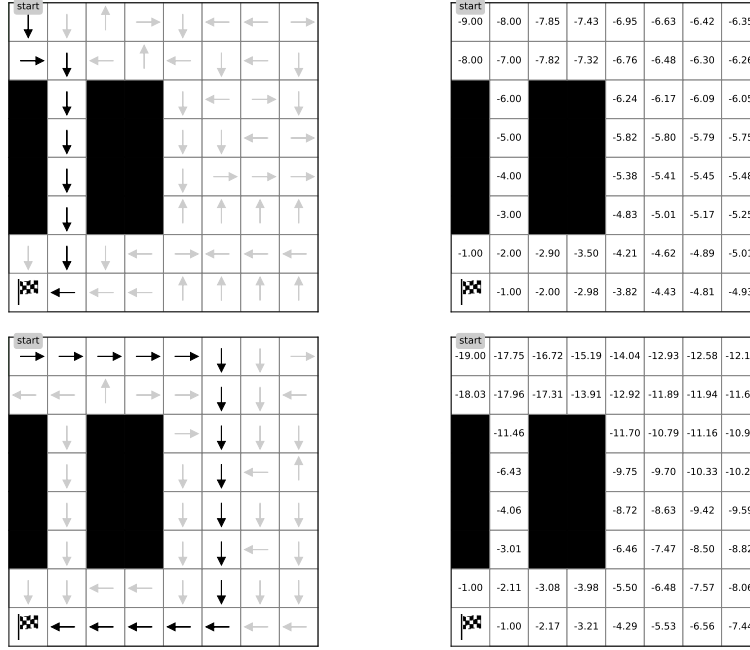


Fig. 2.2 Tabular Q -learning and SARSA in a grid world, see example 2.1. Each step gives a negative reward of minus one, stepping on a black cell costs -100 . The top row shows the greedy policy with respect to the $\hat{Q}$ values learned by Q -learning and the underlying Q values. The row below shows the same for SARSA. The on-policy SARSA learned a value function leading to an optimal ϵ -greedy soft policy. When making random moves, the agent has to keep a safety distance from the black areas, which lead to a large negative reward. In contrast, Q -learning learned a value function leading to an optimal deterministic policy. The learning is based on agent experience, which led to a focus on more relevant states; the value function estimates far from the optimal paths have not (yet) converged to their true values.

for state-action pairs in accordance to (2.18). In the same way we derived (2.24), we can now derive from (2.29) a TD error:

$$\delta_{\text{TD}}^*(s, r, s') = r + \gamma \max_{a' \in A} \hat{Q}(s', a') - \hat{Q}(s, a) \quad (2.30)$$

Given $(s_t, a_t, r_{t+1}, s_{t+1})$, this suggest an update of the Q -function at the observed state-action pair (s_t, a_t) :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_{a' \in A} Q(s_{t+1}, a') - Q(s_t, a_t)) \quad (2.31)$$

Again, $\alpha \geq 0$ is a learning rate that may decreases over time. The resulting algorithm is shown in algorithm 2.

Algorithm 2: Tabular Q -learning

Input: learning rate α , discount factor γ , exploration rate ϵ

```

1 Initialize  $\hat{Q}(s, a)$  arbitrarily
2 for each episode do
3   Initialize state  $s$ 
4   while state  $s$  is not terminal do
5     Choose action  $a$  using policy derived from  $\hat{Q}$  (e.g.,  $\epsilon$ -greedy)
6     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
7      $\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha \left[ r + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s, a) \right]$ 
8      $s \leftarrow s'$ 

```

In the limit of an infinite set of observed transitions, convergence of Q -learning can be guaranteed if all state-action pairs occur infinitely often and some constraints on the decrease of the learning rate α over time are fulfilled (see, e.g., Bertsekas and Tsitsiklis, 1996). If S and A are finite and the task is episodic, all state-action pairs are visited infinitely often if each state-action pair can be sampled as the first state and action in an episode with positive probability. Once we have calculated Q^* , an optimal policy can be greedily derived by selecting the action a in state s that maximizes $Q^*(s, a)$.

Convergence

Rigorously studying the convergence properties of RL algorithms is an important research area, which is beyond the scope of this text. The focus of this text is on deep RL, that is, RL using highly nonlinear function approximators. In this case, all convergence results are constrained by the fact that we can only establish convergence to a local optimum of the models that represent policies and value functions.

Example 2.1 Consider an agent living in a grid-world as illustrated in fig. 2.2. The actions are $A = \{\text{up, down, left, right}\}$. The agent starts in the top left corner and has to reach the lower left corner. Each step gives a negative reward of -1. If the agent accidentally steps on a black grid cell, it receives a negative reward of -100. The top and row in fig. 2.2 shows the greedy policy with respect to the $\hat{Q}$ values learned by Q -learning, the bottom row the corresponding plots for SARSA.

Multi-step temporal difference learning

We have argued that the TD updates improve current estimates because they incorporate additional feedback from the environment in the form of

immediate reward and because values function estimates of subsequent time steps may be more reliable because the states can be expected to be closer to a terminal state. For example, in Q -learning, we move the estimate $\hat{Q}(s_t, a_t)$ closer to $r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1})$. However, with the arguments above, wouldn't $r_{t+1} + \gamma r_{t+2} + \gamma^2 \hat{Q}(s_{t+2}, a_{t+2})$ not be an even more reliable target, as it incorporates even more information in terms of experienced rewards and the bootstrap estimate is further in the future and weighted down for $\gamma < 1$? Indeed, this often gives better results [Sutton and Barto, 2018]. Let us define the general n_R -step return (more precisely, the general corrected n_R -step truncated return) as

$$G_t^{(n_R)}(c) = \sum_{k=0}^{n_R-1} \gamma^k r_{t+k+1} + \gamma^{n_R} c, \quad (2.32)$$

where $c \in \mathbb{R}$, see Sutton and Barto [2018], and n_R is upper bounded by the number of steps observed after time t . In Q -learning, for example, we can then compute the temporal difference between $\hat{Q}(s_t, a_t)$ and $G_t^{(n_R)}(\max_{a'} \hat{Q}(s_{t+n_R}, a'))$. Note the recursive relation

$$G_t^{(n_R)}(c) = r_t + \gamma G_{t+1}^{(n_R)}(c) \quad (2.33)$$

Learning considering several steps before the bootstrap approximation is referred to as multi-step TD learning.

2.1.5 Types of RL algorithms

Reinforcement learning approaches can differ along several dimensions. In this section, we list some important distinctions.

Actors and critics

Reinforcement learning algorithms can be broadly classified into critic-only, actor-only, and actor-critic methods. Critic-only methods include value function approaches as discussed in the following sections. Actor-only methods search directly in a space of policies without relying on value functions (e.g., using a general direct optimization method, see chapter 3). Actor-critic methods learn a value function as well as parameters of a policy. In deep RL, neural networks are used for actors and/or critics.

Model-based and model-free

The algorithms can be further divided into model-based and model-free algorithms, depending on whether the algorithm explicitly learns the transition

probabilities and expected rewards for state-action pairs. Learning the transition probabilities and expected rewards amounts to learning the MDP, that is, the agent tries to learn a model of the world it lives in. Learning the transition probabilities has the advantage that they can be used for different tasks (e.g., when the reward changes over time or there are several agents with different tasks). This text focuses on model-free RL.

On-policy and off-policy learning

The difference between SARSA and Q -learning is small, but decisive. SARSA is an on-policy method where the agent updates its Q -values based on the next action it actually takes, following its current policy (e.g., ϵ -greedy). In on-policy RL the agent learns the value function for the policy it is currently following. The agent evaluates and improves the same policy that it uses to make decisions. This is in contrast to off-policy RL, where the agent learns the value function for a different policy, often the greedy one, than the one it is currently following. It can learn from experiences generated by any policy, including past actions or external data. Q -Learning is off-policy because it updates Q -values based on the best possible action in the next state, not necessarily the action suggested by the current policy. In off-policy RL, it is easier to reuse past experience gathered using a different policy, which is important to reduce the sample complexity, that is, the number of interactions with the environment needed to learn a sufficiently good policy. The difference is exemplified in example 2.1 and fig. 2.2. Here, both SARSA and Q -learning performed ϵ -greedy action selection. The on-policy SARSA learned a value function leading to an optimal ϵ -greedy soft policy, while Q -learning learned a value function leading to an optimal deterministic policy.

Deterministic vs. stochastic policies

Some RL algorithms learn deterministic policies, others stochastic policies. For every finite MDP there exists an optimal deterministic policy [Ross, 1983]. This basically follows from the policy improvement theorem theorem 2.1 (we define some strict total order on A , and if actions in a state have the same value, a deterministic algorithm breaks the tie based on the order). So why should we at all consider stochastic policies? There are settings where stochastic policies outperform deterministic ones. Singh et al. [1994] showed that in a POMDP a stochastic policy can be arbitrarily better than the best deterministic policy. In adversarial environments, adversaries can exploit deterministic policies more easily than stochastic policies. The standard example is rock–paper–scissors; clearly, playing the game with a deterministic strategy is not a good idea.

Stochastic policies can model exploration directly. When learning a deterministic policy, we need a separate mechanism for exploration, for example, random perturbations of the policy.

2.2 Deep learning

Deep learning is a subfield of machine learning; the term is typically used synonymously with neural networks [LeCun et al., 2015, Schmidhuber, 2015]. An introduction to deep learning is beyond the scope of this text; we refer to a deep learning textbook and recommend the one by Bishop and Bishop [2023]. This section provides a brief overview of deep learning concepts that are particularly relevant for RL, mainly to establish the terminology for later chapters.

Neural networks (NNs) are used in computational neuroscience to model biological processes to gain insight into biological information processing. This makes them the natural model class if we study RL in living organisms. However, this text focuses on RL for machine learning, and in this context the term NN refers to a broad class of statistical models – loosely inspired by computational neuroscience – that are considered because they solve some technical problems better than other models.

2.2.1 Feedforward neural networks

A neural network (NN) is a statistical model defined by a directed graph, where the nodes are simple processing units (neurons) that compute numerical values, and the edges indicate how these values are communicated between the units.

Feedforward and recurrent NNs can be distinguished. Let us start with feedforward NNs. Feedforward NNs represent static functions. If we assume D -dimensional real-valued inputs and K -dimensional real-valued outputs, a feedforward NN is a model $f_{\mathbf{w}} : \mathbb{R}^D \rightarrow \mathbb{R}^K$ parameterized by real-valued parameters $\mathbf{w}$. These parameters are referred to as weights. For given weights, the output of the network depends only on the input. Learning or training an NN refers to adapting the weights.

Let us start by defining what a simple single neuron computes. We assume that the inputs $x_1, \dots, x_D$ are collected in the input vector $\mathbf{x} \in \mathbb{R}^D$. The output of the neuron i is denoted by $z_i(\mathbf{x})$. Often we omit the dependency on $\mathbf{x}$ to keep the notation uncluttered. A generic neuron first computes the weighted sum of the input,

$$a_i = \sum_{j=1}^D w_{ij}x_j + w_{i0} \ , \quad (2.34)$$

with weights $w_{1i}, \dots, w_{Di}, \in \mathbb{R}$ and bias (threshold, offset) parameter $w_{i0} \in \mathbb{R}$. The value of this affine linear function is referred to as potential or pre-activation. This computation corresponds to the integration of incoming sig-

nals in a biological neuron. Then a transfer function (activation function) h_i is applied to the pre-activation,

$$z_i = h_i(a_i) = h_i \left(\sum_{j=1}^D w_{ij} x_j + w_{i0} \right) , \quad (2.35)$$

which corresponds to the “firing” of biological neurons.

There are many choices for the activation function; popular activation functions include:

Linear:

$$h(a) = a \quad (2.36)$$

Alternative sigmoid:

$$h(a) = \frac{a}{1 + |a|} \quad (2.40)$$

Step / threshold:

$$h(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{otherwise} \end{cases} \quad (2.37)$$

Rectified linear unit (ReLU):

$$h(a) = \max(0, a) \quad (2.41)$$

Fermi / logistic:

$$h(a) = \frac{1}{1 + e^{-a}} \quad (2.38)$$

Leaky ReLU ($0 \leq \alpha \ll 1$):

$$h(a) = \max(\alpha a, a) \quad (2.42)$$

Hyperbolic tangent:

$$h(a) = \tanh(a) \quad (2.39)$$

Exponential LU (ELU, $\alpha > 0$):

$$h(a) = \begin{cases} a & a \geq 0 \\ \alpha(e^a - 1) & a < 0 \end{cases} \quad (2.43)$$

Some of these activation functions are illustrated in fig. 2.3.

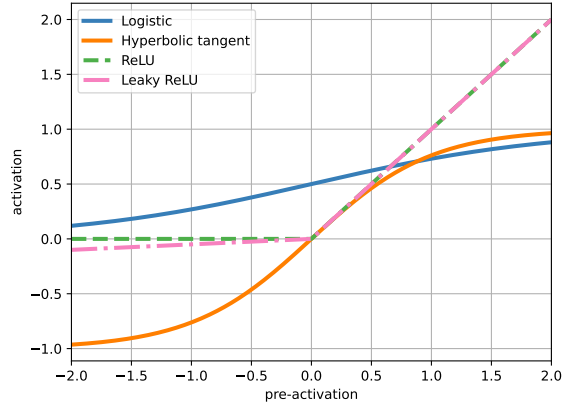


Fig. 2.3 Examples of activation functions

Now, an NN is a set of connected neurons. It can be described by a weighted directed graph, where neurons are the nodes/vertices $\mathcal{V}$, typically numbered by integers, and the connections between neurons are the edges $\mathcal{E}$. The strength of a connection $(j, i) \in \mathcal{E}$ from neuron j to neuron i is described by the weight w_{ij} . All weights are collected in the weight vector $\mathbf{w}$. It is a defining property of feedforward NNs that the underlying graph is acyclic. Some units (input neurons) are set to the components of the external input, and the values of the other units are computed in topological order. We assume that the neuron i can only get input from the neuron j if $j < i$, a condition that ensures that the graph is acyclic. The last K neurons, the output neurons, define the output of the neural network. The neurons $V_{\text{input}} = \{1, \dots, D\}$ are the input neurons. They are simply clamped to the input to the network, $z_i = x_i$ for $i = 1, \dots, D$. All neurons $i > D$ that are not output neurons are called hidden neurons, because they do computations that are not typically not read out from the outside. Sometimes the notation is simplified by assuming an additional neuron indexed by 0 that is connected to all other neurons and has a constant output $z_0 = 1$. This allows to write the output of a neuron $i > D$ as

$$z_i = h_i(a_i) = h \left(\sum_{(i,j) \in \mathcal{E}} w_{ij} z_j \right) . \quad (2.44)$$

Then, for n_{hidden} hidden neurons, we have $\mathcal{V} = \{0, 1, 2, \dots, D + n_{\text{hidden}} + K\}$ and the indices of the hidden and output neurons are $\mathcal{V}_{\text{hidden}} = \{D + 1, \dots, D + n_{\text{hidden}}\}$ and $\mathcal{V}_{\text{output}} = \{D + n_{\text{hidden}} + 1, \dots, D + n_{\text{hidden}} + K\}$, respectively.

Large deep learning systems are typically better described in terms of layers than in terms of individual neurons. Deep learning system are composed of several non-linear processing modules, and in deep neural networks these correspond to neural network layers. Let us define the concept of a layer in a very applicatory way. Layers collect neurons that get the same input and perform the same type of computation. In a feedforward NN, layer l gets only input from layers $l' < l$. The activation function of neurons in layer l is denoted by $h^{(l)}$. Inputs (external inputs as well as outputs from other neurons) to layer l are gathered in the vector $\mathbf{x}^{(l)}$, the outputs of the neurons in layer l in vector $\mathbf{h}^{(l)}$. If the outputs of layer l are the only inputs to layer $l + 1$, we have $\mathbf{x}^{(l+1)} = \mathbf{h}^{(l)}$. If layer $l + 1$ also receives input from another layer, say the first layer, we stack the inputs, for example

$$\mathbf{x}^{(l+1)} = \begin{bmatrix} \mathbf{h}^{(l)} \\ \mathbf{h}^{(1)} \end{bmatrix} . \quad (2.45)$$

Connections from a layer l' to a layer l with $l - l' > 1$ are called shortcut connections.

Denoting the weights of the i th neuron in layer l by $\mathbf{w}_i^{(l)}$ and its bias by $b_i^{(l)}$ we get $h_i^{(l)}(\mathbf{x}) = h^{(l)}\left(\left[\mathbf{w}_i^{(l)}\right]^T \mathbf{x} + b_i^{(l)}\right)$. We stack the $\mathbf{w}_i^{(l)}$ of all neurons in layer l in matrix $\mathbf{W}^{(l)}$ and their bias parameters in vector $\mathbf{b}^{(l)}$ and write

$$\mathbf{h}^{(l)}(\mathbf{x}^{(l)}) = h^{(l)}\left(\mathbf{W}^{(l)}\mathbf{x}^{(l)} + \mathbf{b}^{(l)}\right), \quad (2.46)$$

where $h^{(l)}$ acts component-wise. Now, assume a batch of inputs $\mathbf{x}_1^{(l)}, \dots, \mathbf{x}_B^{(l)}$ gathered as rows in a data matrix $\mathbf{X}^{(l)}$, the output of the layer can be computed for the full batch as:

$$\mathbf{H}^{(l)}(\mathbf{X}^{(l)}) = h^{(l)}\left(\mathbf{W}^{(l)}\left[\mathbf{X}^{(l)}\right]^T + \mathbf{b}^{(l)}\mathbf{1}^T\right), \quad (2.47)$$

where $\mathbf{1}$ denotes the vector of all ones. Equation (2.47) is great for parallel computing, for example on graphics processing units (GPUs).

Example 2.2 Throughout this text, we will often refer to the LunarLander RL task from the Gymnasium framework [Towers et al., 2024]. The goal is to safely land a spacecraft on the moon. The eight-dimensional state-space S encodes the horizontal and vertical coordinates of the spaceship, its velocities in the two directions, its angle, its angular velocity, and two Boolean observations that signal whether the left or right leg is in contact with the surface. The action space is $A = \{0, 1, 2, 3\}$, encoding 'do nothing', 'fire left orientation engine', 'fire main engine', and 'fire right orientation engine'. The figure on the right shows a policy network for that task $\hat{Q}_{\mathbf{w}} : \mathbb{R}^8 \rightarrow \mathbb{R}^4$, mapping states to actions. The network layers are visualized. The leftmost layer corresponds to the eight inputs. It is followed by a fully-connected layer, a layer where each neuron is connected to all neurons of the previous layer, with 64 units having $h(a) = \tanh(a)$ activation functions. The outputs are then fed into another fully-connected layer with 64 units having $h(a) = \tanh(a)$ activation functions. Finally, there is a fully-connected layer with four output neurons with linear activation functions, which also receives the input (i.e., there is a shortcut connection from the input to the output layer). The activations of the four output neurons are mapped to a probability distribution over the four actions by a softmax activation function (2.58).

2.2.2 Loss functions and encoding

A feed-forward neural network model $f_{\mathbf{w}} : \mathbb{R}^D \rightarrow \mathbb{R}^K$ is defined by the structure of the neural network and its parameters $\mathbf{w}$. These parameters are

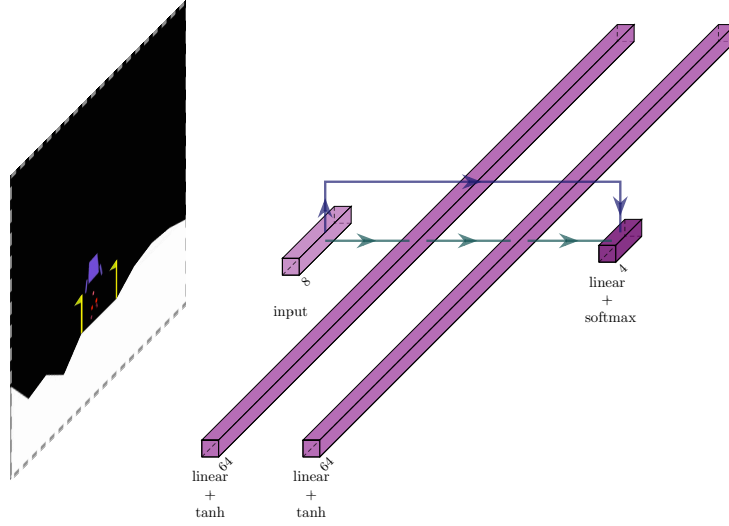


Fig. 2.4 Neural network architecture used in the LunarLander task.

adjusted – learned – to minimize the difference between the network’s output and the actual desired output during the training phase. This difference can be measured using sample data; that is, a collection of inputs and corresponding outputs. After the training phase, the network is usually tested on independent data that were not used during the training phase.

Learning problem

Learning can be defined as goal-directed change of behavior. Let us state this more formally for the standard supervised learning setting. We are given some input space X and an output space Y . In the following, $X = \mathbb{R}^D$ if not stated otherwise. Let p denote some distribution on $X \times Y$. While p is typically unknown, we have access to a sample $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\} = \{(x_i, y_i)\}_{i=1}^n$ drawn from p . For now, we assume that the data are independent and identically distributed (i.i.d.). That is, all pairs are drawn from the same distribution p , and each pair is drawn independently from all other pairs. In RL, the data set contains transitions collected from episodes of the agent interacting with the environment using different policies; thus, in RL the i.i.d. assumption can be violated (which is often simply ignored). We refer to $\mathcal{D}$ as a (training) data set. However, it is not a set in the mathematical sense. It can contain duplicate entries (i.e., it is a rather multi-set) and is best viewed as a sequence.

The goal of supervised learning is to uncover the unknown relation between inputs and outputs defined by p . For example, the goal may be to predict the most likely output $y \in Y$ for inputs $x \in X$. We describe the relation between inputs and outputs by a model or hypothesis, formally defined by a function $f : X \rightarrow Y$ that maps inputs to outputs; a hypothesis class $\mathcal{H}$ is a set of such functions. In the standard supervised learning setting, we want to find a hypothesis h that maps an input x to a good prediction $\hat{y} \in Y$. The quality of a prediction is measured by a task-dependent loss function. A function $\ell : Y \times Y \rightarrow [0, \infty[$ with the property $\ell(y, y) = 0$ for $y \in Y$ is called a (pointwise) loss function. For example, a typical loss for classification is the 0-1 loss, $\ell(y, \hat{y}) = \begin{cases} 0 & \text{if } y = \hat{y} \\ 1 & \text{otherwise} \end{cases}$. A good hypothesis must show good average performance on patterns $(x, y) \sim p$ (e.g., it need not make good predictions for x which never occur in practice). Thus, the goal of supervised statistical learning can be defined as finding h minimizing the risk:

$$\mathcal{R}_p(f) = \mathbb{E}_{(x,y) \sim p}[\ell(y, f(x))] \quad (2.48)$$

given sample data $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$. Thus, applying a learning algorithm means coming up with a hypothesis given sample data; formally, learning algorithm maps from $\{((x_1, y_1), \dots, (x_n, y_n)) \mid 1 \leq i \leq n < \infty, x_i \in X, y_i \in Y\}$ to $\mathcal{H}$.

The empirical risk of h on sample data $\mathcal{D}$ ($|\mathcal{D}| = n$) is defined as:

$$\mathcal{R}_{\mathcal{D}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)) \quad (2.49)$$

Minimizing empirical risk does not imply good generalization, that is, good performance on patterns not seen during training). There is the risk of overfitting, which occurs if a model faithfully reflects idiosyncrasies of the training data rather than the underlying distribution.

In this section, we briefly discuss how outputs are encoded and how the difference to the target is measured depending on the task.

Regression

Let us first consider the task of regression, where targets are real-valued vectors. For example, in an RL task with a finite set of actions $A = \{a_1, \dots, a_K\}$, we may want to map a state $s \in S = \mathbb{R}^D$ to the state-action values $(\hat{Q}(s, a_1), \dots, \hat{Q}(s, a_K))^T$. Given the data $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$, $\mathbf{x}_i \in \mathbb{R}^D$, $\mathbf{y}_i \in \mathbb{R}^K$, $1 \leq i \leq N$, a natural choice for measuring how well the model fits the data is the mean-squared error (MSE) defined by

$$E_{\mathcal{D}}(f_{\mathbf{w}}) = \frac{1}{N} \sum_{n=1}^N \|f_{\mathbf{w}}(\mathbf{x}_n) - \mathbf{y}_n\|^2 = \frac{1}{N} \sum_{n=1}^N \sum_{i=1}^K ([f_{\mathbf{w}}(\mathbf{x}_n)]_i - [\mathbf{y}_n]_i)^2 \quad (2.50)$$

Usually linear output neurons are used, that is, $h_i(a) = a$ for $i \in \mathcal{V}_{\text{output}}$. We refer to the objective function $E_{\mathcal{D}}$ defining a learning problem as the error function. In (2.50), the error function $E_{\mathcal{D}}$ equals the empirical risk with loss $\ell(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|^2$, that is, the learning corresponds to empirical risk minimization.

The mean-squared error is a natural choice as an error function for regression because it corresponds to maximizing the likelihood of the parameters $\mathbf{w}$, that is, the probability of observing $\mathcal{D}$ when the model has parameters $\mathbf{w}$, under the assumption of Gaussian noise. W.l.o.g. let $D = 1$ and $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$. We assume that the observations y given an input $\mathbf{x}$ are normally distributed (with variance s^2) around an underlying deterministic function $f_{\mathbf{w}}$:

$$p_{\mathbf{w}}(y|\mathbf{x}) = \frac{1}{s\sqrt{2\pi}} \exp \frac{-(y - f_{\mathbf{w}}(\mathbf{x}))^2}{2s^2}, \quad (2.51)$$

which gives likelihood and negative log-likelihood:

$$p(\mathcal{D} | \mathbf{w}) = \prod_{n=1}^N \frac{1}{s\sqrt{2\pi}} \exp \frac{-(y_n - f_{\mathbf{w}}(\mathbf{x}_n))^2}{2s^2} \Rightarrow \quad (2.52)$$

$$-\ln p(\mathcal{D} | \mathbf{w}) = \underbrace{\frac{1}{2s^2}}_{\text{independent of } \mathbf{w}} \sum_{n=1}^N (y_n - f_{\mathbf{w}}(\mathbf{x}_n))^2 + \underbrace{N \ln(s\sqrt{2\pi})}_{\text{independent of } \mathbf{w}} \quad (2.53)$$

Thus minimizing the sum-of-squares error corresponds to maximum likelihood estimation under the Gaussian noise assumption. The Gaussian noise assumption can be justified by the central limit theorem (CLT) and the maximum entropy principle. Roughly speaking, the CLT stated that the distribution of averages of randomly drawn samples, where each observation is drawn independently from the others but not necessarily from the same distribution, approaches a Gaussian distribution. The maximum entropy principle states that the probability distribution which best represents a random variable is the one with largest entropy given the current knowledge, and, roughly speaking, for a real-valued random variable and given mean and variance this is the corresponding Gaussian.

Classification

For classification, we assume that the output of the NN can be mapped to a probability distribution over the finite set Y of classes. For example, mapping a state to one from a discrete set of actions can be viewed as a classification

task. For a neural network model $f_{\mathbf{w}}$, we denote the corresponding probability distribution over the classes given an input $\mathbf{x} \in X$ by $p_{\mathbf{w}}(\cdot | \mathbf{x})$. The predicted class of $f_{\mathbf{w}}$ for an input $\mathbf{x}$ can be determined by $\hat{y} = \arg \max_{y \in Y} p_{\mathbf{w}}(y | \mathbf{x})$.

For binary classification, we assume $Y = \{0, 1\}$ and that the output of the NN model $f_{\mathbf{w}}$ is in $[0, 1]$, which can be achieved by a single output neuron with sigmoid activation function (2.38). The distribution $p_{\mathbf{w}}(\cdot | \mathbf{x})$ is then a Bernoulli distribution:

$$p_{\mathbf{w}}(y | \mathbf{x}) = f_{\mathbf{w}}(\mathbf{x})^y [1 - f_{\mathbf{w}}(\mathbf{x})]^{1-y} = \begin{cases} f(\mathbf{x}) & \text{for } y = 1 \\ 1 - f(\mathbf{x}) & \text{for } y = 0 \end{cases} \quad (2.54)$$

Moving to the negative logarithm of $p(\mathcal{D} | \mathbf{w}) = \prod_{n=1}^N p_{\mathbf{w}}(y_n | \mathbf{x}_n)$ leads to cross-entropy error function:

$$-\ln p_{\mathbf{w}}(\mathcal{D} | \mathbf{w}) = -\sum_{n=1}^N [y_n \ln f_{\mathbf{w}}(\mathbf{x}_n) + (1 - y_n) \ln(1 - f_{\mathbf{w}}(\mathbf{x}_n))] \quad (2.55)$$

For multi-class classification into K classes, we use a one-hot encoding (1 out of K encoding). Class i is encoded by the K -dimensional standard unit vector $\mathbf{e}_i$. That is, the j th component of $\mathbf{y}_i$, denoted by $[\mathbf{y}_i]_j$, is one, if $\mathbf{x}_i$ belongs to the j th class, and zero otherwise. We have $Y = \{\mathbf{e}_1, \dots, \mathbf{e}_K\}$ and For example, if $|Y| = 4$ and $\mathbf{x}_i$ belongs to the third class, then $\mathbf{y}_i = (0, 0, 1, 0)^T$. We do not use a single output ($K = 1$) discretized to K classes, which would introduce an ordering and a notion of distance between classes, which is typically not desired, as the values in Y are not on an interval or ordinal scale.

Let us consider one-hot encoded data $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$, $\mathbf{y}_i \in \mathbb{R}^{|Y|}$. Let $p_{\mathbf{w}}(\mathbf{e}_j | \mathbf{x}) = [f_{\mathbf{w}}(\mathbf{x})]_j \in [0, 1]$ denote the predicted probability that an input $\mathbf{x}$ belongs to the j th class. For input $\mathbf{x}_i$ with one-hot encoded class label $\mathbf{y}_i$ it holds

$$p_{\mathbf{w}}(\mathbf{y}_i | \mathbf{x}_i) = \prod_{k=1}^K p_{\mathbf{w}}(\mathbf{e}_k | \mathbf{x}_i)^{[\mathbf{y}_i]_k} . \quad (2.56)$$

The negative log likelihood is then

$$-\ln p(\mathcal{D} | \mathbf{w}) = -\sum_{n=1}^N \sum_{k=1}^K [\mathbf{y}_n]_k \ln [f_{\mathbf{w}}(\mathbf{x}_n)]_k , \quad (2.57)$$

which is also called cross-entropy for multiple classes.

To ensure that the network prediction is a proper probability distribution, the K -dimensional output of the NN is passed through the soft-max (activation) function

$$[f_{\mathbf{w}}(\mathbf{x})]_j = \frac{\exp a_{M+D+j}}{\sum_{k=1}^K \exp a_{M+D+k}} , \quad (2.58)$$

which naturally extends the logistic function to multiple classes and ensures that $\sum_{j=k}^K [f_{\mathbf{w}}(\mathbf{x})]_k = 1$ and all $[f_{\mathbf{w}}(\mathbf{x})]_k > 0$.

Example 2.3 In the LunarLander example, the action 'fire main engine' could be encoded by $\mathbf{e}_3 = (0, 0, 1, 0)^T$.

It is more appropriate to view the soft-max normalization as part of the loss function than as activation function of the final NN layer, not only for computational reasons but also because an activation function of the form (2.58) strictly speaking violates the assumption of acyclic network connectivity.

2.2.3 Gradient-based optimization

Assume that we want to minimize a differentiable function $f_{\text{obj}} : \mathbb{R}^D \rightarrow \mathbb{R}$, for example an error function defined by the negative logarithmic likelihood given by (2.50), (2.55), or (2.57). Locally, the gradient of a differentiable function $f_{\text{obj}} : \mathbb{R}^D \rightarrow \mathbb{R}$

$$\nabla f_{\text{obj}}(\mathbf{w}) = \left(\frac{\partial f_{\text{obj}}(\mathbf{w})}{\partial x_1}, \frac{\partial f_{\text{obj}}(\mathbf{w})}{\partial x_2}, \dots, \frac{\partial f_{\text{obj}}(\mathbf{w})}{\partial x_D} \right)^T \quad (2.59)$$

points in the direction $\nabla f_{\text{obj}}(\mathbf{w}) / \|\nabla f_{\text{obj}}(\mathbf{w})\|$ giving the maximum rate of change. Thus, it is a common strategy to take advantage of gradient information for optimization. Many modern machine learning frameworks can compute gradients of models such as neural networks automatically using symbolic differentiation; the well-known backpropagation algorithm can be viewed as a special case of reverse mode (automatic) differentiation.

Let us consider a non-linear error function $E_{\mathcal{D}}(f_{\mathbf{w}})$ depending on training data $\mathcal{D}$ and the model's parameters $\mathbf{w}$. We assume that the gradient of $E_{\mathcal{D}}$ w.t.t. $\mathbf{w}$ exists, where existence of a subgradient is sufficient. We consider iterative gradient-based minimization of $E_{\mathcal{D}}(f_{\mathbf{w}})$. In its simplest form, steepest descent, in iteration t we update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} E_{\mathcal{D}}(f_{\mathbf{w}})|_{\mathbf{w}^{(t)}}$ with $\eta > 0$ until some termination criterion is met. Approximating $E_{\mathcal{D}}$ by $E_{\mathcal{D}^{(t)}}$, where $\mathcal{D}$ is a subset of $\mathcal{D}$, gives mini-batch learning, $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} E_{\mathcal{D}^{(t)}}(f_{\mathbf{w}})|_{\mathbf{w}^{(t)}}$.

Weight-decay

Deep neural networks can learn highly non-linear functions. Weight decay is a common way to favor simple mappings. Weight decay refers to penalize large weights by adding $\gamma_{\text{reg}} \frac{1}{2} \|\mathbf{w}\|^2$ with regularization hyperparameter $\gamma_{\text{reg}} \geq 0$ to the objective function (the weights of linear output neurons should not be considered when computing the norm of the weight vector). The smaller

the weights, the less drastic the output of the NN changes when the input changes.

To see this, let us restrict the hypothesis space to

$$\mathcal{H}_C = \left\{ f(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}) \mid \mathbf{w} \in \mathbb{R}^{\bar{D}} \wedge \|\mathbf{w}\| \leq C \right\} \quad (2.60)$$

with $C > 0$. If $\Phi : \mathbb{R}^D \rightarrow \mathbb{R}^{\bar{D}}$ is Lipschitz with constant α (i.e., $\|\Phi(\mathbf{x}_1) - \Phi(\mathbf{x}_2)\| \leq \alpha \|\mathbf{x}_1 - \mathbf{x}_2\|$ for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^D$), then the functions in f_C are Lipschitz with constant $C \cdot \alpha$. That is, C limits how fast the functions can change – it constrains a notion of complexity of the hypotheses class.

Adam

Mini-batch learning is the most prominent optimization approach in deep learning. Numerous gradient-based optimization algorithms that work with mini-batches exist. Very popular in deep learning is the Adam (Adaptive Moment Estimation) algorithm [Kingma and Ba, 2015]. Combining it with weight decay gives the AdamW algorithm outlines in algorithm 3 [Loshchilov and Hutter, 2019], which is used in several experiments later in this text.

Algorithm 3: AdamW algorithm

Input: $\mathbf{w}^{(0)}$, learning rate η , weight-decay parameter γ_{reg}
Set: $\beta_1, \beta_2, \epsilon$

```

1  $t \leftarrow 0$ 
2  $\hat{\mathbf{v}}^{(0)}, \mathbf{m}^{(0)} \leftarrow \mathbf{0}$ 
3 while stopping criterion not met do
4   foreach  $w_i$  do
5      $g_i^{(t)} \leftarrow \frac{\partial}{\partial w_i} E_{\mathcal{D}^{(t)}}(f_{\mathbf{w}}) |_{\mathbf{w}^{(t)}}$ 
6      $m_i^{(t+1)} \leftarrow \beta_1 \cdot m_i^{(t)} + (1 - \beta_1) \cdot g_i^{(t)}$ 
7      $v_i^{(t+1)} \leftarrow \beta_2 \cdot v_i^{(t)} + (1 - \beta_2) \cdot (g_i^{(t)})^2$ 
8      $\hat{m}_i^{(t+1)} \leftarrow m_i^{(t+1)} / (1 - \beta_1^t)$ 
9      $\hat{v}_i^{(t+1)} \leftarrow v_i^{(t+1)} / (1 - \beta_2^t)$ 
10     $w_i^{(t+1)} \leftarrow w_i^{(t)} - \eta \gamma_{\text{reg}} w_i^{(t)}$ 
11     $w_i^{(t+1)} \leftarrow w_i^{(t+1)} - \eta \cdot \hat{m}_i^{(t+1)} / (\sqrt{\hat{v}_i^{(t+1)}} + \epsilon)$ 
12   $t \leftarrow t + 1$ 
```

The algorithm considers each objective variable w_i independently. First, the partial derivative of the error function with respect to w_i is estimated using a mini batch, see line 5 in algorithm 3. The algorithm keeps track of exponential moving averages of the partial derivative and the partial derivative squared, see lines 6 and 7, with smoothing factors $\beta \in [0, 1[$ with default

$\beta_1 = 0.9$ and $\beta_2 \in [0, 1[$ with default $\beta_2 = 0.999$, respectively. The moving averages are then normalized by $(1 - \beta_1^t)$ and $(1 - \beta_2^t)$, respectively, to compensate for the initialization bias (which can be derived from the geometric series and assuming stationarity of the estimates of the partial gradients), where a superscript t denotes an exponent while a superscript (t) indicates the iteration step (lines 8 and 9). Line 10 implements the weight decay [see Loshchilov and Hutter, 2019]. The weights are then changed in line 11. The update is the negative global learning rate ϵ times the normalized moving average of the partial derivative $\hat{m}_i^{(t+1)}$ divided by the normalized moving average of the partial derivative squared $\hat{v}_i^{(t+1)}$. A small number ϵ with default $\epsilon = 10^{-8}$ is added to the denominator to avoid division by zero.

If $1 - \beta_1 = \sqrt{1 - \beta_2}$ (as it is the case for the default values), we have $\sqrt{\hat{v}_i^{(t+1)}} + \epsilon > \hat{m}_i^{(t+1)} \geq 0$, that is, η can be viewed as an upper bound on how much a parameter can be changed in one update step (similar to trust region or restricted-step methods, in which you do not trust the update direction beyond a region around your current search point). Note that sign changes in the partial derivative – which occur when local optima are overstepped – tend to make the absolute value of $\hat{m}_i$ smaller but do not have the same effect on $\hat{v}_i$. Thus, sign changes tend to reduce the update amount.

2.2.4 Convolutional neural networks

Although not a new approach, the current boom of neural networks started around 2010 with breakthrough successes in image classification using convolutional neural networks (CNNs). These breakthroughs are based on advances in algorithms and theoretical understanding, as well as on the availability of large amounts of data and computing power.

In deep neural networks, the outputs of layers are often referred to as feature maps and can be viewed as new representations of the input data. The idea of deep learning is to successively generate new representations of the data that are more suited for the task at hand. Convolutional neural networks are specifically designed for processing and analyzing data with spatial or temporal structure, such as images, natural language, or audio signals. This makes them very relevant for RL, when, for example, an agent receives visual input.

A CNN characteristically comprises convolutional layers as well as pooling layers. Convolutional and pooling layers take neighborhood relations into account, which is only meaningful if the neighborhood relations are meaningful, as in images and ordered sequences. Convolutional layers implement cross-correlation, a basic operation in signal processing. A filter, also called a kernel, is applied to small portions of the input. Let us assume that the inputs are images and that an input image is described by

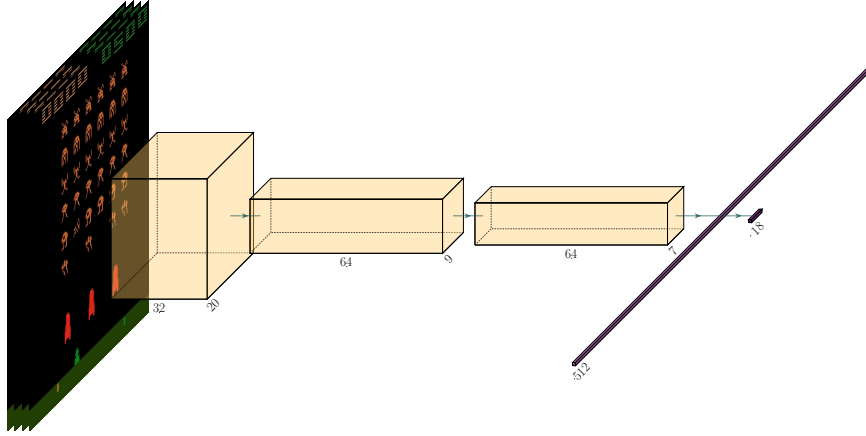


Fig. 2.5 In this example of a neural network for learning to play Atari video games based on Mnih et al. [2015], a sequence of four color video frames defines the input to allow to infer the velocity of moving objects. There are three convolutional layers with 32, 64, and 64 output feature maps, respectively. The first convolutional layer has 8×8 kernels (i.e., $C_{\text{out}} = 32$, $H_w = W_w = 8$) using stride 4. The second convolutional layer has 4×4 kernels with stride 2. The $20 \times 20 \times 32$ input is mapped to an $9 \times 9 \times 64$ output. Width and height are reduced because no padding is used and because of the stride ($\lfloor (20 - 4)/2 \rfloor + 1 = 9$). The third convolutional layer has $64 \times 3 \times 3$ kernels with stride 1, reducing height and width to 7. The output of the third convolutional layer is vectorized and serves as input to a standard neural network layer with 512 neurons. All hidden layers use the ReLU activation function. The final layer has one output for each action.

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}} \quad (2.61)$$

with elements $x_{i,j}^{(d)}$ where $i = 1, \dots, H$, $j = 1, \dots, W$, and $d = 1, \dots, C_{\text{in}}$. For an input image, H and W are height and width, respectively, and C_{in} is the number of input channels or input feature maps, for example $C_{\text{in}} = 3$ for RGB color images. fig. 2.5 shows an example where a sequence of four color video frames defines the input, where $C_{\text{in}} = 4 \times 3$.

In the following, we describe the computation of a canonical convolutional layer with output is $\mathbf{Z} \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$. In general, several kernels are applied in each convolutional layer. If C_{out} filters are applied, the resulting output consists of C_{out} feature maps. If the input to the layer is C_{in} feature maps (C_{in} input channels), each output feature map adds the results of C_{in} cross-correlations using C_{in} independent filters before applying the activation function. Let the kernels of the layer be given by

$$\mathbf{W}^{(c)} \in \mathbb{R}^{H_w \times W_w \times C_{\text{in}}} \quad (2.62)$$

with elements $w_{u,v}^{(c,d)}$ where $u = 1, \dots, H_w \leq H$, $v = 1, \dots, W_w \leq W$, and $d = 1, \dots, C_{\text{in}}$. For each output channel $c = 1, \dots, C_{\text{out}}$ and spatial indices $i = 1, \dots, H' = H - H_w + 1$ and $j = 1, \dots, W' = W - W_w + 1$, the output component in $\mathbf{Z}$ is computed as

$$z_{i,j}^{(c)} = h \left(b^{(c)} + \sum_{u=1}^{H_w} \sum_{v=1}^{W_w} \sum_{d=1}^{C_{\text{in}}} w_{u,v}^{(c,d)} x_{i+u-1,j+v-1}^{(d)} \right), \quad (2.63)$$

where $b^{(c)}$ denotes the bias for output channel c and h is an activation function. For h being the identity, $C_{\text{in}} = C_{\text{out}} = 1$, and $b^{(1)} = 0$, this can be regarded as canonical discrete two-dimensional cross-correlation. Convolutional layers can be efficiently computed on parallel hardware such as GPUs. fig. 2.6 shows a cross-correlation with a simple vertical edge filter $K = \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix}$ for $C_{\text{in}} = C_{\text{out}} = 1$. fig. 2.7 illustrates the computation in (2.63) for $H = W = 3$, $h_H = W_w = 2$, and $C_{\text{in}} = C_{\text{out}} = 1$ without bias parameter and h being the identity.

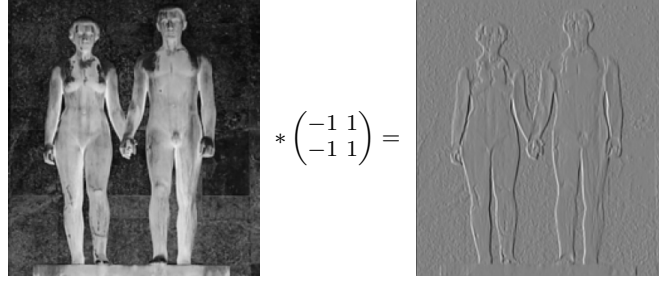


Fig. 2.6 Showing $\mathbf{X} * \mathbf{W} = \mathbf{Z}$, where $*$ denotes the cross-correlation operation, implementing a simple vertical edge filter.

As a second example, we consider a layer that preserves spatial dimensionalities, $H = H'$ and $W = W'$. For each output channel $c = 1, \dots, C_{\text{out}}$ and spatial indices (i, j) , the output component in $\mathbf{Z}$ is given by

$$z_{i,j}^{(c)} = h \left(b^{(c)} + \sum_{u=1}^{H_w} \sum_{v=1}^{W_w} \sum_{d=1}^{C_{\text{in}}} w_{u,v}^{(c,d)} \tilde{x}_{i+u-\lceil H_w/2 \rceil, j+v-\lceil W_w/2 \rceil}^{(d)} \right). \quad (2.64)$$

Here, the zero-padded input $\tilde{x}_{\alpha,\beta}^{(d)}$ is defined as

$$\tilde{x}_{\alpha,\beta}^{(d)} = \begin{cases} x_{\alpha,\beta}^{(d)} & \text{if } 1 \leq \alpha \leq H \text{ and } 1 \leq \beta \leq W \\ 0 & \text{otherwise} \end{cases} \quad (2.65)$$

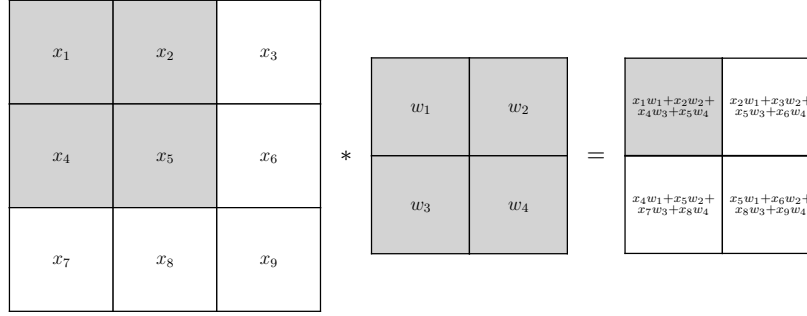


Fig. 2.7 Cross-correlation $\mathbf{X} * \mathbf{W}$ as computed by (2.63) without bias parameter for $H = W = 3$, $h_H = W_w = 2$, and $C_{\text{in}} = C_{\text{out}} = 1$.

Whereas in classical signal processing the filters are handcrafted, such as the simple vertical edge filter above, the coefficients of each convolution kernel $\mathbf{W}^{(c)}$ and bias parameter $b^{(c)}$ are learned as weights in a neural network. Equations (2.63) and (2.64) resemble (2.35), they can be viewed as definitions of neurons with the kernel parameters being shared weights. The kernel is typically smaller than the input. That is, each neuron is not fully connected. This increased memory and statistical efficiency (i.e., less parameters have to be estimated, which requires less data). Neurons that are part of the same feature map have the same weights; this again increases memory and statistical efficiency.

Sliding these filters across the entire input helps detect various features, such as edges, textures, and simple patterns in the case of input feature maps from images. Convolutional layers support equivariance: apart from boundary effects, a translated input evokes an output that is translated in the same way, which supports that an object at a certain position in the training data will also be recognized at a different position. This is demonstrated in fig. 2.8.

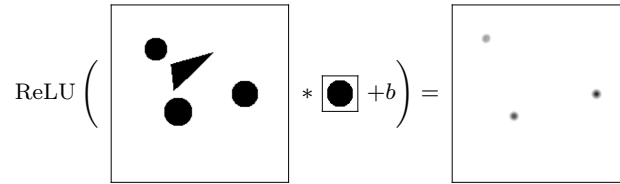


Fig. 2.8 Example demonstrating equivariance: For similar but translated inputs, the convolutional layer produces similar output activations translated in the same way. In this example, using the rectified linear unit function as activation function h sets all negative values to zero.

Pooling layers compute the maximum or average over a region of a feature map. They are used to reduce the dimensionality, change the scale, and support translation invariance. For example, a standard pooling layer with a 2×2 (pooling) kernel combines four neighboring pixels into one, by taking either their mean or their maximum (referred to as average pooling and max pooling, respectively), resulting in down-sampling by a factor of two.

The equivariance properties induced by the convolution layers can be turned into desired invariance properties by using pooling. For example, consider the toy example of a disk detector in fig. 2.8. A binary classifier should detect a disk regardless of where it is in the input image. The convolutional filter in fig. 2.8 produces an output feature map with a response for each disk in the image. Applying a max-pooling over the full feature map and thresholding the scalar response yields a simple disk detector.

Another way to reduce the spatial dimensionalities is through strided convolution. In (2.63) and (2.64), the kernel moves over the input feature maps in steps of one, in the case of an input image pixel by pixel. If we increase the step size, the stride, then the input is downsampled. For example, a stride of two skips every other position. Strided convolutions are used in the example in fig. 2.5.

The output of a convolutional layer can be vectorized, also referred to as flattening: The output is transformed into a single vector by stacking all elements of $\mathbf{Z}$. The resulting vector can then be processed by standard neural network layers such as the ones defined by (2.46) or in the following section on recurrent neural networks, as in the example in fig. 2.5.

2.2.5 Recurrent neural network models

For RL, recurrent neural networks (RNNs, Doya, 1995) are particularly relevant. An RNN model does not just represent a static mapping, but a dynamical system (i.e., it is endowed with a time parameter). The network graph does not need to be acyclic and each unit has some memory, a state. The output of the RNN depends on the weights and the current state of the units. We employ RNNs when a dynamical system is a proper statistical model for the task at hand. Thus, the data from the process underlying our model should have some sequential structure. In RL, the internal state of the RNN allows the network to memorize previous observations and to build a better representation of the world, in particular when the current observations are not sufficient to determine the optimal next action. That is, RNNs are particularly well suited for POMDPs. Let the state-space of the RNN be S_{RNN} . The idea is that a process that is non-Markovian given the observations $o \in \Omega$ from the environment is turned into a Markovian process on the extended state space $\Omega \times S_{\text{RNN}}$. In the example in fig. 2.5, providing a single frame is not enough to select the optimal action because the velocity information

is missing. Therefore, four frames are provided [Mnih et al., 2015]. If only a single frame is provided, which is an observation of the POMDP underlying the video game, a feed-forward neural network cannot learn an optimal policy. In contrast, an RNN can store information about previous frames in its internal state and learn to extract information related to velocity from the sequence of observations. This makes RL strategies for MDPs applicable to the partially observed process.

To keep the notation uncluttered, we consider the processing of a single time series indexed by t . We consider a layer of K recurrent units. The input to these units, which could be the external input or the output of another layer, is assumed to be D -dimensional, $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots \in \mathbb{R}^D$. Here, the superscript in parentheses indicates the time step and distinguishes quantities in the RNN from other quantities in this text. The K outputs of the recurrent units are gathered in the output state $\mathbf{h}^{(t)} = (h_1^{(t)}, \dots, h_K^{(t)})^T \in \mathbb{R}^K$. These outputs serve either as inputs to another layer of recurrent units or are mapped to the output of the network. This feed-forward mapping can be a simple affine linear function or a multi-layer feedforward neural network.

The (output) state $h_i^{(t)}$ of a basic RNN unit i at time step t , is given by

$$h_i^{(t)} = \sigma^{\text{out}}(\mathbf{u}_i \cdot \mathbf{x}^{(t)} + \mathbf{w}_i \cdot \mathbf{h}^{(t-1)} + b_i) \quad (2.66)$$

for $1 \leq i \leq K$ [Elman, 1990]. Each unit i corresponds to a single neuron with weight vectors $\mathbf{w}_i \in \mathbb{R}^K$ and $\mathbf{u}_i \in \mathbb{R}^D$ plus bias parameter $b_i \in \mathbb{R}$. The dynamics starts from an initial state $\mathbf{h}^{(0)}$. With a linear activation function σ^{out} , (2.66) reduces to a linear dynamical system. However, the activation function is typically a non-linear squashing function, such as $\sigma^{\text{out}}(x) = \tanh(x)$ or the logistic function $\sigma^{\text{out}}(x) = 1/(1 + \exp(-x))$. Bounded activation functions increase the stability of the network dynamics. Figure 2.9 visualizes a basic RNN unit.

Training an RNN can be done by gradient-based error minimization. Given an input time series $\mathcal{D}_{\mathbf{x}} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N_x)}\}$ of length N_x and output values $\mathcal{D}_{\mathbf{y}} = \{\mathbf{y}^{(t_1)}, \dots, \mathbf{y}^{(t_{N_y})}\}$ at some time steps $\{t_1, \dots, t_{N_y}\} \subseteq \{1, 2, \dots, N_x\}$, we can, for example, compute the MSE similar to (2.50) as

$$E_{\mathcal{D}}(f_{\mathbf{w}}) = \frac{1}{N_y} \sum_{\mathbf{y}^{(t)} \in \mathcal{D}_{\mathbf{y}}} \|\mathbf{h}^{(t)} - \mathbf{y}^{(t)}\|^2, \quad (2.67)$$

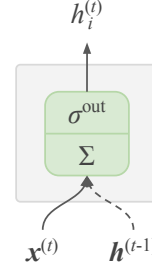


Fig. 2.9 Schema of a basic recurrent unit. The dashed line stand for input from the previous time step. The Σ sign denotes weighted summation and adding a bias.

where $\mathbf{h}^{(t)}$ depends on $\mathbf{x}^{(t)}$ and $\mathbf{h}^{(t-1)}$. Here we assume for simplicity that the training data set contains only a single time series. If there are several time series, the error is computed as the average error over the time series in the training data. Furthermore, we assumed that the states $\mathbf{h}^{(t)}$ of the recurrent units directly correspond to the outputs $\mathbf{y}^{(t)}$. This is not the case in general, but the states $\mathbf{h}^{(t)}$ can be mapped by a feed-forward neural (sub-)network to $\mathbb{R}^K$. Note that the initial state $\mathbf{h}^{(0)}$ matters and that an RNN may need some burn-in phase in the beginning to reach a sufficiently stable state.

Models based solely on these basic recurrent units are known to have problems learning long-term temporal dependencies [Hochreiter, 1991, Bengio et al., 1994, Hochreiter et al., 2001]. Networks with gated recurrent units, such as LSTM and GRU models, offer a remedy to this problem.

Long short-term memory units

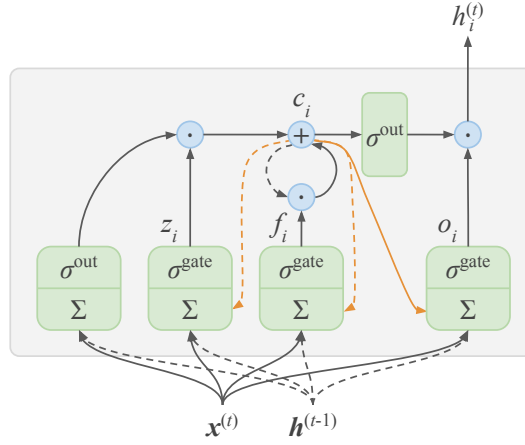


Fig. 2.10 Schema of a single LSTM unit inspired by Greff et al. [2017]. Dashed lines indicate input from the previous time step, additional paths when using peephole connections are marked in orange. The $\odot$ symbols stand for multiplication and the Σ signs for weighted summation and adding a bias.

The basic idea of long short-term memory neural networks (LSTMs) is presented by Hochreiter and Schmidhuber [1997]. The most commonly used LSTMs are based on Graves and Schmidhuber [2005], see also the report in Zaremba et al. [2014]. For a summary and discussion of further variants, we refer to Greff et al. [2017].

Recurrent LSTM networks have been suggested for solving non-Markovian RL tasks in the seminal paper by Bakker [2001]. Hausknecht and Stone [2015] replaced the first non-convolutional layer in the architecture shown in fig. 2.5

by an LSTM to learn to achieve high scores when playing Atari video games even if only a single video frame is provided as input in each time step.

The state of an LSTM unit i at time t is divided into internal state $s_i^{(t)}$ and the output state $h_i^{(t)}$. The update of these states depends on the previous states and the current input and is controlled by gates. The gates take values between 0 and 1 which are multiplied with incoming signals. There is an input gate $z_i^{(t)}$, an output gate $o_i^{(t)}$, and a forget gate $f_i^{(t)}$. The update dynamics of a basic LSTM unit reads:

$$z_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^z \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^z \cdot \mathbf{h}^{(t-1)} + b_i^z) \quad \text{[input gate]} \quad (2.68)$$

$$o_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^o \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^o \cdot \mathbf{h}^{(t-1)} + b_i^o) \quad \text{[output gate]} \quad (2.69)$$

$$f_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^f \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^f \cdot \mathbf{h}^{(t-1)} + b_i^f + b^f) \quad \text{[forget gate]} \quad (2.70)$$

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + z_i^{(t)} \sigma^{\text{out}}(\mathbf{u}_i \cdot \mathbf{x}^{(t)} + \mathbf{w}_i \cdot \mathbf{h}^{(t-1)} + b_i) \quad \text{[internal state]} \quad (2.71)$$

$$h_i^{(t)} = \sigma^{\text{out}}(s_i^{(t)}) o_i^{(t)} \quad (2.72)$$

Each unit i has parameters $\mathbf{w}_i^z, \mathbf{w}_i^o, \mathbf{w}_i^f, \mathbf{w}_i \in \mathbb{R}^K$, $\mathbf{u}_i^z, \mathbf{u}_i^o, \mathbf{u}_i^f, \mathbf{u}_i \in \mathbb{R}^D$, and $b_i^z, b_i^o, b_i^f, b_i \in \mathbb{R}$. The non-linearity σ^{gate} for the gating is the logistic function. Typically, $\sigma^{\text{out}}(x) = \tanh(x)$ is used. The constant b^f is called forget bias and is not adapted during training, setting it to zero recovers the standard model [e.g., [Zaremba et al., 2014](#)]. Increasing b^f increases the influence of the previous state $s_i^{(t-1)}$ on the current state $s_i^{(t)}$, see (2.70) and (2.71). The constant b^f , often set to one, can be compensated by the adaptive b_i^f and has been introduced to influence the initial learning phase (which could also be achieved by a tailored initialization of b_i^f). Figure 2.10 shows a diagram of a single LSTM unit.

[Gers and Schmidhuber \[2000\]](#) introduced peephole connections to LSTMs. This changes equations (2.68), (2.69), and (2.70) to

$$z_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^z \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^z \cdot \mathbf{h}^{(t-1)} + p_i^z s_i^{(t-1)} + b_i^z) \quad (2.73)$$

$$o_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^o \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^o \cdot \mathbf{h}^{(t-1)} + p_i^o s_i^{(t-1)} + b_i^o) \quad (2.74)$$

$$f_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^f \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^f \cdot \mathbf{h}^{(t-1)} + p_i^f s_i^{(t)} + b_i^f + b^f) \quad (2.75)$$

with additional parameters $p_i^z, p_i^o, p_i^f \in \mathbb{R}$ for each unit i . In their empirical comparison, [Greff et al. \[2017\]](#) concluded that peephole connections do not significantly change LSTM performance, which is in accordance with our experience.

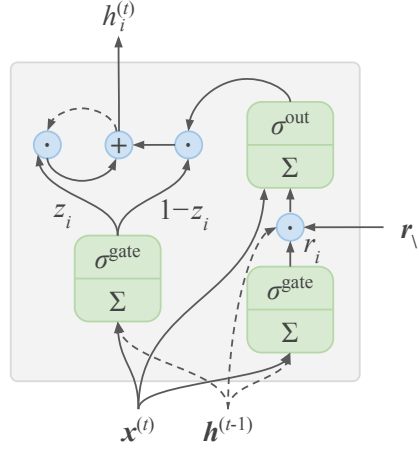


Fig. 2.11 Schema of a single GRU unit. Here $\mathbf{r}_{\setminus i} = (r_1, \dots, r_{i-1}, r_{i+1}, \dots, r_K)$ denotes the values from the reset gates of the GRU units in the same layer. Dashed lines indicate input from the previous time step. The $\odot$ symbols stand for (element-wise) multiplication and the Σ signs for weighted summation and adding a bias.

Gated recurrent units

Gated recurrent units (GRU, [Choi et al. \[2014\]](#)) can be viewed as simplifications of LSTM units, see Figure 2.11. The update of the output states $\mathbf{h}^{(t)} \in \mathbb{R}^K$ is controlled by reset gates $\mathbf{r}^{(t)} \in \mathbb{R}^K$ and update gates $\mathbf{z}^{(t)} \in \mathbb{R}^K$,

$$z_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^z \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^z \cdot \mathbf{h}^{(t-1)} + b_i^z) \quad [\text{update gate}] \quad (2.76)$$

$$r_i^{(t)} = \sigma^{\text{gate}}(\mathbf{u}_i^r \cdot \mathbf{x}^{(t)} + \mathbf{w}_i^r \cdot \mathbf{h}^{(t-1)} + b_i^r) \quad [\text{reset gate}] \quad (2.77)$$

$$h_i^{(t)} = z_i^{(t)} h_i^{(t-1)} + (1 - z_i^{(t)}) \sigma^{\text{out}}(\mathbf{u}_i \cdot \mathbf{x}^{(t)} + \mathbf{w}_i \cdot [\mathbf{h}^{(t-1)} \odot \mathbf{r}^{(t)}] + b_i) \quad (2.78)$$

for $1 \leq i \leq K$, where $\odot$ denotes element-wise multiplication. Each unit i has parameters $\mathbf{w}_i^z, \mathbf{w}_i^r, \mathbf{w}_i \in \mathbb{R}^K$, $\mathbf{u}_i^z, \mathbf{u}_i^r, \mathbf{u}_i \in \mathbb{R}^D$, and $b_i^z, b_i^r, b_i \in \mathbb{R}$.

Recurrent neural networks for RL

There is no reason to employ recurrent neural networks in deep RL systems for solving fully observable MDPs. In a recent comparison by [Smirnov and Gu \[2025\]](#), LSTM and GRU as reviewed in section 2.2.5 performed well for POMDPs. If fast training is required, [Smirnov and Gu](#) recommend the recurrent model Mamba-2 [\[Dao and Gu, 2024\]](#), and they conclude that transformer models (e.g., [Parisotto et al. \[2020\]](#)) “should be considered only for environments that are extremely memory-intensive, such as long-horizon planning or complex partially observable tasks” and that their “high computational

cost and implementation complexity make them unsuitable for most practical applications unless [...]”.

2.3 Monte Carlo methods

Let f be a deterministic function depending on a random variable z with probability distribution p parameterized by θ . Assume that we are interested in an integral

$$\mathbb{E}[f(z)] = \int f(z)p(z)dz \quad (2.79)$$

that is difficult to compute analytically, for example, because we do not know the distribution p but can only sample from it. Crude Monte Carlo methods approximate the integral by

$$\frac{1}{n} \sum_{i=1}^n f(z_i) \ , \quad (2.80)$$

where

$$z_i \sim p(z) \quad (2.81)$$

for $i = 1, \dots, n$. That is, an expectation is approximated by a finite sample of size n . We have

$$\frac{1}{n} \sum_{i=1}^n f(z_i) \xrightarrow{n \rightarrow \infty} \mathbb{E}[f(z)] \quad (2.82)$$

because of the law of large numbers, which, informally speaking, implies that if you sample many times and compute the average, then the result should be close to the expected value. The larger the sample size, here n , the better the approximation.

In RL, an example of a Monte Carlo approximation would be the approximation of the return R from a finite number of episodes, as done in example [3.2](#).

2.3.1 Importance weighting

To compute the Monte Carlo approximation (2.80) of (2.79), we need access to samples z_i from the distribution p . But what can we do if we have only samples z_i from a distribution q defined over the same domain as p ?

If we can compute $p(z_i)/q(z_i)$, then we can apply importance weighting. For two distributions p and q with $p(z) > 0 \Rightarrow q(z) > 0$ we have

$$\mathbb{E}_{z \sim p}[f(z)] = \int \frac{p(z)}{q(z)} f(z) q(z) dz = \mathbb{E}_{z \sim q} \left[\frac{p(z)}{q(z)} f(z) \right] \quad (2.83)$$

and thus

$$\frac{1}{n} \sum_{i=1}^n \frac{p(z_i)}{q(z_i)} f(z_i) \xrightarrow{n \rightarrow \infty} \mathbb{E}[f(z)] \quad , \quad (2.84)$$

where $z_i \sim q(z)$ for $i = 1, \dots, n$, that is, we approximate the expectation over p using samples from q . The weighting in (2.84) can be interpreted as follows. We have $\int \frac{p(z)}{q(z)} q(z) dz = 1$. If a sample z_i is twice as common when sampling from p than from q , then $p(z_i)/q(z_i) = 2$ and the sample will get a higher weight; if the sample is twice as common when sampling from q than from p , then $p(z_i)/q(z_i) = \frac{1}{2}$ and the sample will get a lower weight, because it will be less likely when drawing from p .

2.3.2 Control variates

Assume we want to estimate $\mathbb{E}[f] = \mathbb{E}_{z \sim p}[f(z)]$ via Monte Carlo (2.80) and are concerned about the variance of our estimator. That is, while in expectation our Monte Carlo approach will converge to the right quantity because of the law of large numbers, for a finite sample the estimate may strongly vary from the expectation, that is, the variance of the estimator could be large. Importance weighting can increase the variance due to the weighting in (2.84). Assume that a sample $z^\dagger$ is much more common when sampling from p than from the q , the distribution from which we are actually sampling. Then this rare sample $z^\dagger$ will have a large weight because $p(z^\dagger)/q(z^\dagger)$ is huge. That is, a rare occurrence of $z^\dagger$ in our finite Monte Carlo sample will drastically influence the approximation.

As a general strategy to reduce the variance of the estimator is to introduce a control variate. We introduce a helper function – the control variate – ϕ correlated with f for which we know $\bar{\phi} = \mathbb{E}[\phi]$ and write

$$\mathbb{E}[f] = \mathbb{E}[f - \phi] + \mathbb{E}[\phi] = \mathbb{E}[f - \phi] + \bar{\phi} = \mathbb{E}[f - \phi + \bar{\phi}] \quad . \quad (2.85)$$

We have

$$\text{Var}[f - \phi + \bar{\phi}] = \text{Var}[f - \phi] = \text{Var}[f] - 2 \text{Cov}[f, \phi] + \text{Var}[\phi] \quad . \quad (2.86)$$

Thus, the control variate can reduce the variance in the sense that

$$\text{Cov}[f, \phi] > \text{Var}[\phi] \Rightarrow \text{Var}[f - \phi] < \text{Var}[f] \quad . \quad (2.87)$$

Analogously,

$$\text{Cov}[f, \phi] < \text{Var}[\phi] \Rightarrow \text{Var}[f + \phi] < \text{Var}[f] \quad . \quad (2.88)$$

Here, $\text{Var}[\cdot]$ and $\text{Cov}[\cdot, \cdot]$ denote variance and covariance, respectively. As we will see, control variates are commonly used in deep RL.

2.3.3 Differentiating expectations

In the following, we introduce mathematical techniques that help in estimating the gradients of expectations with respect to parameters of a probability distribution using Monte Carlo sampling¹

Let f be a deterministic function depending on a random variable $z \sim p_{\theta}$. We would like to compute the gradient of $\mathbb{E}[f(z)]$ with respect to θ :

$$\nabla_{\theta} \mathbb{E}[f(z)] \quad (2.89)$$

For example, p_{θ} could be a normal distribution over $\mathbb{R}$ with parameter vector θ comprising its mean and standard deviation. Because θ are the parameters of the random variable, it is not obvious how to apply a Monte Carlo method to estimate (2.89) given a sample $z_1, \dots, z_n \sim p_{\theta}$.

2.3.3.1 Log-derivative trick

From

$$\nabla_{\theta} \log p_{\theta}(z) = \frac{\nabla_{\theta} p_{\theta}(z)}{p_{\theta}(z)} \quad (2.90)$$

follows

$$\nabla_{\theta} p_{\theta}(z) = p_{\theta}(z) \nabla_{\theta} \log p_{\theta}(z) . \quad (2.91)$$

This implies

$$\nabla_{\theta} \mathbb{E}[f(z)] = \int f(z) \nabla_{\theta} p_{\theta}(z) dz = \mathbb{E}[f(z) \nabla_{\theta} \log p_{\theta}(z)] . \quad (2.92)$$

Why is this exciting? Because $\mathbb{E}[f(z) \nabla_{\theta} \log p_{\theta}(z)]$ lends itself to Monte Carlo estimation via

$$\frac{1}{n} \sum_{i=1}^n f(z_i) \nabla_{\theta} \log p_{\theta}(z_i) \quad (2.93)$$

with $z_i \sim p_{\theta}(z)$ for $i = 1, \dots, n$.

The function $\nabla_{\theta} \log p_{\theta}(z)$ is called score function.

¹ In machine learning, in particular in deep learning, it has become common to refer to useful mathematical transformations as “tricks”. However, there is no trickery; it is just – often very simple - math. I don’t like this trend, it devalues mathematical understanding, which I regard as essential in machine learning.

2.3.3.2 Reparameterization trick

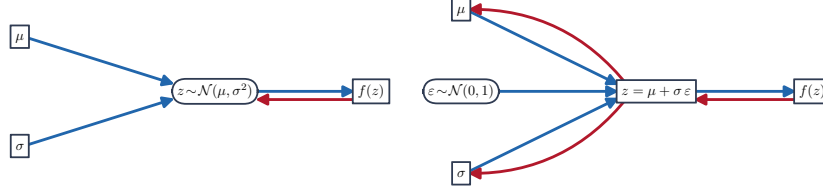


Fig. 2.12 Illustration of the reparameterization trick, see example 2.4. The graphs are compute graphs, where square and round nodes indicate deterministic nodes (a parameter or deterministic function) and stochastic nodes, respectively. The blue arrows indicate the dependencies for computing the function f . If we do reverse-mode automatic differentiation, as often used in machine learning, we follow the red arrows when computing the gradient (referred to backpropagation in the context of neural networks). The left graph is the compute graph for a $f(z)$ with $z \sim \mathcal{N}(\mu, \sigma^2)$, the right graph depicts the corresponding reparameterization $z = g_{\theta}(\epsilon) = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$. In the right graph, there is a path following red arrows from the output to the parameters μ and σ only connecting deterministic nodes, which does not exist in the left graph.

Let us assume that we can rewrite the definition of the random variable z as

$$z = g_{\theta}(\epsilon) \text{ with } \epsilon \sim q \quad (2.94)$$

by introducing a new random variable ϵ with distribution q such that when sampling ϵ then z still has the distribution p_{θ} . The reparameterization offers a way to estimate the gradient given an i.i.d. sample $\epsilon_1, \dots, \epsilon_n$ drawn from q :

$$\nabla_{\theta} \mathbb{E}_{z \sim p_{\theta}} [f(z)] = \nabla_{\theta} \mathbb{E}_{\epsilon \sim q} [f(g_{\theta}(\epsilon))] \approx \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} f(g_{\theta}(\epsilon_i)) \quad (2.95)$$

Example 2.4 Consider $f(z)$ with $z \sim \mathcal{N}(\mu, \sigma^2)$, that is, using the terminology above $\mathcal{N}(\mu, \sigma^2) = p_{\theta}$ with $\theta = (\mu, \sigma)^T$. Assume that we want to minimize $f(z)$ w.r.t. μ and σ given a sample $z_1, \dots, z_n \sim p_{\theta}$. It is not obvious how to compute the gradient w.r.t. μ and σ , as illustrated in the left graph in fig. 2.12, because $f(z)$ depends on μ and σ via a random variable.

Now we reparameterize $z = g_{\theta}(\epsilon) = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$. Now g_{θ} is a deterministic function of μ and σ (see right graph). Given $\epsilon_1, \dots, \epsilon_n \sim \mathcal{N}(0, 1)$, we can now use standard gradient-based optimization to minimize a Monte Carlo (2.95) estimate of $\mathbb{E}[f(z)]$.

Chapter 3

Direct policy search

Abstract Direct search methods solve optimization problems without relying on gradient information. Reinforcement learning can be phrased as maximizing the expected return. The expected return can be estimated using a Monte Carlo approximation. Applying direct search methods to optimize an estimate of expected return is known as a direct policy search. This chapter introduces two direct policy search methods that have successfully been used for RL and neural network policies, the cross-entropy method (CE) and the covariance matrix adaption evolution strategy (CMA-ES).

3.1 Direct search

Reinforcement learning can be expressed as an optimization problem. Given a space of policies $\{\pi_{\theta} \mid \theta \in \Theta\}$, we want to find a policy that maximizes the expected return (cumulative sum of rewards):

$$\pi^* = \arg \max_{\theta \in \Theta} J(\pi_{\theta}) \quad \text{with} \quad J(\pi_{\theta}) = \mathbb{E}^{\pi_{\theta}}[R_0] \quad (3.1)$$

But how to compute $J(\pi_{\theta})$ and how to optimize it? Monte Carlo approximation (see section 2.3) and direct search methods are a straightforward answer.

Direct search refers to the optimization without the use of gradient information. Such methods are particularly useful when the objective function (i.e., the function to be optimized) is not differentiable, noisy, or in general when gradient information is difficult or expensive to obtain. They only require the ability to evaluate the objective function on the search space.

Monte Carlo methods allow to approximate $J(\pi_{\theta})$ by an estimate of the return

$$J^{\text{MC}}(\pi_{\theta}) = \frac{1}{n_{\text{MC}}} \sum_{m=1}^{n_{\text{MC}}} \sum_{t=0}^{T_m} \gamma^t r_{t+1}^m \approx \mathbb{E}^{\pi_{\theta}}[R_0] = J(\pi_{\theta}) \quad , \quad (3.2)$$

which is based on n_{MC} episodes

$$s_0^m, a_0^m, r_1^m, s_1^m, a_1^m, r_2^m, \dots, a_{T_m-1}^m, r_{T_m}^m, s_{T_m}^m \quad (3.3)$$

following π starting from $s_0^m \sim p_{\text{start}}$ and having length T_m for $m = 1, \dots, n_{\text{MC}}$. These episodes are typically sampled anew for every evaluation of $J^{\text{MC}}(\pi_{\theta})$. Thus, $J^{\text{MC}}(\pi_{\theta})$ is a random variable, which leads to a noisy objective function when maximized. If the task is not naturally episodic (or single episodes are too long), we must break down the sequence of interactions into subsequences of finite length.

Direct search and Monte Carlo approximation can be combined for RL to find policies with high expected return. The idea is to consider policies π_{θ} parameterized by $\theta \in \mathbb{R}^n$ and apply direct search to optimize a Monte Carlo estimate of the expected return $\mathbb{E}^{\pi_{\theta}}[R_0]$. This RL approach is called direct policy search. For example, the class of policies can be defined by a neural network, mapping a state to an action, see fig. 5.1. In this case, the parameters θ correspond to the weights of the network, which is then called a policy network.

Optimizing policies directly without learning a value function as in Q -learning has some advantages. For example, value function approaches can be difficult to apply when the state and/or action space is continuous. If the action space is large, finding the best action in each time step can be computationally expensive. The drawbacks of value function approaches will be discussed further in the beginning of chapter 5.

In the following, we introduce two direct policy search methods that have successfully been used for RL, the cross-entropy method (CE) and the covariance matrix adaption evolution strategy (CMA-ES). We then discuss their application to RL tasks in section 3.4.

3.2 Cross-entropy method

The cross-entropy method (CE) is a popular direct search approach [Rubinstein and Kroese, 2004], which has been applied to RL, see the work by Szita and Lőrincz [2006] and Mannor et al. [2003], and will now be presented in its basic form.

The CE method is an iterative random search technique. The algorithm starts from some initial (prior) search distribution over Θ . In each iteration, the algorithm draws candidate policies from the search distribution, evaluates the policies, and then updates the search distribution based on the better candidates. The idea is to update the search distribution so that sampling high-quality policies in the next iteration becomes more likely.

We presume $\Theta = \mathbb{R}^n$ and consider a multivariate normal search distribution with mean θ and covariance matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$. We assume that the

covariance matrix is diagonal, $\mathbf{C} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_n)$, parameterized by $\boldsymbol{\sigma} \in \mathbb{R}^n$. In each iteration, we draw λ candidate policy parameter vectors $\boldsymbol{\theta}_l$ and evaluate them by computing $J^{\text{MC}}(\pi_{\boldsymbol{\theta}_l})$. Let the i -th best of the λ parameter vectors be denoted by $\boldsymbol{\theta}_{i:\lambda}$. Then the mean of the search distribution is updated to be the mean of the μ best of the λ candidates, and the covariance matrix is updated to be the variance of the selected μ candidates:

$$\boldsymbol{\theta} \leftarrow \frac{1}{\mu} \sum_{i=1}^{\mu} \boldsymbol{\theta}_{i:\lambda} \quad (3.4)$$

$$\boldsymbol{\sigma} \leftarrow \text{Var}[\{\boldsymbol{\theta}_{1:\lambda}, \dots, \boldsymbol{\theta}_{\mu:\lambda}\}] \quad (3.5)$$

The variance of the set of vectors in line 5 is computed component-wise (i.e., we compute the variance for each of the n dimensions separately). In practice, a small constant ϵ is added to the diagonal of $\mathbf{C}$ to ensure minimal variation along each axis. This prevents the search distribution from collapsing into a lower subspace of Θ . Of course, μ must be smaller than λ . The appropriate values of λ and μ depend on the problem difficulty; λ has to be large enough to ensure progress and a sufficiently good estimation of $\boldsymbol{\sigma}$. Pseudo-code of the CE method is shown in algorithm 4. One has to specify the parameters of the initial search distribution. The initial $\boldsymbol{\sigma}_{\text{init}}$ can incorporate prior knowledge about the scaling of the parameters. The initial mean $\boldsymbol{\theta}_{\text{init}}$ should be a meaningful initialization of policy parameters. It can be based on prior knowledge, in which case the initial variances $\boldsymbol{\sigma}_{\text{init}}$ should be small enough to ensure that the algorithm does not jump directly away from its starting point.

Algorithm 4: Cross-entropy method

Input: $\boldsymbol{\theta} = \boldsymbol{\theta}_{\text{init}}$, $\boldsymbol{\sigma} = \boldsymbol{\sigma}_{\text{init}}$, μ , λ , ϵ

```

1 repeat
    // sample new policies
2   for  $l = 1, \dots, \lambda$  do  $\boldsymbol{\theta}_l \sim \mathcal{N}(\boldsymbol{\theta}, \text{diag}(\sigma_1 + \epsilon, \sigma_2 + \epsilon, \dots))$ 
    // evaluate new policies
3   for  $l = 1, \dots, \lambda$  do evaluate  $J^{\text{MC}}(\boldsymbol{\theta}_l)$ 
    // update mean
4    $\boldsymbol{\theta} \leftarrow \frac{1}{\mu} \sum_{i=1}^{\mu} \boldsymbol{\theta}_{i:\lambda}$ 
    // update variance
5    $\boldsymbol{\sigma} \leftarrow \text{Var}[\{\boldsymbol{\theta}_{1:\lambda}, \dots, \boldsymbol{\theta}_{\mu:\lambda}\}]$ 
6 until some stopping criterion is met

```

Moving the mean of the search distribution to the mean of the μ selected samples places a higher probability in the area where the better policies were found. If this area is small, as measured by the variance, the search will be localized accordingly. If the area is large, the algorithm will search broadly.

The simplicity of the cross-entropy method is appealing. The selection is based on ranking policies, not on absolute values of return estimates. This makes the algorithm invariant to order-preserving transformations of the objective function, in the RL case of the reward.

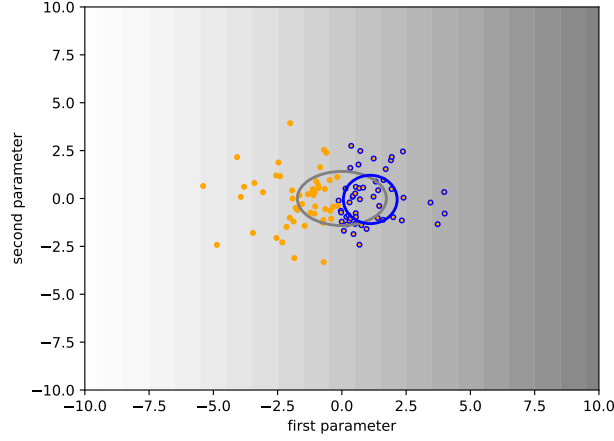


Fig. 3.1 This figure illustrates a fundamental problem with updating the search distribution as in the CE method. The example shows the optimization of a two-dimension linear function. The gray contours indicate the objective function values. The plot shows $\lambda = 100$ points generated from a Gaussian distribution. The shape of the covariance of that distribution is indicated by the gray ellipse. The best $\mu = \lambda/2$ points are highlighted in blue. Estimating a covariance matrix from these points as describe in algorithm 4 leads to a search distribution with less variance in the gradient direction as indicated by the blue ellipse.

The direct policy search approach has some general drawbacks, which we will discuss at the end of this chapter. However, the CE method has particular conceptual weaknesses. Assume that the objective function is (affine) linear (i.e., $J^{\text{MC}}(\pi_{\theta}) = \mathbf{a}^T \theta + b$ for $\mathbf{a} \in \mathbb{R}^n, b \in \mathbb{R}$). Then the update scheme will then tend to reduce the variance in the direction of maximum progress (i.e., in the direction of $\mathbf{a}$); for an illustration, see fig. 3.1. In the basic version shown in algorithm 4, the CE method does not learn dependencies between variables and does not exploit information from previous iterations beyond the current mean θ . Furthermore, μ and λ are difficult to choose in practice and have to be rather large, which slows the algorithm down. These shortcomings are addressed by the CMA-ES algorithm, which is discussed in the next section.

3.3 Covariance matrix adaption evolution strategy

The CMA-ES [Hansen and Ostermeier, 2001, Hansen, 2023] is a direct search method that has been praised to be “widely regarded as the state of the art in numerical optimization” [Eiben and Smith, 2015], and has been successfully applied to RL [Heidrich-Meisner and Igel, 2009]. The CMA-ES is an evolutionary algorithm; Bai et al. [2023] review applications of algorithms from this family to RL. Similar to the CE algorithm described in the previous section, the CMA-ES relies on a multivariate Gaussian search distribution $\mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{C})$, however, the CMA-ES can also learn covariances. The covariance matrix $\sigma^2 \mathbf{C}$ is parametrized by a symmetric matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$ scaled by the global step-size $\sigma \in \mathbb{R}$ squared.

Why do we need the apparently redundant step-size parameter σ ? The reason for this parameterization is that less information is required to learn a scalar step-size parameter compared to learning the $O(n^2)$ entries of the full covariance matrix $\mathbf{C}$. Therefore, the CMA-ES decouples step-size adaptation and covariance matrix learning in order to adjust the step-size on a faster timescale.

Updating the mean

The CMA-ES iteratively samples candidate policies and updates its search distribution similar to the CE method. Let us write the sampling of the new λ candidate policies as

$$\mathbf{y}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{C}) \quad (3.6)$$

$$\boldsymbol{\theta}_l = \boldsymbol{\theta} + \sigma \mathbf{y}_l \quad (3.7)$$

That is, $\mathbf{y}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$ and $\boldsymbol{\theta}_l \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{C})$. The mean $\boldsymbol{\theta}$ is updated by the weighted sum

$$\mathbf{y}_w \leftarrow \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \quad (3.8)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \sigma \mathbf{y}_w \quad (3.9)$$

with

$$\sum_{i=1}^{\mu} w_i = 1 \quad \text{and} \quad w_1 \geq w_2 \geq \dots \geq w_{\mu} \quad (3.10)$$

For $w_1 = w_2 = \dots = w_{\mu} = \mu^{-1}$ and $w_{\mu+1} = \dots = w_{\lambda} = 0$, we recover the CE update (3.4). If only non-negative weights are used,

$$w_i = \frac{w'_i}{\sum_{j=1}^{\mu} w'_j} \quad \text{with} \quad w'_i = \ln(\mu + 0.5) - \ln i \quad (3.11)$$

for $i = 1, \dots, \mu$ and $w_{\mu+1} = \dots = w_\lambda = 0$ is a common choice.

How is $\mathbf{y}_w$ distributed? When adding independent random variables, their variances add up. If the μ best policies were selected uniformly at random from the λ candidates, we would have $w_i \mathbf{y}_{i:\lambda} \sim \mathcal{N}(\mathbf{0}, w_i^2 \mathbf{C})$ and $\mathbf{y}_w = \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \sim \mathcal{N}(\mathbf{0}, \mu_w^{-1} \mathbf{C})$ with $\mu_w = (\sum_{i=1}^{\mu} w_i^2)^{-1}$. This implies that $\sqrt{\mu_w} \mathbf{y}_w \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$, and we can treat $\sqrt{\mu_w} \mathbf{y}_w$ in the same way as the vectors $\mathbf{y}_l$ under the assumption that the order of the λ candidates is random.

Covariance matrix update

The covariance matrix is updated to make recent beneficial steps more likely (“What has been good in the recent past will also be good in the near future”). How should a covariance matrix $\mathbf{C}$ of a Gaussian distribution with zero mean be changed to make a step $\sqrt{\mu_w} \mathbf{y}_w$ more likely? Consider the distributions $\mathcal{N}(\mathbf{0}, \mathbf{C}' + \epsilon \mathbf{I})$ for $\epsilon \rightarrow 0$. Which covariance matrix $\mathbf{C}'$ makes sampling the vector $\sqrt{\mu_w} \mathbf{y}_w$ most likely? Intuitively, the best choice is the rank-1 matrix $\mathbf{C}' = \mu_w \mathbf{y}_w \mathbf{y}_w^T$ (note the outer product and that the following argument is not mathematically rigorous): $\mathbf{y}_w$ is up to scalar multiplication the only eigenvector of $\mathbf{y}_w \mathbf{y}_w^T$ with non-zero eigenvalue. That is, for $\epsilon \rightarrow 0$ we approach a distribution that only generates points in the span of $\mathbf{y}_w$. And the zero-mean one-dimensional Gaussian that makes observing a certain value most likely is the one where the standard deviation equals that value.

Now we could make $\mathbf{C}$ more like $\mu_w \mathbf{y}_w \mathbf{y}_w^T$, but it is beneficial to also take the steps from previous iterations into account, similar to adding a momentum term in stochastic gradient descent. We keep track of a special exponential moving average called evolution path

$$\mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_w} \mathbf{y}_w, \quad (3.12)$$

where the weights are chosen such that no change in variance happens because of the weighted combination as $(1 - c_c)^2 + \sqrt{1 - (1 - c_c)^2}^2 = 1$. That is, lifted to the level of covariance matrices, (3.12) corresponds to the exponential moving average

$$\mathbf{p}_c \mathbf{p}_c^T \leftarrow (1 - \alpha) \mathbf{p}_c \mathbf{p}_c^T + \alpha \mu_w \mathbf{y}_w \mathbf{y}_w^T \quad (3.13)$$

with smoothing factor $\alpha = 1 - (1 - c_c)^2$. This exponential smoothing stabilizes the update, which allows for smaller values of λ . It is important to use (3.12) instead of the simpler looking (3.13), because sign information is important, which is lost in (3.13) ($\mathbf{y}_w \mathbf{y}_w^T = (-\mathbf{y}_w)(-\mathbf{y}_w)^T$). This type of covariance update is called rank-1 update, because $\mathbf{C}$ is updated by the matrix $\mathbf{y}_w \mathbf{y}_w^T$ of rank one.

If we sample many candidate policies in each iteration, a rank- μ update as introduced by Hansen et al. [2003] increases learning speed. The rank- μ covariance matrix update with smoothing factor c_μ reads

$$\mathbf{C} \leftarrow (1 - c_\mu)\mathbf{C} + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T, \quad (3.14)$$

where for non-zero $w_1, \dots, w_\mu$ the matrix added to $\mathbf{C}$ has rank $\min(\mu, n)$ almost surely.

Putting rank-1 and rank- μ update together, we get:

$$\begin{aligned} \mathbf{C} &\leftarrow \mathbf{C} + c_\mu \sum_{i=1}^{\lambda} w_i (\mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T - \mathbf{C}) + c_1 (\mathbf{p}_c \mathbf{p}_c^T - \mathbf{C}) \\ &= (1 - c_1 - c_\mu) \mathbf{C} + \underbrace{c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T}_{\text{rank-}\mu} + \underbrace{c_1 \mathbf{p}_c \mathbf{p}_c^T}_{\text{rank-1}} \end{aligned} \quad (3.15)$$

For c_1 and c_μ default values can be used (see algorithm 5), which scale with $O(1/n^2)$ following the $O(n^2)$ degrees of freedom of the covariance matrix.

Step-size adaptation

The global step-size σ is adapted on a faster timescale. The idea is to look at the length of the steps taken and compare them to the expected length under the assumption that the policies were ranked randomly, that is, the selection of the best μ policies was random. If we then see a pattern that the selection led to shorter or longer steps than expected without selection, we can adjust the step-size accordingly. For this comparison, we combine the previous steps, putting more weight on the more recent ones. The difficulty lies in the proper normalization of the steps when combining them, as the steps were produced under a changing covariance of the search distribution.

Again, exponential smoothing is used to combine steps. We consider a different evolution path, the conjugate evolution path, which takes the more complex form

$$\mathbf{p}_\sigma \leftarrow (1 - c_\sigma) \mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_{\mathbf{w}}} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_{\mathbf{w}}. \quad (3.16)$$

Assume that $\mathbf{p}_\sigma$ on the RHS (i.e., the $\mathbf{p}_\sigma$ before the update) is standard normally distributed, $\mathbf{p}_\sigma \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Under the random selection assumption, we have $\sqrt{\mu_{\mathbf{w}}} \mathbf{y}_{\mathbf{w}} \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$ and thus $\sqrt{\mu_{\mathbf{w}}} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_{\mathbf{w}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then the weighting in (3.16) parameterized by c_c ensures that the updated $\mathbf{p}_\sigma$ on the LHS is also standard normally distributed. The parameter c_c can be set to its default value, which scales with $O(n)$, reflecting the faster timescale of learning the global step-size compared to the covariance matrix.

Now we compare the length of the conjugate evolution path $\|\mathbf{p}_\sigma\|$ with the expected length under random selection, which is the expected length of a standard normally distributed random vector $\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$. If $\|\mathbf{p}_\sigma\| > \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$, the step-size should become larger, if $\|\mathbf{p}_\sigma\| < \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$, it

should be smaller, and for $\|\mathbf{p}_\sigma\| = \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$ the step-size should not change. As σ has to remain positive, this rule is implemented in a multiplicative way:

$$\sigma \leftarrow \sigma \exp \left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1 \right) \right) \quad (3.17)$$

Here, $0 < \frac{c_\sigma}{d_\sigma} < 1$ is a damping factor for which default settings can be used. What is the motivation for this rule? Hansen [2023] argues that this update should lead to approximately $\mathbf{C}^{-1}$ -conjugate update steps. More informally, one can think of the rule implementing the following heuristic. Assume that the recent steps were, due to the selection of the best policies, correlated in the sense that they had a tendency to go in the same direction, which implies $\|\mathbf{p}_\sigma\| > \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$. Then it would be better to have used a larger step-size in hindsight to achieve the same change in fewer but larger steps. That is, the current step-size is too small and should be increased. Conversely, assume that the recent steps were anti-correlated, which leads to $\|\mathbf{p}_\sigma\| < \mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$. The steps being anti-correlated means that the algorithm jumped more back-and-forth than expected, which happens when the search oversteps (local) optima. In this case, reducing the global step-size to make the search more local is a good strategy.

CMA-ES for RL

Now we have all the ingredients for a basic variant of the CMA-ES, shown in algorithm 5. More details about the CMA-ES can be found in the tutorial by Hansen [2023].

Parameters of the CMA-ES

Sometimes, the number of parameters and constants in the CMA-ES pseudocode discourages people from using the algorithm. However, the CMA-ES is actually very robust, and normally a user only has to specify a single parameter apart from the starting point $\boldsymbol{\theta}_{\text{init}}$, namely the initial global step-size σ_{init} . The global step-size plays the same role as the initial/maximum learning rate parameter in adaptive iterative gradient-based optimization methods. If it is too small, optimization can be slow; with a small initial σ the optimization tends to converge to the (local) minimum in the basin of attraction of the initial $\boldsymbol{\theta}_{\text{init}}$, which can be a feature if our goal is to fine-tune an already good policy $\pi_{\boldsymbol{\theta}_{\text{init}}}$. For the number of candidate solutions, the heuristic $\lambda = 4 + \lfloor 3 \ln n \rfloor$ can be used. If the RL is run on parallel hardware, then λ may be set depending on the available compute nodes. All other values in algorithm 5 are either constants, which follow from the mathematical derivations, or can be viewed as constants as robust default values can be used. These defaults were found by theoretical considerations and expensive experimental parameter studies.

Algorithm 5: CMA-ES

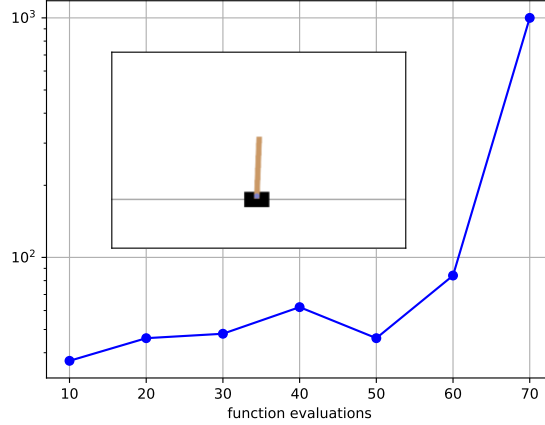
Input: $\theta = \theta_{\text{init}}, \sigma = \sigma_{\text{init}}, \lambda \geq 5$ (default $4 + \lfloor 3 \ln n \rfloor$)
Set: $c_1 \approx 2/n^2, c_\mu \approx \mu_{\mathbf{w}}/n^2, c_c \approx 4/n, c_\sigma \approx 1/\sqrt{n}, d_\sigma \approx 1,$
 $w_1 \geq w_2 \cdots \geq w_\lambda \geq 0$ with $\sum_{i=1}^\mu w_i = 1, \mu_{\mathbf{w}}^{-1} = \sum_{i=1}^\mu w_i^2 \approx 3/\lambda$

- 1 Initialize $\mathbf{p}_\sigma = \mathbf{p}_c = \mathbf{0}$ and covariance matrix $\mathbf{C} = \mathbf{I}$ (unit matrix)
- 2 **repeat**
 - // sample new policies
 - 3 **for** $l = 1, \dots, \lambda$ **do**
 - 4 $\mathbf{y}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$
 - 5 $\theta_l \leftarrow \theta + \sigma \mathbf{y}_l$
 - // evaluate new policies
 - 6 **for** $l = 1, \dots, \lambda$ **do** evaluate $J^{\text{MC}}(\pi_{\theta_l})$
 - // update mean
 - 7 $\mathbf{y}_{\mathbf{w}} = \sum_{i=1}^\mu w_i \theta_{i:\lambda}$
 - 8 $\theta \leftarrow \theta + c_\mu \sigma \mathbf{y}_{\mathbf{w}}$
 - // step size update
 - 9 $\mathbf{p}_\sigma \leftarrow (1 - c_\sigma) \mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_{\mathbf{w}}} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_{\mathbf{w}}$
 - 10 $\sigma \leftarrow \sigma \exp\left(\frac{c_\sigma}{d_\sigma} \left[\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}[\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|]} - 1\right]\right)$
 - // covariance matrix update
 - 11 $\mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_{\mathbf{w}}} \mathbf{y}_{\mathbf{w}}$
 - 12 $\mathbf{C} \leftarrow \mathbf{C} + c_\mu \sum_{i=1}^\lambda w_i (\mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T - \mathbf{C}) + c_1 (\mathbf{p}_c \mathbf{p}_c^T - \mathbf{C})$
- 13 **until** some stopping criterion is met

Example 3.1 An example of the CMA-ES optimizing an RNN to solve a POMDP is shown in fig. 3.2, where the algorithm is applied to non-Markovian pole balancing, see Heidrich-Meisner and Igel [2009] for details. This classic RL task goes back to the seminal paper by Barto et al. [1983], and it is implemented in the Gymnasium framework [Towers et al., 2024]. The goal is to balance a pole hinged on a wheeled cart, which can move on a finite length track, by exerting forces either left or right on the cart. The movements of the cart and the pole are restricted within the vertical plane. The state space of the underlying MDP comprises the position and velocity of the cart, as well as the pole angle and angular velocity. However, here we restrict the observations to the positions of the cart and the pole. This implies, for example, that the agent cannot decide from the state observation alone if the pole is falling down or rising up. Thus, we have to solve a POMDP. The agent receives a reward for each time step it balances the pole. If it succeeds for more than 500 steps, the return is set to 1000. In the depicted example, we used a simple RNN to represent the policy. The network had two hidden nodes with recurrent connections. The outputs of these nodes were processed by a (feedforward) neuron with a single output. The sign of this output

defined the action. No bias parameters were used. The network had 10 weights. A (5, 10)-CMA-ES was applied, that is, $\mu = 4$ and $\lambda = 10$. It took 7 iterations to find an RNN that solved the task.

Fig. 3.2 CMA-ES applied to non-Markovian pole balancing. The policy receives only the position of the cart and pole angle (i.e., not the velocities) and is represented by an RNN. In each iteration of the algorithm, $\lambda = 10$ candidate policies are sampled.



When using a neural network to directly represent the policy of an agent, the network weights parameterize the space of policies the network can realize. This parametrization is usually complex, there are strong correlations and thus the optimization problem is far from being separable. Hence, the ability of an optimization algorithm to detect dependencies between network parameters seems to be crucial for its performance. By efficiently adapting the covariance matrix of the mutation distribution, the CMA-ES accounts for correlations between the parameters. In doing so, the algorithm infers a promising search direction from the scalar reinforcement signals. It is often argued that RL problems are so difficult to solve because reinforcement signals do not provide a direction for adaptation, in contrast to a gradient in supervised learning (we will discuss how to estimate $\frac{\partial}{\partial \theta} J^{\text{MC}}(\theta)$ in detail in chapter 5). The evolution path in the CMA-ES can be viewed as a means of learning such a direction. This is a decisive difference between the CMA-ES and other direct search methods. If the dimensionality of the search space, that is, the number n of policy parameters, becomes too large, adapting the full covariance matrix with $O(n^2)$ becomes infeasible. However, the CMA-ES can be restricted to diagonal matrices.

Invariance properties, which define sets of problems for which an algorithm behaves equivalently, are an important design criterion for optimization techniques. As the CE method, the CMA-ES is invariant to order-preserving transformation of the objective function and translation of the search space if the starting point is transformed accordingly. In contrast to the CE method as described in algorithm 4, the CMA-ES is also invariant under rotation of the search space if the starting point is transformed accordingly; see Hansen

[2023] for a more detailed discussion of the invariance properties of the CMA-ES.

A fundamental difference of the CMA-ES compared to the CE and related methods is that the CMA-ES does not model the distribution of the selected candidate policies, which tends to decrease the variance of the search distribution in the direction of the steepest improvement (as discussed at the end of section 3.2), but focuses on the steps that led to the best policies, which tends to increase the variance in the direction of steepest improvement.

3.4 Direct search for reinforcement learning

The presented direct search methods seem to have a lot of attractive properties for RL. They are easy to apply and can be used if the model representing the policies is non-differentiable, whereas most methods require a differentiable structure for gradient-based optimization steps. The presented methods are based on ranking policies, which has been argued to be less susceptible to noise (which arises in RL for several reasons, e.g., due to random rewards and transitions, random initialization, and noisy state observations) compared to estimating a value function or a gradient of a performance measure w.r.t. policy parameters. In any case, it makes the algorithms invariant to order-preserving transformations of the reward (e.g., the algorithm behaves the same if the reward is measured in Euro or Danish kroner). Direct policy methods work well in non-Markovian environments as shown in example 3.1.

The CMA-ES is robust with respect to the tuning of hyperparameters. It is a variable-metric algorithm that learns an appropriate metric (by means of adapting the covariance matrix and, thereby, considering correlations between parameters) for searching for better policies. The use of information from past iterations and the efficient step-size adaptation make the CMA-ES highly efficient – for direct search method.

Unfortunately, direct search approaches to RL have severe drawbacks. They do not exploit intermediate rewards, just final Monte Carlo returns. That is, they do not exploit the available knowledge about which state transition given an action led to which immediate reward. They only look at the final sum of rewards. For example, assuming two actions $\{\text{left}, \text{right}\}$, the action-reward sequence $\langle \text{left}, 1, \text{left}, 1, \text{left}, 1, \text{right}, -1, \text{right}, -1, \text{right}, -1 \rangle$ clearly provides more information than the final accumulated reward of zero. That useful information is discarded in the computation of the objective shows that direct policy search is in general suboptimal. Direct optimization methods have no concept of states, a concept that can support credit assignment.

Example 3.2 An example given by Sutton and Barto [2018] shows nicely how the knowledge about the data coming from an MDP can influence learning. Assume the following eight state-reward sequences:

$$\{\langle A, 0, B, 0 \rangle, \langle B, 1 \rangle, \langle B, 1 \rangle, \langle B, 1 \rangle, \langle B, 1 \rangle, \langle B, 1 \rangle, \langle B, 1 \rangle, \langle B, 0 \rangle\}$$

The task is to learn a value function from these eight observations. Estimating $V(B) = \frac{6}{8} = \frac{3}{4}$ is straightforward. But what about $V(A)$? Every time we visited A , we got a zero reward, so $V(A) = \frac{0}{1} = 0$ is a reasonable estimate. This approach is referred to as a Monte Carlo estimate of the state value; see section 2.3. However, having the underlying MDP in mind, one could also argue that in 100 percent of all cases we transition from state A to state B , and thus $V(A) = V(B) = \frac{3}{4}$ in accordance with temporal difference learning.

There is no right or wrong here; the difference occurs because of the (small) finite sample. If we increase the number of sample trajectories, both the Monte Carlo estimation approach and temporal difference learning will converge to the same result.

Direct policy search requires that the interaction with the environment can be split into episodes. However, the biggest issue is the scaling w.r.t. the number of policy parameters.

Although direct policy search often performs surprisingly well for RL tasks when the number n of policy parameters is small, they break down if n increases. Evaluating $\ln n$ policies in each iteration becomes infeasible if n becomes large. And unfortunately, in deep RL the neural networks representing policies tend to get larger and larger. AlphaZero [Silver et al., 2017], an AI system for playing the game of Go, has in the order of 10^7 parameters, the network described by Berner et al. [2019] for playing the Dota 2 video game has approximately 158 million parameters, and RL for tuning large language models faces models that are large by definition. Thus, the general direct search methods are not sufficient, we need tailored approaches to do deep RL, and these approaches will be presented in the upcoming chapters.

Chapter 4

Deep Q -learning

Abstract This chapter scales tabular Q -learning to large state spaces through the use of function approximators such as deep neural networks. Function approximators add generalization to the RL algorithm in the sense that value estimates can be updated even if the corresponding states or state-action pairs were not recently observed. The chapter explains how approximate value functions can be learned via gradient-based optimization, deriving updates from temporal-difference errors. When the deep neural networks are employed to approximate value functions, stable learning based on noisy gradient information is challenging. To address this issues, the chapter introduces key innovations such as experience replay, robust loss functions, and delayed target networks. It also covers prioritized experience replay and discusses double Q -Learning as a method to reduce maximization bias in value estimates.

4.1 Function approximation

The algorithms described so far are based on the calculation of fixed points of value functions (V or Q). We assumed that we can somehow represent the value function in an appropriate way, for example, by storing it in a lookup table. However, value functions represented as tables are very limited. They are not suitable for very large (e.g., conceptually infinite) state and action spaces. Even if the tables can be stored, filling large tables requires a huge amount of observed transitions. Tabular representations do not generalize, in the sense that they do not take relations between states or state-action pairs into account, and do not extend the learning beyond the observed states and actions.

Example 4.1 Consider a Markovian version of the pole-balancing task in Example 3.1. We have taken action a in state $s_t = (x, \dot{x}, \vartheta, \dot{\vartheta})^T \in$

$\mathbb{R}^4$ (spatial and angular position and velocity) and observed r_{t+1} . Now assume $\varepsilon_s \in \mathbb{R}^4$ and the state $s'_t = s_t + \varepsilon_s$. For $\|\varepsilon_s\|$ small enough, it is reasonable to assume that $R_{s_t}^a \approx R_{s'_t}^a$. Similarly, assume that the action space A is continuous, and let the sign of an action $a \in \mathbb{R}$ indicate the direction and the magnitude $|a|$ encode the strength of the force applied. For small $\varepsilon_a \in \mathbb{R}$, we would expect $R_{s_t}^a \approx R_{s_t}^{a'}$ for $a' = a + \varepsilon_a$. Tabular representations do not exploit this similarity between states and actions.

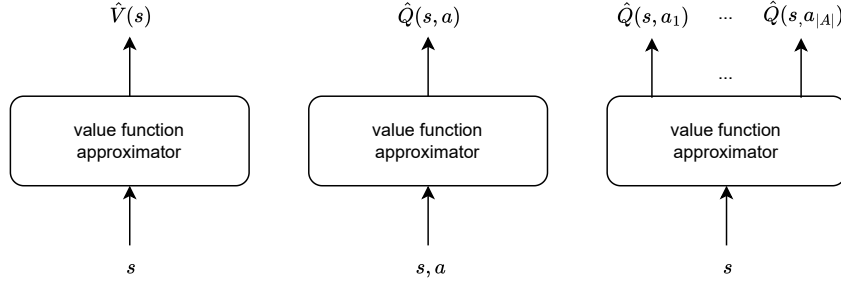


Fig. 4.1 Different ways how neural networks can be employed for function approximation. Given a state s as input, the leftmost network models a state value function $\hat{V}(s)$. The middle network gets the current state s and action a as inputs and predicts $\hat{Q}(s, a)$. The right network can be used in a setting with a discrete action space. Given a state as input, it predicts the $\hat{Q}$ -value function for all possible actions, $s \mapsto (\hat{Q}(s, a_1), \dots, \hat{Q}(s, a_{|A|}))^T$, that is, it maps from S to $\mathbb{R}^{|A|}$.

To overcome issues with large S and/or A , value functions are typically represented using parametrized function approximators. We can use function approximators with parameters $\mathbf{w}$ for the state-value function $\hat{V} : S \rightarrow \mathbb{R}$, see the illustration to the left in fig. 4.1, or the state-action value function $\hat{Q}_{\mathbf{w}} : S \times A \rightarrow \mathbb{R}$, see the illustration in the middle of fig. 4.1. Instead of updating individual entries in look-up tables, the parameters of the function approximator are changed. Common examples of function approximators in RL are:

- Look-up tables used in tabular Q -learning (see section 2.1.4) can be viewed as special cases of function approximators that do not generalize. Consider a discrete state space and a discrete action space, where we mapped states and action to non-negative integers, that is, $S = \{0, 1, \dots\}$ and $A = \{0, 1, \dots\}$. Then the function approximator with parameters $\mathbf{w} \in \mathbb{R}^{|S| \cdot |A|}$ defined by $\hat{Q}_{\mathbf{w}}(s, a) = w_{s+|S| \cdot a}$ corresponds to a look-up table.
- Assume a feature mapping $\phi : S \rightarrow \mathbb{R}^d$ and discrete actions $A = \{1, \dots, K\}$. Before neural networks became popular, it was common to consider function approximators linear in the features from an a priori fixed feature map:

$$\hat{Q}_{\mathbf{w}}(s, a_i) = \mathbf{w}_i^T \phi(s) \quad (4.1)$$

Here, $\mathbf{w} = (\mathbf{w}_1^T, \dots, \mathbf{w}_K^T)^T \in \mathbb{R}^{K \cdot d}$ with $\mathbf{w}_i^T \in \mathbb{R}^d$ for each $i = 1, \dots, K$. An offset parameter can be modeled by a feature mapping ϕ with an additional constant feature.

- For discrete actions $A = \{a_1, \dots, a_K\}$, instead of a mapping $S \times A \rightarrow \mathbb{R}$, we can learn

$$\hat{Q}_{\mathbf{w}} : S \mapsto \mathbb{R}^{|A|} \quad (4.2)$$

approximating $Q(s, a_i)$ by $[\hat{Q}_{\mathbf{w}}(s)]_i$, see the illustration on the right in fig. 4.1. In the following, we do not distinguish between $\hat{Q}_{\mathbf{w}}(s, a_i)$ and $[\hat{Q}_{\mathbf{w}}(s)]_i$.

- If the function approximators are deep neural networks, we talk of deep RL.

Non-linear function approximators such as neural networks are necessary because value functions are often highly non-linear. A small change in state can drastically change the expected return. Linear approximators cannot represent sharp decision boundaries or complex interactions between state variables.

Example 4.2 Consider a continuous version of the navigation task in example 2.1 visualized in fig. 2.2 with state space $S \subset \mathbb{R}^2$ encoding the spatial position in the world. For most $s, s' \in S$ with small $\varepsilon = \|s - s'\|$, it holds that $R_s^a \approx R_{s'}^a$, except, however, if the small ε makes a difference whether the agent steps on a black area or not.

Deep neural networks can learn highly non-linear functions, which allows them to model strong non-linear interactions, represent multimodal functions, and approximate discontinuities (e.g., near terminal states). They have proven to be applicable to large input spaces. Ideally, deep neural networks learn high-level representations (e.g., of goals and threats) from low-level (sensory) input. Prior knowledge about the task can be incorporated when choosing the network architecture. For example, the structure of CNNs (see section 2.2.4) supports learning spatial (and temporal) invariance properties that support spatial generalization of the RL agent. Recurrent neural networks as discussed in section 2.2.5 offer the ability to learn multi-step dependencies for addressing POMDPs.

Neural networks allow iterative optimization using stochastic gradient descent (see section 2.2.3), which we will use to learn value functions and, as we will see later, also policies. In the next section, we will discuss how to derive gradients for value function updates.

4.2 Gradient-based adaptation of value functions

We learn the value function approximators from data as in standard supervised learning (see section 2.2.2). Given some data

$$\mathcal{D} = \{((s_{t_1}, a_{t_1}), y_{t_1}), ((s_{t_2}, a_{t_2}), y_{t_2}), \dots\} , \quad (4.3)$$

which could be from a single episode or multiple episodes (in the latter case, we assume that all time steps from the episodes are concatenated, allowing us to use a single time index), we minimize, for example, the halved squared loss $\ell(y, y') = \frac{1}{2}(y - y')^2$

$$E_{\mathcal{D}}(\mathbf{w}) = \frac{1}{|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} \ell(y, \hat{Q}_{\mathbf{w}}(s, a)) = \frac{1}{2|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} (y - \hat{Q}_{\mathbf{w}}(s, a))^2 \quad (4.4)$$

similar to (2.50). But what should be the targets y ? In the same way as we derived (2.23), we get the Bellman error for a state-action value function approximation

$$\delta_{\text{BE}}^{\pi}(s, a) = \sum_{s' \in S} P_{s,s'}^a \left(R_{ss'}^a + \gamma \sum_{a' \in A} \pi(a' | s') \hat{Q}_{\mathbf{w}}(s', a') \right) - \hat{Q}_{\mathbf{w}}(s, a) \quad (4.5)$$

by replacing the state-action value function Q in the Bellman equation (2.13) by the approximation $\hat{Q}_{\mathbf{w}}$ and measuring the difference between the LHS and the RHS. If the Bellman error is zero, then $\hat{Q}_{\mathbf{w}}$ is the state-value function for the current policy π . Estimating the Bellman error from interactions following π gives the TD error

$$\delta_{\text{TD}}^{\pi}(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}) = \underbrace{r_{t+1} + \gamma \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1})}_{y_t} - \hat{Q}_{\mathbf{w}}(s_t, a_t) , \quad (4.6)$$

see (2.24). In (2.30), we already established a TD error based on the Bellman optimality condition:

$$\delta_{\text{TD}}^*(s_t, a_t, r_{t+1}, s_{t+1}) = \underbrace{r_{t+1} + \gamma \max_{a' \in A} \hat{Q}_{\mathbf{w}}(s_{t+1}, a')}_{y_t} - \hat{Q}_{\mathbf{w}}(s_t, a_t) \quad (4.7)$$

To get zero TD errors, we minimize the (halved) squared TD errors. Let us define $y_t = r_{t+1} + \gamma \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1})$ in the case of (4.6) and $y_t = r_{t+1} + \gamma \max_{a' \in A} \hat{Q}_{\mathbf{w}}(s_{t+1}, a')$ in the case of (4.7). Then $\frac{1}{2}(\delta_{\text{TD}}^{\pi}(s_t, a_t, r_{t+1}, s_{t+1}))^2 = \frac{1}{2}(y_t - \hat{Q}_{\mathbf{w}}(s_t, a_t))^2$ and

$$\begin{aligned}\nabla_{\mathbf{w}} \frac{1}{2} (\delta_{\text{TD}}^*(s_t, a_t, r_{t+1}, s_{t+1}))^2 &= \nabla_{\mathbf{w}} \frac{1}{2} (y_t - \hat{Q}_{\mathbf{w}}(s, a))^2 \\ &= (\hat{Q}_{\mathbf{w}}(s, a) - y_t) \nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s, a)\end{aligned}\quad (4.8)$$

under the assumption that $\nabla_{\mathbf{w}} y_t = 0$.

For the special case of linear function approximators, minimizing the error above can be formulated as a linear optimization problem, and for a given $\mathcal{D}$ an analytical solution can be calculated [Bradtke and Barto, 1996, Boyan, 2002]. However, in deep RL using non-linear function approximators, we iteratively minimize the error using gradient descent as in section 2.2.3. Each gradient step corresponds to a step in generalized policy iteration. eq. (4.9) leads to the steepest descent learning rule

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \underbrace{\frac{1}{|\mathcal{D}|} \sum_{((s,a),y) \in \mathcal{D}} \overbrace{(y - \hat{Q}_{\mathbf{w}}(s,a))}^{\text{TD error}} \nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s,a)}_{-\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})}} \quad (4.10)$$

with learning rate $\alpha > 0$.

Let us step back and consider tabular function approximation, that is, discrete A and S and a table with $|A| \times |S|$ entries corresponding to $|A| \times |S|$ -dimensional parameter vector $\mathbf{w}$ as above, where $\hat{Q}_{\mathbf{w}}(s, a) = w_{s+|S|\cdot a}$. For a single tuple $((s_t, a_t), y_t)$ we have

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y_t - \hat{Q}_{\mathbf{w}}(s_t, a_t)) \nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s_t, a_t) \quad (4.11)$$

For our tabular function approximator, $[\nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}_{\text{target}}}(s, a)]_i$ is one for $i = s_t + |S| \cdot a_t$ corresponding to (s_t, a_t) and zero otherwise, the different states in a table do not influence each other. Thus, (4.11) only updates the component $w_{s_t+|S|\cdot a_t}$. We can replace $w_{s_t+|S|\cdot a_t}$ by $\hat{Q}_{\mathbf{w}}(s, a)$ and recover SARSA

$$\hat{Q}_{\mathbf{w}}(s_t, a_t) \leftarrow \hat{Q}_{\mathbf{w}}(s_t, a_t) + \alpha \underbrace{[r_{t+1} + \gamma \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1})]}_{y_t} - \hat{Q}_{\mathbf{w}}(s_t, a_t) \quad (4.12)$$

and Q -learning

$$\hat{Q}_{\mathbf{w}}(s_t, a_t) \leftarrow \hat{Q}_{\mathbf{w}}(s_t, a_t) + \alpha \underbrace{[r_{t+1} + \gamma \max_a \hat{Q}_{\mathbf{w}}(s_{t+1}, a)]}_{y_t} - \hat{Q}_{\mathbf{w}}(s_t, a_t) \quad , \quad (4.13)$$

see (2.27) and (2.31), respectively. Thus, the tabular RL algorithms fit the presented gradient-based adaptation framework.

However, there is a problem in the above derivation. We assumed that y_t is independent of $\mathbf{w}$, and this is clearly not true in general for $y_t = r_{t+1} + \gamma \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1})$ and $y_t = r_{t+1} + \gamma \max_{a' \in A} \hat{Q}_{\mathbf{w}}(s_{t+1}, a')$. In the former case,

for example, the update in (4.11) would correctly read

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(y_t - \hat{Q}_{\mathbf{w}}(s_t, a_t) \right) \left(\nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s_t, a_t) - \gamma \nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1}) \right) . \quad (4.14)$$

In the tabular case, this would not make much difference for the update of $\hat{Q}_{\mathbf{w}}(s_t, a_t)$, because for $(s_t, a_t) \neq (s_{t+1}, a_{t+1})$ we have $\nabla_{\mathbf{w}} \hat{Q}_{\mathbf{w}}(s_{t+1}, a_{t+1}) = 0$, and for the case of $(s_t, a_t) = (s_{t+1}, a_{t+1})$, the new term has the effect of reducing α by a factor of $1 - \gamma$. However, (4.14) also suggests updates of $\hat{Q}_{\mathbf{w}}(s, a)$ for $(s, a) \neq (s_t, a_t)$ even in the tabular case. Anyway, in deep RL we generally do not consider a dependency of y_t on $\mathbf{w}$, which can be justified when introducing a target network, see below.

In general, the convergence of RL using gradient-based optimization of value functions cannot be guaranteed. For linear approximators, convergence has been proven under very restrictive conditions [Tsitsiklis and Van Roy, 1997]. It can be shown that even small deviations from these conditions can lead to divergence [Merke and Schoknecht, 2002]. However, as we cannot establish convergence to the global optimum for supervised training of deep neural networks, theoretical guaranties for deep RL are conceptually difficult to obtain anyway.

4.3 Dealing with terminal states

With the results above, we can derive Q -learning with a value function approximator learned using iterative gradient descent by adapting algorithm 2. We could simply replace line 7 in algorithm 2 by computing the target y and then performing the update of $\hat{Q}_{\mathbf{w}}$ by

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \ell(y, \hat{Q}_{\mathbf{w}}(s, a)) , \quad (4.15)$$

where ℓ is typically the squared loss. We know that terminal states cannot be left and any action in a terminal state leads to zero reward. Therefore, we need not resort to approximated values when computing $\max_{a'} \hat{Q}_{\mathbf{w}}(s'_i, a')$ if s' is a terminal state. Thus, we define the target

$$y = r + \begin{cases} 0 & \text{if } s' \text{ is a terminal state} \\ \gamma \max_{a'} \hat{Q}_{\mathbf{w}}(s'_i, a') & \text{otherwise} \end{cases} \quad (4.16)$$

$$= r + (1 - d) \gamma \max_{a'} \hat{Q}_{\mathbf{w}}(s', a') . \quad (4.17)$$

In the last line, we introduce an indicator variable d that is zero except if s' is a terminal state or the episode is truncated after transitioning to s' . We assume that d is observed together with the reward r and s' (as is the case in many software frameworks for deep RL).

4.4 Gradient accumulation and mini-batch updates

Updating a neural network using a single (stochastic) gradient step (4.15) based on a single observed transition (i.e., $|\mathcal{D}| = 1$), as typically done in generalized policy iteration when using tabular value functions (e.g., in tabular Q -learning), is not recommended in deep RL because the variance of the gradient estimate is too high. This can lead to steps that severely deteriorate the value function estimate and to forgetting important information gained in previous steps.

A straight-forward way to address this issue is to accumulate the gradients over T_{update} steps

$$\sum_{i=0}^{T_{\text{update}}-1} \nabla_{\mathbf{w}} \ell \left(y_{t-i}, \hat{Q}_{\mathbf{w}}(s_{t-i}, a_{t-i}) \right) \quad (4.18)$$

and to perform the gradient-based updates only every T_{update} steps based on the accumulated gradients (4.18). If the gathering of experience is parallelized, this update can be easily parallelized [Mnih et al., 2016]. algorithm 7 shows a variant of Q -learning that uses gradient accumulation.

This can be viewed as a special case of mini-batch learning, where the mini-batch gradient $\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})$ is computed for

$$\mathcal{D} = \{((s_{t-T_{\text{update}}+1}, a_{t-T_{\text{update}}+1}), y_{t-T_{\text{update}}+1}), \dots, ((s_t, a_t), y_t))\} . \quad (4.19)$$

However, typically mini-batches do not simply contain the experiences from the last T_{update} steps as discussed in the next section.

4.5 Experience replay

The standard solution to create mini-batches is experience replay. Experiences (also referred to as observations or transitions) $\langle s, a, r, s', d \rangle$ are stored in a finite FIFO (first-in-first-out) buffer M . If the buffer is full, the oldest element will be removed if a new one is added. The mini-batches $\mathcal{D}$ for gradient steps are sampled from M [Mnih et al., 2015].

Storing experiences in a buffer allows the learning algorithm to use (potentially costly) observations several times. In supervised learning, neural networks typically need several training epochs to converge, that is, the same training data points are presented to the network several times. The same holds for RL, where the optimization problem additionally changes when the policy changes (e.g., because $\hat{Q}$ changes). It is important to randomly sample the mini-batches from a larger buffer. Simply combining the last observations in a mini-batch $\mathcal{D}$ as in (4.19) is not a good strategy because then the elements in the mini-batch may be too similar to each other because they

typically stem from the same episode, which may lead to a bias in the gradient estimation. A minimal version of deep Q-learning with a replay buffer is shown in algorithm 6.

Algorithm 6: Simple deep Q-learning with experience replay

Input: network $\hat{Q}_{\mathbf{w}}$ with parameters $\mathbf{w}$, experience buffer M , batch size n_{batch} , discount factor γ , learning rate α

```

1 initialize experience buffer  $M$ 
2 foreach episode do
3   initialize  $s \sim p_{\text{start}}$ 
4   repeat
5     choose  $a$  based on  $\hat{Q}_{\mathbf{w}}$  // e.g.,  $\epsilon$ -greedy
6     take action  $a$ , observe  $r, s', d$ 
7     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
8     sample batch  $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
9     for  $i = 1, \dots, n_{\text{batch}}$  do  $y_i = r_i + (1 - d)\gamma \max_{a'} \hat{Q}_{\mathbf{w}}(s'_i, a')$ 
        // update network
10     $\mathbf{w} \leftarrow \mathbf{w} - \alpha \sum_{i=1}^{n_{\text{batch}}} \nabla_{\mathbf{w}} \ell(y_i, \hat{Q}_{\mathbf{w}}(s_i, a_i))$ 
11     $s \leftarrow s'$ 
12 until  $s$  is terminal
```

Using mini-batches is crucial in deep RL. As a side effect, it also allows to employ advanced gradient-based optimization techniques, in particular methods that adjust individual step-sizes such as Adam, see section 2.2.3. When doing the gradient-based update, the RL algorithm performs $n_{\text{gradient-steps}}$ steps of the optimization algorithm. There is a lot of flexibility in how interactions with the environment and gradient-based updates can be interleaved. If an update is performed after each interaction, as in many example algorithms in this text, then often only a single step is taken ($n_{\text{gradient-steps}} = 1$). However, if the gradient-based optimization happens more rarely, then typically $n_{\text{gradient-steps}} > 1$ as in algorithm 14.

The idea of storing all experiences to allow batch learning of neural networks representing $\hat{Q}$ was already suggested in Neural Fitted Q-Learning (NFQ) by Riedmiller [2005], however, without removing (or prioritizing) samples.

Data in a batch $\mathcal{D}$ uniformly sampled from the experience replay buffer are not i.i.d. They are not independent of each other because the states encountered during an episode (the states in a trajectory) are not independent of each other, and the elements in the buffer stem from trajectories. The states are also not sampled from the same state distribution, because the state distribution depends on the policy; and the policy is updated after each or after $T_{\text{target-update}}$ interactions with the environment. The first issue is mitigated by the random sampling, the second by removing old experiences from the buffer.

Prioritized experience replay

Not all transitions in M are, in general, equally helpful to improve the policy. Instead of uniformly sampling from M , Schaul et al. [2016] proposed a non-uniform distribution over the elements in the buffer. The idea is to give experiences that led to large temporal difference errors $|\delta_{\text{TD}}^*|$ a higher priority. If a stored experience $\langle s, a, r, s', d \rangle$ did not give a large TD error (i.e., it did not change $\hat{Q}$ much), this means that $\hat{Q}$ currently predicts the value for (s, a) rather accurately. Thus, the focus should be on training on examples where $|\delta_{\text{TD}}|$ was large.

Let n_{buffer} be the number of elements in the buffer, and let the experiences in M be indexed by $j = 1, \dots, n_{\text{buffer}}$ (to distinguish from the indexing of the elements in a mini-batch by i , e.g., in algorithm 6). Each of the elements gets an initial priority $\tilde{p}_j$, which is updated every time the pattern is used in a mini-batch. In the most common setting, it is set to be the absolute TD error observed for the pattern (plus some small constant to avoid a zero priority). This prioritized experience replay (PER) can speed up learning by presenting informative transitions more often.

The elements in M are sampled with probabilities $p_j = \tilde{p}_j^{\alpha_{\text{PER}}} / \sum_{k=1}^{n_{\text{buffer}}} \tilde{p}_k^{\alpha_{\text{PER}}}$ with $\alpha_{\text{PER}} \in [0, 1]$ controlling the strength of prioritization, which is cooled during learning (see section 2.3.1). Importance-sampling weights are used to mitigate the bias introduced by the non-uniform sampling. The strength of the effect of the importance weights is controlled by a parameter β_{PER} . Let n_{buffer} be the number of elements in M . Then uniform sampling would correspond to probabilities n_{buffer}^{-1} . This suggests importance weights $n_{\text{buffer}}^{-1} / p_j = (n_{\text{buffer}} \cdot p_j)^{-1}$. Introducing the parameter $\beta_{\text{PER}} \in [0, 1]$ we define $\tilde{w}_j = (n_{\text{buffer}} \cdot p_j)^{-\beta_{\text{PER}}}$. The effect of importance weighting is often increased from $\beta_{\text{PER}} < 1$ to $\beta_{\text{PER}} = 1$ during learning. To stabilize the learning, the weights are normalized to a maximum of one by dividing by $\max_j \tilde{w}_j$, that is, the weighting can only reduce the change in network parameters. This gives the final weights $w_{\text{PER},j} = \tilde{w}_j / \max_j \tilde{w}_j$, which are then used to weight the gradient update, for example, by using $w_{\text{PER},i} \nabla_{\mathbf{w}} \ell(y_i, \hat{Q}_{\mathbf{w}}(s_i, a_i))$ in line 10 in algorithm 6.

4.6 Delayed Q -update

Updating the $\hat{Q}_{\mathbf{w}}$ network also changes the targets for updating the $\hat{Q}_{\mathbf{w}}$ network, which leads to instabilities during learning. Therefore, we introduce a different target network $\hat{Q}_{\mathbf{w}_{\text{target}}}$ for computing the temporal difference errors for $\hat{Q}_{\mathbf{w}}$, which is also referred to as the online network. The target computation becomes:

$$y_i = r_i + (1 - d) \gamma \max_{a'} \hat{Q}_{\mathbf{w}_{\text{target}}}(s'_i, a') \quad (4.20)$$

Note that by defining the targets like this, $\nabla_{\mathbf{w}} y$ is indeed zero, as the computation does not depend on the parameters $\mathbf{w}$ of $\hat{Q}_{\mathbf{w}}$. This addresses the issue with the gradient computation discussed at the end of section 4.2.

The target network is updated on a slower time scale [Mnih et al., 2015, 2016]. We use the same network architecture for $\hat{Q}_{\mathbf{w}}$ and $\hat{Q}_{\mathbf{w}_{\text{target}}}$, only the parameters vary; we distinguish between $\mathbf{w}$ and $\mathbf{w}_{\text{target}}$.

There are two common ways to implement delayed updates. Firstly, we can copy the parameters (e.g., $\mathbf{w}_{\text{target}} \leftarrow \mathbf{w}$) every $T_{\text{target-update}}$ steps [Mnih et al., 2016]. Algorithm 7 uses gradient accumulation and updates the online network every T_{update} steps and the target network every $T_{\text{target-update}}$ steps,

Algorithm 7: *Q-learning with gradient accumulation*

Input: policy π_{θ} , value function approximator $\hat{Q}_{\mathbf{w}}$, update intervals T_{update} and $T_{\text{target-update}}$ for online and target networks, parameters of gradient-based learning algorithm

```

1  $\mathbf{w}_{\text{target}} \leftarrow \mathbf{w}$ 
2  $\theta_{\text{target}} \leftarrow \theta$ 
3  $\delta_{\mathbf{w}} \leftarrow 0$  // accumulated gradient
4  $s \sim p_{\text{start}}$ 
5  $t \leftarrow 0$ 
6 repeat
7   take action  $a$   $\epsilon$ -greedy based on  $\hat{Q}_{\mathbf{w}}(s, a)$ 
8   observe  $r$ , and  $s'$  with indicator  $d$ 
9    $y = r + (1 - d)\gamma \max_a \hat{Q}_{\mathbf{w}_{\text{target}}}(s, a)$ 
   // accumulate gradients
10   $\delta_{\mathbf{w}} \leftarrow \delta_{\mathbf{w}} + \frac{1}{2} \nabla_{\mathbf{w}} (y - \hat{Q}_{\mathbf{w}}(s, a))^2$ 
11  if  $d = 0$  then  $s \leftarrow s'$ 
12  else  $s \sim p_{\text{start}}$ 
13   $t \leftarrow t + 1$ 
   // update target value function
14  if  $t \bmod T_{\text{target-update}} = 0$  then
15     $\mathbf{w}_{\text{target}} \leftarrow \mathbf{w}$ 
   // update value function
16  if  $t \bmod T_{\text{update}} = 0$  then
17    update  $\mathbf{w}$  based on  $\delta_{\mathbf{w}}$ 
18     $\delta_{\mathbf{w}} \leftarrow 0$  // reset gradient accumulation
19 until until stopping criterion is met

```

Secondly, we can smoothly blend the parameters, for example by setting $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ for $\rho \in]0, 1[$, which is also referred to as Polyak averaging.

Algorithm 8 shows deep Q-learning similar to Mnih et al. [2015] but with Polyak averaging instead of updating the target network after a fixed number

of steps. Algorithms of this type are referred to as DQN, which stands for a deep Q -network.

Algorithm 8: Deep Q -learning (DQN)

Input: networks $\hat{Q}_{\mathbf{w}}$ and $\hat{Q}_{\mathbf{w}_{\text{target}}}$, experience buffer M of size n_{buffer} , discount factor γ , parameters controlling exploration $(\epsilon_{\text{start}}, \epsilon_{\text{stop}}, \beta_{\epsilon})$, parameters of gradient-based learning algorithm, target network blending parameter ρ

```

1 initialize experience buffer  $M$  // e.g., by running  $> n_{\text{buffer}}$  episodes
  w/o learning)
2 foreach episode  $e = 1, \dots$  do
3   initialize  $s \sim p_{\text{start}}$ 
4   repeat
5      $\epsilon = \epsilon_{\text{stop}} + (\epsilon_{\text{start}} - \epsilon_{\text{stop}})e^{-\beta_{\epsilon}(e-1)}$ 
6     choose  $a$  based on  $\hat{Q}_{\mathbf{w}}$   $\epsilon$ -greedy
7     take action  $a$ , observe  $r, s', d$ 
8     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
9     sample  $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
      // set targets
10    for  $i = 1, \dots, n_{\text{batch}}$  do  $y_i = r_i + (1 - d_i)\gamma \max_{a'} \hat{Q}_{\mathbf{w}_{\text{target}}}(s'_i, a')$ 
      // compute loss
11     $E_{\mathcal{D}}(\mathbf{w}) = \sum_{i=1}^{n_{\text{batch}}} \ell(y_i, \hat{Q}_{\mathbf{w}}(s_i, a_i))$ 
      // update value function
12    update  $\mathbf{w}$  based on  $\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})$  // (e.g., AdamW step)
      // update target value function
13     $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ 
14     $s \leftarrow s'$ 
15  until  $s$  is terminal

```

Example 4.3 We apply deep Q -learning to the LunarLander task described in example 2.2. We used the neural network architecture visualized in fig. 2.4. We employ AdamW (algorithm 3) with learning rate $\alpha = 0.001$ to adapt the $\hat{Q}_{\mathbf{w}}$ -network. Experiences are stored in a buffer with capacity $n_{\text{buffer}} = 8192$. The mini-batch size is $n_{\text{batch}} = 128$.

We select the actions following the ϵ -greedy policy. We decrease the exploration exponentially from ϵ_{start} to ϵ_{stop} following

$$\epsilon = \epsilon_{\text{stop}} + (\epsilon_{\text{start}} - \epsilon_{\text{stop}})e^{-\beta_{\epsilon}(e-1)} \quad (4.21)$$

with decay rate β_{ϵ} and e being the episode counter. We use exploration probability at start $\epsilon_{\text{start}} = 1.0$, minimum exploration probability $\epsilon_{\text{stop}} = 0.0001$, and exponential decay rate for exploration $\beta_{\epsilon} = 0.05$.

The target network is updated using Polyak averaging. The results for 25 trials for each choice of blending parameter $\rho \in \{0.01, 0.1, 0.2, 0.5, 0.9\}$ are shown in fig. 4.2.

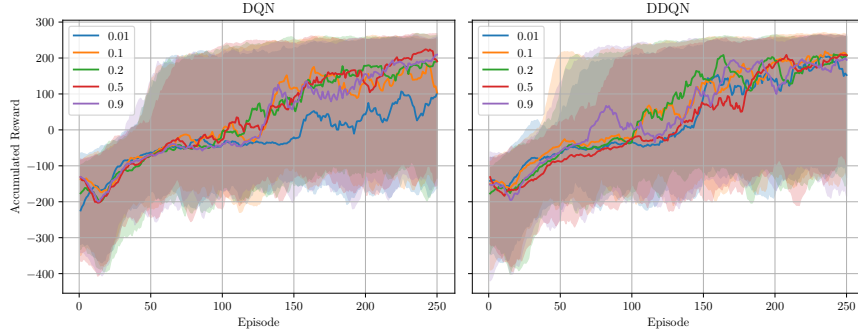


Fig. 4.2 Deep Q -learning following algorithm 8 (DQN, left) and algorithm 9 (DDQN, right) applied to the LunarLander task as described in example 4.3 for different choices of the blending parameter ρ when updating the target network. Shown are the medians over 25 trials, the curves are smoothed with a box filter of width 10. The shaded areas indicate the 10th and 90th percentile.

4.7 Double Q -learning

Double Q -learning is a common modification of deep Q -learning and other deep RL algorithms. It addresses maximization bias, also called overestimation bias, an issue in Q -learning pointed out by [Thrun and Schwartz \[1994\]](#) and [van Hasselt \[2010\]](#), which is rooted in taking the maximum over noisy value estimates.

Let us first describe the issue independent of RL. Consider n_A random variables $X_1, \dots, X_{n_A}$. We are interested in $\max_i \mathbb{E}[X_i]$. For each X_i , we have n_i i.i.d. realizations $\{x_{i,1}, \dots, x_{i,n_i}\}$ with average $\hat{x}_i$. This average is also a random variable, which we denote by $\hat{X}_i$. From Jensen's inequality, we know $\max_i \mathbb{E}[X_i] = \max_i \mathbb{E}[\hat{X}_i] \leq \mathbb{E}[\max_i \hat{X}_i]$. That is, $\max_i \hat{x}_i$ tends to overestimate $\max_i \mathbb{E}[X_i]$.¹

Example 4.4 Consider 100 fair coins $(X_1, \dots, X_{100})$, each giving one (head) and zero (tail) with equal probability. We have $\mathbb{E}[X_1] = \mathbb{E}[X_2] =$

¹ For the same reason, in supervised learning, the error on a validation data set is not an unbiased estimate of the model's performance if the error was used to select the model from a set of models; the estimate can be overoptimistic.

$\dots = \mathbb{E}[X_{100}] = \max_i \mathbb{E}[X_i] = 0.5$. You toss each of them 10 times ($n_1, n_2, \dots, n_{100} = 10$) and write down the average number of ones $\hat{x}_i$ for each coin. With very high probability, there will be at least one coin $i^\dagger$ with more than 5 ones. That is, $\max_i \hat{x}_i$ can be expected to be greater than 0.5.

We can address this issue by maintaining two i.i.d. samples from each X_i with means $\hat{X}_i$ and $\hat{X}'_i$, respectively. We use the first one to estimate the best variable by selecting an index $\hat{I} \in \arg \max_i \hat{X}_i$ and the second one to estimate the expectation of the selected variable by $\hat{X}'_{\hat{I}}$. For each i , we have $\mathbb{E}[\hat{X}'_{\hat{I}} | \hat{I} = i] = \mathbb{E}[\hat{X}'_i] = \mathbb{E}[\hat{X}_i] = \mathbb{E}[X_i]$, where the first equality holds because the $\hat{X}'_i$ are independent of the process that selects an index. Thus, $\hat{X}'_{\hat{I}}$ is an unbiased estimator of the true mean of the selected index. If $\hat{I}$ is equal to a maximizer $i^* \in \arg \max_i \mathbb{E}[X_i]$, then $\hat{X}'_{i^*}$ is an unbiased estimator of $\max_i \mathbb{E}[X_i]$. However, because it is based on finite samples, $\hat{i}^*$ may not belong to $\arg \max_i \mathbb{E}[X_i]$. We have $\mathbb{E}[\hat{X}_{\hat{I}}] = \sum_{i=1}^{n_A} \mathbb{E}[X_i | \hat{I} = i] \Pr\{\hat{I} = i\} = \sum_{i=1}^{n_A} \mathbb{E}[X_i] \Pr\{\hat{I} = i\} \leq \max_i \mathbb{E}[\hat{X}_i]$. Thus, the procedure may lead to an underestimation of $\max_i \mathbb{E}[\hat{X}_i]$.

The Q -learning algorithm, see algorithm 2, uses maximum operations over estimated values, which are noisy estimates due to the limited sample size and environmental stochasticity. Also computing the target involves a maximum operation. The resulting overestimation can lead to suboptimal behavior, as the agent may focus on suboptimal actions that appear better randomly due to overestimated $\hat{Q}$ -values. Therefore, van Hasselt [2010] propose double Q -learning. The algorithm keeps track of two estimators for the value function. These can then be used to decoupling action selection – taking the maximum – and evaluation of the selected action.

In deep Q -learning with delayed Q -update, we already keep track of two value function approximators, the online network $\hat{Q}_{\mathbf{w}}$ and the target network $\hat{Q}_{\mathbf{w}_{\text{target}}}$. Thus, van Hasselt et al. [2016] suggest to exploit this and propose the double deep Q -learning network (DDQN), where the action is selected by the online network

$$a^* = \arg \max_a \hat{Q}_{\mathbf{w}}(s', a) ,$$

but the target network evaluates it:

$$y = r + \gamma \hat{Q}_{\mathbf{w}_{\text{target}}}(s', a^*)$$

This DDQN approach is very common, it is shown in algorithm 9 with Polyak averaging; see example 4.3 and fig. 4.2 for an example application.

An alternative is to maintain two estimators for the value function and take their minimum when computing targets [Fujimoto et al., 2018] as will be used in section 6.6.

Another approach to combat overestimation bias is the “clipped double- Q trick”. Here the RL agent maintains two value function approximators,

Algorithm 9: Double deep Q -learning (DDQN)

Input: networks $\hat{Q}_{\mathbf{w}}$ and $\hat{Q}_{\mathbf{w}_{\text{target}}}$ with parameters $\mathbf{w}$ and $\mathbf{w}_{\text{target}}$, experience buffer M of size n_{buffer} , discount factor γ , parameters controlling exploration $(\epsilon_{\text{start}}, \epsilon_{\text{stop}}, \beta_{\epsilon})$, parameters of gradient-based learning algorithm, target network blending parameter ρ

```

1 initialize experience buffer  $M$  // e.g., by running  $> n_{\text{buffer}}$  episodes
  w/o learning)
2 foreach episode  $e = 1, \dots$  do
3   initialize  $s \sim p_{\text{start}}$ 
4   repeat
5      $\epsilon = \epsilon_{\text{stop}} + (\epsilon_{\text{start}} - \epsilon_{\text{stop}})e^{-\beta_{\epsilon}(e-1)}$ 
6     choose  $a$  based on  $\hat{Q}_{\mathbf{w}}$   $\epsilon$ -greedy
7     take action  $a$ , observe  $r, s', d$ 
8     store  $\langle s, a, r, s', d \rangle$  in  $M$ 
9     sample  $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
      // set targets
10    for  $i = 1, \dots, n_{\text{batch}}$  do
       $y_i = r_i + (1 - d_i)\gamma\hat{Q}_{\mathbf{w}_{\text{target}}}(s'_i, \arg \max_{a'} \hat{Q}_{\mathbf{w}}(s', a'))$ 
      // compute loss
11     $E_{\mathcal{D}}(\mathbf{w}) = \sum_{i=1}^{n_{\text{batch}}} \ell(y_i, \hat{Q}_{\mathbf{w}}(s_i, a_i))$ 
      // update value function
12    update  $\mathbf{w}$  based on  $\nabla_{\mathbf{w}} E_{\mathcal{D}}(\mathbf{w})$  // e.g., AdamW step
      // update target value function
13     $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ 
14     $s \leftarrow s'$ 
15  until  $s$  is terminal
  
```

$\hat{Q}_{\mathbf{w}_1}$ and $\hat{Q}_{\mathbf{w}_2}$ plus the corresponding target networks, $\hat{Q}_{\mathbf{w}_{\text{target},1}}$ and $\hat{Q}_{\mathbf{w}_{\text{target},2}}$ for delayed updating. The actor uses the minimum of the two value estimates when computing targets Fujimoto et al. [2018]. That is, the target computation becomes

$$y = r + \gamma \min \left(\hat{Q}_{\mathbf{w}_{\text{target},1}}(s, a), \hat{Q}_{\mathbf{w}_{\text{target},2}}(s, a) \right) . \quad (4.22)$$

This update scheme is referred to as the clipped double- Q trick because taking the minimum “clips” optimistic estimates. It is an important ingredient of the Soft Actor Critic (SAC) algorithm, see section 6.6. The clipped double- Q trick goes beyond double Q -learning and even introduces a (slight) underestimation bias.

4.8 Making the loss function more robust

The magnitude of a single observation can be very large, for example, when the state has not been visited before or an explorative action was taken. We want to limit the influence of such single incidents on the loss function, because they can dominate the update and could lead to forgetting important information.

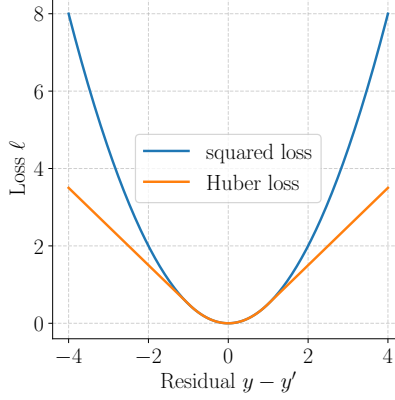


Fig. 4.3 Comparison of (halved) squared loss and Huber loss with parameter $\delta_{\text{Huber}} = 1$.

Limiting the impact of a single update step can be achieved by clipping. The clip operator is defined by

$$\text{clip}(x, x_{\text{low}}, x_{\text{high}}) = \min(x_{\text{high}}, \max(x, x_{\text{low}})) = \begin{cases} x_{\text{high}} & \text{if } x > x_{\text{high}} \\ x_{\text{low}} & \text{if } x < x_{\text{low}} \\ x & \text{otherwise} \end{cases} \quad (4.23)$$

One can do either component-wise gradient-clipping

$$\begin{aligned} \nabla_{w_i} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i})) \\ \leftarrow \text{clip}\left(\nabla_{w_i} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i})), g_{\min}, g_{\max}\right) \end{aligned} \quad (4.24)$$

or (better) clipping the norm

$$\begin{aligned} & \nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i})) \\ & \leftarrow \nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i})) \frac{\min\{g_{\max}, \|\nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i}))\|\}}{\|\nabla_{\mathbf{w}} \ell(y_{t+i}, \hat{Q}_{\mathbf{w}}(a_{t+i}, s_{t+i}))\|} \end{aligned} \quad (4.25)$$

with $g_{\min}, g_{\max} > 0$. We can also clip the loss function directly:

$$\ell(y_{t+1}, \hat{Q}_{\mathbf{w}}(a_{t+1}, s_{t+1})) \leftarrow \text{clip}\left(\ell(y_{t+1}, \hat{Q}_{\mathbf{w}}(a_{t+1}, s_{t+1})), \ell_{\min}, \ell_{\max}\right) \quad (4.26)$$

Instead or in addition to modifying the loss function by clipping, one can directly employ a robust loss function, for example, the Huber-loss

$$\ell_{\text{Huber}}(y, y') = \begin{cases} \frac{1}{2}|y - y'|^2, & \text{if } |y - y'| \leq \delta_{\text{Huber}}, \\ \delta_{\text{Huber}}(|y - y'| - \frac{1}{2}\delta_{\text{Huber}}), & \text{otherwise} \end{cases} \quad (4.27)$$

with parameter $\delta_{\text{Huber}} > 0$. To avoid too a strong impact of large differences between the predicted value and the target, the Huber loss switches from a quadratic loss to a linear absolute loss for differences with magnitudes greater than δ_{Huber} .

Another change in the loss function to increase the robustness of training is using weight-decay, see section 2.2.3.

4.9 Rainbow

An important baseline for deep Q -learning is the Rainbow algorithm [Hessel et al., 2018]. In 2018, Hessel et al. combined many promising modifications of the original DQN approach into one algorithm. They showed that using all these individual advances together leads to superior results on the common Atari benchmark [Bellemare et al., 2013].

Rainbow is based on DDQN to avoid overestimation bias van Hasselt et al. [2016], see section 4.7. It uses prioritized experience replay Schaul et al. [2016] section 4.5 to exploit data more efficiently.

It uses multi-step bootstrapping as discussed in section 2.1.4 and section 6.2.1 in the context of the algorithms by Mnih et al. [2016].

Furthermore, the learning objective is changed to distributional Q -learning [Bellemare et al., 2017]. The critic in distributional Q -learning does not predict the expectation of the return (i.e., the expected accumulated future reward), but the distribution of the returns.

In addition, Rainbow considers two neural network architecture elements that were developed for neural value function approximators in RL. Rainbow uses noisy networks [Fortunato et al., 2018, a preprint was already available in 2017], which contain stochastic layers to support exploration.

Another modification of the neural network architecture for Q -learning is called dueling network [Wang et al., 2016]. After a CNN encoder, the computation is split into two heads, one predicting the state value $\hat{V}(s)$ and the other head predicting the advantage of executing action a in state s , see Combining these heads then gives a prediction for $\hat{Q}(s, a)$.

The ablation study by Schaul et al. [2016] showed that prioritized replay and multi-step TD learning were the most important features of Rainbow. In later stages of learning, the removal of distributional Q -learning also significantly reduced performance. Using the dueling network architecture and double Q -learning did not have a significant impact in the experiments.

Chapter 5

Policy gradient methods

Abstract Policy gradient methods optimize the agent’s behavior using gradient-based optimization. They follow the policy gradient, which is the gradient of the expected return with respect to the policy parameters. This chapter states and proves the policy gradient theorems in different settings. These results are the basis for estimating policy gradients from sampled experience. The REINFORCE algorithm as the most basic policy gradient method is introduced.

5.1 Gradient-based policy optimization

Policy gradient methods maintain a parameterized policy π_{θ} and optimize its parameters θ using gradient-based optimization following the gradient of the expected return with respect to θ . This is in contrast to pure value function approaches, which estimate the value of states or state-action pairs (i.e., learn approximate V - or Q -functions) to directly derive the policy (e.g., choose the action a in state s that maximizes $\hat{Q}(s, a)$).

What are the advantages of policy gradient methods compared to value-function approaches? Policy gradient methods are often preferred for high-dimensional state and action spaces, including continuous states and actions. Value function approaches struggle with continuous action spaces, because selecting the best action becomes impractical. [Peters \[2010\]](#) adds “that the policy representations can be chosen so that it is meaningful for the task and can incorporate domain knowledge, that often fewer parameters are needed in the learning process than in value-function based approaches” and that policy gradient methods can be employed in both model-based and model-free RL. Policy gradient methods allow policies to change gradually, while value-function approaches can violate the principle of strong causality in the sense that small changes may have drastic effects. A small change in a value function can change which action maximizes it in a given state. This leads

to a discontinuous change of a behavior derived from the greedy policy w.r.t. that value function [Sutton et al., 1999].

Policy gradients are at the heart of actor-critic methods, which combine learning a value function and a policy and will be discussed in detail in chapter 6. But first, we will lay the mathematical foundations for estimating policy gradients from observations.

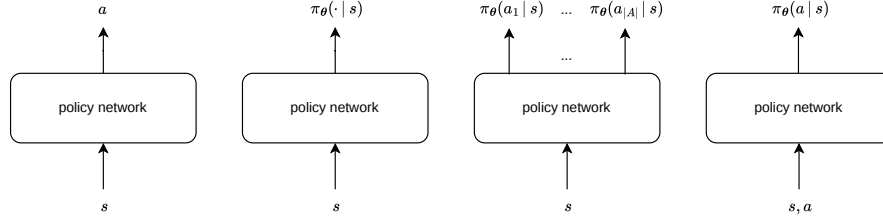


Fig. 5.1 For deterministic policies, the policy network directly maps from a state s to the action as shown in the leftmost drawing. For stochastic policies, a given input state s is mapped to a probability distribution over actions from which the concrete action is sampled, as shown in second drawing. If the action space is finite, a state s can be mapped to a $|A|$ dimensional output, one for each action $\pi_{\theta}(a | s)$ for $a \in A$. The rightmost drawing shows the case where the policy network receives both a s and an action a as input and outputs $\pi_{\theta}(a | s)$.

5.2 Differentiable policies

In policy gradient methods, the policy is given by a differentiable function approximator, the actor. The parameters of the function approximator are denoted by $\theta \in \mathbb{R}^n$. In the deterministic case, the actor maps a state to an action, in the stochastic case to a probability distribution over the potential actions, see fig. 5.1.

For example, for a discrete action set, the policy could be given by

$$\pi_{\theta}(a | s) = \frac{e^{h_{\theta}(s, a)}}{\sum_{a'} e^{h_{\theta}(s, a')}} \quad (5.1)$$

with preferences $h_{\theta}(s, a)$ depending on θ . This corresponds to the rightmost drawing in fig. 5.1. The preferences can be the logits predicted by a deep neural network. For $h_{\theta}(s, a) = \theta^T \phi(s, a)$ equation (5.1) covers the setting where we are given some fixed features embedding $\phi(s, a) : S \times A \rightarrow \mathbb{R}^n$ and learn a linear combination of these features. A policy of the form (5.1) is often called a soft-max policy.

Alternatively, for discrete actions, one can consider a feature map $\phi : S \rightarrow \mathbb{R}^{n'}$ with $n = n'|A|$ and

$$\pi_{\theta}(a | s) = \frac{e^{\theta_a^T \phi(s)}}{\sum_b e^{\theta_b^T \phi(s)}} \quad (5.2)$$

with parameters $\theta = (\theta_1, \dots, \theta_{|A|})$ and w.l.o.g. assuming $A = \{1, \dots, |A|\}$.

Policy gradient methods adapt the behavior by adding

$$\Delta\theta \approx \alpha \nabla_{\theta} J(\pi_{\theta}) \quad (5.3)$$

to the policy parameters, where $J(\pi)$ is a performance measure of the policy π and α a positive step-size/learning-rate. How to estimate the gradient for different performance measures based on sampled episodes is the topic of this chapter.

Gradients

Throughout this section, we assume that the partial derivative $\frac{\partial}{\partial \theta_i} \pi_{\theta}(s, a)$ exists and is finite for every θ , $a \in A$, $s \in S$, and each component θ_i of θ .

Although we refer to gradients throughout this text, all results can be extended to subgradients. Thus, the models need not be differentiable, the existence of subderivatives is sufficient. For example, the maximum function does not cause any problems.

Policy gradient methods often consider stochastic policies. If the action space is discrete, then stochastic policies can smooth the optimization problem to enable gradient-based optimization, as illustrated in the following example.

Example 5.1 Consider a toy MDP with $S = \{s, s_T\}$ and $A = \{\text{left}, \text{right}\}$, where s_T is the terminal state and s is the start state. All actions lead to s_T , $R_s^{\text{left}} = 1$, $R_s^{\text{right}} = 0$. We assume $\gamma = 1$. The goal is to maximize the expected return. In this example, we have $J(\pi_{\theta}) = \mathbb{E}^{\pi_{\theta}}[R] = V^{\pi_{\theta}}(s)$. Thus, the gradient-based optimization of $J(\pi_{\theta})$ corresponds to gradient ascent following $\frac{\partial}{\partial \theta} V^{\pi_{\theta}}$. Assume a deterministic policy given by

$$\pi_{\theta}(s) = \begin{cases} \text{left} & \text{if } \theta \geq 0 \\ \text{right} & \text{otherwise} \end{cases} \quad (5.4)$$

and a stochastic policy:

$$\pi'_\theta(s) = \begin{cases} \text{left} & \text{with probability } \sigma(\theta) \\ \text{right} & \text{with probability } 1 - \sigma(\theta) \end{cases} \quad \text{with} \quad \sigma(\theta) = \frac{1}{1 + e^{-\theta}} \quad (5.5)$$

The value functions of these policies are $V^{\pi_\theta}(s) = \mathbb{I}[\theta \geq 0]$ and $V^{\pi'_\theta}(s) = \sigma(\theta)$.

We have $\frac{\partial}{\partial \theta} V^{\pi'_\theta}(s) = (1 - \sigma(\theta))\sigma(\theta) > 0$, but $\frac{\partial}{\partial \theta} V^{\pi_\theta}(s) = 0$ (where we ignore the discontinuity at $\theta = 0$). That is, the stochastic policy provides reasonable gradient information, leading to an increase of θ at each update step. In contrast, the gradient for the deterministic policy does not provide any information to guide learning.

In the following, we will consider both deterministic and stochastic policies. We will also study the episodic and continuous RL setting. This leads to different performance measures $J(\pi)$. We will derive policy gradients for episodic tasks. This will include results for stochastic policies based on [Sutton et al. \[1999\]](#), [Sutton and Barto \[2018\]](#), [Nota and Thomas \[2020\]](#), and [Mannor et al. \[2026\]](#) as well as the policy gradient theorem for deterministic policies by [Silver et al. \[2014\]](#). To appreciate the results, we will present an RL algorithm in section 5.4 that more or less directly follows from the episodic policy gradient theorem. Finally, we will discuss how baselines can be added to the policy gradient theorems that can act as control variates in Monte Carlo estimation.

5.3 Policy gradient theorems

We consider the more common episodic setting. For episodic tasks, our goal is to maximize the expected return

$$J(\pi) = \mathbb{E}^\pi[R_0] = \mathbb{E}^\pi \left\{ \sum_{t=1}^{\infty} \gamma^{t-1} r_t \right\} = \mathbb{E}_{s_0 \sim p_{\text{start}}} \{V^\pi(s_0)\} \quad (5.6)$$

with $\gamma \in [0, 1]$. The choice of $\gamma = 1$ is only feasible for episodic tasks, because in the continuous setting the sum could diverge without discounting (e.g., if $\forall a \in A, s \in S : R_s^a > 0$). The Q -value function is given by

$$Q^\pi(s, a) = \mathbb{E}^\pi \left\{ \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} \mid s_t = s, a_t = a \right\} . \quad (5.7)$$

Let $\eta^\pi(s)$ denote the number of time steps spent, on average, in $s \in S \setminus S_{\text{goal}}$ in a single episode:

$$\eta^\pi(s) = \mathbb{E}_{s_0 \sim p_{\text{start}}} \left[\sum_{k=0}^{\infty} \Pr\{s_0 \xrightarrow{k} s \mid \pi\} \right] \quad (5.8)$$

The on-policy distribution $\mu^\pi(s)$ is the fraction of time spent in each state:

$$\mu^\pi(s) = \eta^\pi(s) / \sum_{s'} \eta^\pi(s') \quad (5.9)$$

Let us introduce the discount factor γ in (5.8) and define the discounted state distribution

$$\eta_\gamma^\pi(s) = \mathbb{E}_{s_0 \sim p_{\text{start}}} \left[\sum_{k=0}^{\infty} \gamma^k \Pr\{s_0 \xrightarrow{k} s \mid \pi\} \right] \quad (5.10)$$

$$= \sum_{t=0}^{\infty} \gamma^t \Pr\{s_t = s \mid \pi\} , \quad (5.11)$$

where $\mathbb{E}_{s_0 \sim p_{\text{start}}}[\Pr\{s_0 \xrightarrow{0} s \mid \pi\}]$ is the probability of s being the start state. The quantity η_γ^π is not normalized in the sense that $\sum_s \eta_\gamma^\pi(s) \neq 1$ for $\gamma < 1$, because a factor $(1-\gamma)$ is missing. This follows from $\sum_s \Pr\{s_0 \xrightarrow{k} s \mid \pi\} = 1$ for all k (you have to be in some state after k steps) and from the geometric series $\sum_{k=0}^{\infty} \gamma^k = (1-\gamma)^{-1}$. Recall that the expectations are over all trajectories following π and $s_0 \sim p_{\text{start}}$. Silver et al. [2014] refer to (5.10) as the improper state distribution.

If we assume that a terminal state is reached in a finite number of steps with probability one, then any given state k can appear only finitely many times before termination with probability one. Then, with probability one, any given state s can only appear finitely many times before termination, and the expectation considers finite numbers of terms, which implies that $\eta^\pi(s)$ and $\eta_\gamma^\pi(s)$ finite for a non-goal state $s \in S \setminus S_{\text{goal}}$ almost surely.

We can link the improper state distribution to expectations over trajectories:

Proposition 5.1 *For any function of states $g(s)$ and MPD with discount factor γ , we have*

$$\sum_s \eta_\gamma^\pi(s) g(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t g(s_t) \right] .$$

Proof Using (5.11) we have

$$\sum_s \eta_\gamma^\pi(s) g(s) = \sum_{t=0}^{\infty} \Pr\{s_t = s \mid \pi\} \gamma^t g(s) \quad (5.12)$$

$$= \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t g(s_t) \right] . \quad (5.13)$$

The last step uses that $\Pr\{s_t = s \mid \pi\}$ is the probability of arriving in $s_t = s$ when starting from $s_0 \sim p_{\text{start}}$ and following π , which equals the probability of being in $s_t = s$ in a trajectory randomly sampled from all trajectories starting from $s_0 \sim p_{\text{start}}$ and following π . $\square$

This result corresponds to Proposition 13.12 in the textbook by [Mannor et al. \[2026\]](#) for the discounted case.

Because π_{θ} is differentiable w.r.t. θ in any state, $\Pr\{s_0 \xrightarrow{1} s \mid \pi_{\theta}\}$ is differentiable w.r.t. θ for all $s_0, s \in S$ [\[Mannor et al., 2026\]](#). Thus, $\Pr\{s_0 \xrightarrow{k} s \mid \pi_{\theta}\}$ is differentiable w.r.t. θ for any k , which implies that $\eta^{\pi_{\theta}}(s)$ and $\eta_{\gamma}^{\pi_{\theta}}(s)$ are differentiable w.r.t. to θ .

To derive policy gradients, we make use of the performance difference lemma [\[Kakade and Langford, 2002, Schulman et al., 2015\]](#):

Lemma 5.1 (performance difference lemma) *For any two policies π_{ref} and π we have:*

$$J(\pi) = J(\pi_{\text{ref}}) + \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (Q^{\pi_{\text{ref}}}(s_t, a_t) - V^{\pi_{\text{ref}}}(s_t)) \right] \quad (5.14)$$

$$= J(\pi_{\text{ref}}) + \sum_s \eta_{\gamma}^{\pi}(s) \sum_a \pi_{\theta}(a \mid s) (Q^{\pi_{\text{ref}}}(s, a) - V^{\pi_{\text{ref}}}(s)) \quad (5.15)$$

Proof The proof of the first equality follows [Schulman et al. \[2015\]](#):

$$\begin{aligned} & \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (Q^{\pi_{\text{ref}}}(s_t, a_t) - V^{\pi_{\text{ref}}}(s_t)) \right] \\ &= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_{t+1} + \gamma V^{\pi_{\text{ref}}}(s_{t+1}) - V^{\pi_{\text{ref}}}(s_t)) \right] \end{aligned} \quad (5.16)$$

$$= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} + \sum_{t=0}^{\infty} (\gamma^{t+1} V^{\pi_{\text{ref}}}(s_{t+1}) - \gamma^t V^{\pi_{\text{ref}}}(s_t)) \right] \quad (5.17)$$

$$= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} - V^{\pi_{\text{ref}}}(s_0) \right] = J(\pi) - J(\pi_{\text{ref}}) \quad (5.18)$$

To prove the second equality (5.15), we start from the just proven (5.14) and plug in (5.11), giving

$$J(\pi) = J(\pi_{\text{ref}}) + \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t (Q^{\pi_{\text{ref}}}(s_t, a_t) - V^{\pi_{\text{ref}}}(s_t)) \right] \quad (5.19)$$

$$= J(\pi_{\text{ref}}) + \sum_{t=0}^{\infty} \sum_s \Pr\{s_t = s \mid \pi\} \sum_a \pi_{\theta}(a \mid s) \gamma^t (Q^{\pi_{\text{ref}}}(s, a) - V(s)) \quad (5.20)$$

$$= J(\pi_{\text{ref}}) + \sum_s \sum_{t=0}^{\infty} \gamma^t \Pr\{s_t = s \mid \pi\} \sum_a \pi_{\theta}(a \mid s) (Q^{\pi_{\text{ref}}}(s, a) - V(s)) \quad (5.21)$$

$$= J(\pi_{\text{ref}}) + \sum_s \eta_{\gamma}^{\pi}(s) \sum_a \pi_{\theta}(a \mid s) (Q^{\pi_{\text{ref}}}(s, a) - V(s)) , \quad (5.22)$$

which completes the proof. $\square$

This leads to the policy gradient theorem for episodic tasks:

Theorem 5.1 (policy gradient for episodic tasks) *For any differentiable stochastic policy π_{θ} and any MDP in the start-state setting, it holds*

$$\nabla_{\theta} J(\pi_{\theta}) = \sum_s \eta_{\gamma}^{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a) . \quad (5.23)$$

Proof The proof is inspired by [Mannor et al. \[2026\]](#). Consider the i th component of the gradient, and let $\mathbf{e}_i$ be the i th unit vector. We have

$$\frac{\partial}{\partial \theta_i} J(\pi_{\theta}) = \lim_{\epsilon \rightarrow 0} \frac{J(\pi_{\theta + \epsilon \mathbf{e}_i}) - J(\pi)}{\epsilon} \quad (5.24)$$

and because of lemma 5.1

$$= \lim_{\epsilon \rightarrow 0} \frac{\sum_s \eta_{\gamma}^{\pi_{\theta + \epsilon \mathbf{e}_i}}(s) \sum_a \pi_{\theta + \epsilon \mathbf{e}_i}(a \mid s) (Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s))}{\epsilon} \quad (5.25)$$

and because $\sum_a \pi_{\theta + \epsilon \mathbf{e}_i}(a \mid s) V^{\pi_{\theta}}(s) = V^{\pi_{\theta}}(s) = \sum_a \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a)$

$$= \lim_{\epsilon \rightarrow 0} \frac{\sum_s \eta_{\gamma}^{\pi_{\theta + \epsilon \mathbf{e}_i}}(s) \sum_a (\pi_{\theta + \epsilon \mathbf{e}_i}(a \mid s) - \pi_{\theta}(a \mid s)) Q^{\pi_{\theta}}(s, a)}{\epsilon} \quad (5.26)$$

$$= \sum_s \eta_{\gamma}^{\pi_{\theta}}(s) \sum_a \frac{\partial \pi_{\theta}(a \mid s)}{\partial \theta_i} Q^{\pi_{\theta}}(s, a) . \quad (5.27)$$

As this holds for all i , we get the expression for $\nabla_{\theta} J(\pi_{\theta})$. $\square$

We would like to estimate (5.23) from trajectories following π_{θ} . The following result helps us to do so:

Corollary 5.1 *For any differentiable stochastic policy π_θ and any MDP in the start-state setting it holds*

$$\nabla_\theta J(\pi_\theta) \propto \mathbb{E}^{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k \frac{1}{\pi_\theta(a_k | s_k)} \nabla_\theta \pi_\theta(a_k | s_k) Q^{\pi_\theta}(s_k, a_k) \right] \quad (5.28)$$

$$= \mathbb{E}^{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k \nabla_\theta \log \pi_\theta(a_k | s_k) Q^{\pi_\theta}(s_k, a_k) \right]. \quad (5.29)$$

Proof Applying proposition 5.1 to (5.23) we get

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}^{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \sum_a \pi_\theta(a | s_t) Q^{\pi_\theta}(s_t, a) \right] \quad (5.30)$$

$$\propto \mathbb{E}^{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \frac{1}{\pi_\theta(a_t | s_t)} \pi_\theta(a_t | s_t) Q^{\pi_\theta}(s_t, a_t) \right]. \quad (5.31)$$

To understand the last step, assume that we are given a trajectory $s_0, a_0, r_1, \dots, r_T$ following π_θ . In this sequence, the states s_t occur with $\Pr\{s_t = s | \pi_\theta\}$. However, the action a_k in the sequence was selected with probability $\pi_\theta(a_k | s_k)$, not uniformly as in (5.30). Thus, we need to weight it proportional to $\frac{1}{\pi_\theta(a_k | s_k)}$ accordingly. This is importance weighting, where we ignore the numerator, which does not depend on k , in the importance weight. Therefore, we have proportionality instead of equality in (5.28).

The equality (5.29) follows from the chain rule and the derivative of the logarithm, see section 2.3.3.1. $\square$

This is an exciting result, because the expressions for the gradient do not contain any $\nabla_\theta \mu^{\pi_\theta}(s)$ or $\nabla_\theta \eta_\gamma^\pi$ terms, which would make the estimation impractical. This is not trivial, and we will prove the theorem below.

That (5.28) equals the gradient only up to a constant factor is not a problem. The scaling, does not negatively influence gradient-based optimization. For example, the missing factor can be absorbed in the learning rate of vanilla stochastic gradient ascent.

Equation (5.28) is well suited for Monte Carlo approximation. If s_k and a_k are sampled following π_θ starting from $s_0 \sim p_{\text{start}}$, then

$$\gamma^k \frac{1}{\pi_\theta(a_k | s_k)} \nabla_\theta \pi_\theta(a_k | s_k) Q^{\pi_\theta}(s_k, a_k) \quad (5.32)$$

is an unbiased estimate of a quantity proportional to the policy gradient

The expressions (5.28), (5.29), and (5.32) contain a γ^k factor. If $\gamma = 1$, we can drop it. Otherwise, we need to keep track of the step number k when using a state-action pair in the gradient computation. The gradient in the

discounted setting $\gamma \neq 1$ is often incorrectly stated in the literature; see [Nota and Thomas \[2020\]](#) for an overview.

So far, we have considered stochastic policies. However, policy gradients can also be computed for deterministic policies [\[Silver et al., 2014\]](#):

Theorem 5.2 (deterministic policy gradient for discounted episodic tasks) *For any MDP with discount factor γ in the start-state setting and any deterministic policy π_θ it holds*

$$\nabla_\theta J(\pi_\theta) = \sum_s \eta_\gamma^{\pi_\theta}(s) \nabla_\theta Q^{\pi_\theta}(s, \pi_\theta(s)) \quad (5.33)$$

$$= \sum_s \eta_\gamma^{\pi_\theta}(s) \nabla_\theta \pi_\theta(s) \nabla_a Q^{\pi_\theta}(s, a)|_{a=\pi_\theta(s)} \quad (5.34)$$

$$\propto \mathbb{E}^{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k \nabla_\theta \pi_\theta(s_k) \nabla_a Q^{\pi_\theta}(s_k, a)|_{a=\pi_\theta(s_k)} \right] \quad (5.35)$$

$$= \mathbb{E}^{\pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k \nabla_\theta Q^{\pi_\theta}(s_k, \pi_\theta(s_k)) \right] . \quad (5.36)$$

Proof Lemma 5.1 also holds for deterministic policies. We follow the proof of theorem 5.1 and consider the i th component of the gradient. Because π_θ is deterministic, (5.25) becomes

$$\frac{\partial}{\partial \theta_i} J(\pi_\theta) = \frac{\partial}{\partial \theta_i} J(\pi_\theta) = \lim_{\epsilon \rightarrow 0} \frac{\sum_s \eta^{\pi_{\theta+\epsilon \mathbf{e}_i}}(Q^{\pi_\theta}(s, \pi_{\theta+\epsilon \mathbf{e}_i}(s)) - V^{\pi_\theta}(s))}{\epsilon} . \quad (5.37)$$

For deterministic π it holds $V^\pi(s) = Q^\pi(s, \pi(s))$. Thus, we have

$$\frac{\partial}{\partial \theta_i} J(\pi_\theta) = \lim_{\epsilon \rightarrow 0} \frac{\sum_s \eta^{\pi_{\theta+\epsilon \mathbf{e}_i}}(Q^{\pi_\theta}(s, \pi_{\theta+\epsilon \mathbf{e}_i}(s)) - Q^{\pi_\theta}(s, \pi_\theta(s)))}{\epsilon} \quad (5.38)$$

$$= \sum_s \eta^{\pi_\theta} \frac{\partial}{\partial \theta_i} Q^{\pi_\theta}(s, \pi_\theta(s)) . \quad (5.39)$$

That is, $\nabla_\theta J(\pi_\theta) = \sum_s \eta^{\pi_\theta} \nabla_\theta Q^{\pi_\theta}(s, \pi_\theta(s))$. By the chain rule we have

$$\nabla_\theta Q^{\pi_\theta}(s, \pi_\theta(s)) \stackrel{\text{chain rule}}{=} \nabla_a Q^{\pi_\theta}(s, a)|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s) , \quad (5.40)$$

proving (5.34).

The proportionality (5.35) follows using the same line of arguments as in the proof of corollary 5.1 realizing that if an action a_k was taken in a state s_k then the deterministic policy selected it with probability one. The last equality follows by undoing the step from (5.33) to (5.34). $\square$

5.4 REINFORCE algorithm

The policy gradient theorem [5.1](#) leads directly to the RL algorithm REINFORCE for episodic tasks introduced in the seminal paper by [Williams \[1992\]](#).

If a state s_t is taken from an episode following a stochastic policy π_θ , then corollary [5.1](#) implies that

$$\gamma^t \frac{1}{\pi_\theta(a_t | s_t)} \nabla_\theta \pi_\theta(a_t | s_t) Q^{\pi_\theta}(s_t, a) \quad (5.41)$$

is an unbiased estimate of $\nabla_\theta J(\pi_\theta)$ up to scaling. The scaling factor does not matter, it gets absorbed by the learning rate in the gradient-based optimization (α in [\(5.3\)](#)). The unknown $Q^{\pi_\theta}(s_t, a_t)$ can be estimated by the actual return after being in s_t , that is, $R_t = \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k}$ or $R_t = \sum_{k=1}^{T-t} \gamma^{k-1} r_{t+k}$ for a finite episode of length T . In the original episodic REINFORCE algorithm by [Williams \[1992\]](#), $Q^{\pi_\theta}(s_t, a_t)$ was estimated by the total reward of the trajectory $R_0 = \sum_{t=1}^T r_t$.

This yields the REINFORCE parameter update for a state-action pair $\langle s_t, a_t \rangle$:

$$\Delta \theta_t \propto \gamma^t \frac{1}{\pi_\theta(a_t | s_t)} \nabla_\theta \pi_\theta(a_t | s_t) R_t \stackrel{\text{"log derivative trick"}}{=} \gamma^t \underbrace{\nabla_\theta \ln \pi_\theta(a_t | s_t)}_{\text{score function}} R_t \quad (5.42)$$

Algorithm [10](#) describes the REINFORCE algorithm as presented by [Sutton and Barto \[2018\]](#). Note the γ^t factor in the parameter update, which follows from theorem [5.1](#) and was missing in early versions of the algorithm.

Algorithm 10: REINFORCE

```

1 policy  $\pi_\theta$ , learning rate  $\alpha > 0$ 
2 repeat
3    $s_0 \sim p_{\text{start}}$ 
4   generate episode  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$  following  $\pi_\theta$ 
5   foreach  $t = 0, \dots, T-1$  do
6     // compute return  $R_t$  starting from  $s_t$ 
7      $R_t = \sum_{k=1}^{T-t} \gamma^{k-1} r_{t+k}$ 
8     // update policy parameters
9      $\theta \leftarrow \theta + \alpha R_t \gamma^t \nabla_\theta \ln \pi_\theta(a_t | s_t)$ 
10  until stopping criterion is met

```

5.5 Adding a baseline

The policy gradient theorems can be generalized to include a baseline. For example, in theorem 5.1 we can write

$$\nabla_{\theta} J(\pi_{\theta}) = \sum_s \eta_{\gamma}^{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) (Q^{\pi_{\theta}}(s, a) - b(s)) \quad , \quad (5.43)$$

where $b(s) : S \rightarrow \mathbb{R}$ is an arbitrary baseline function.

This corresponds to introducing a control variate to reduce the variance (see section 2.3.2). The control variate in (5.43) is $\phi(s, a) = \nabla_{\theta} \pi_{\theta}(a | s) b(s)$. Note that $\mathbb{E}[\phi] = 0$ for any distribution over states and a uniform distribution over actions.

A natural choice for $b(s)$ is some estimate of the value function $V^{\pi_{\theta}}(s)$. The value of a state s should be positively correlated with the state-action values in s . If following π_{θ} starting from s promises a high return, that is, s is a state with a high value, then taking an action a in the state s and following π_{θ} afterwards is rather likely to also give a high return and vice versa for states with a low value. It can be proven that for baselines only depending on the state, $b(s) = V(s)$ is the optimal choice if the magnitudes $\|\nabla_{\theta} \pi_{\theta}(a | s)\|$ of the gradients are the same for all actions $a \in A$ [Mannor et al., 2026, sec. 13.6.1].

Adding the baseline $b(s) = V(s)$ in the policy gradient estimation corresponds to considering $Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s)$ instead of $Q^{\pi_{\theta}}(s, a)$ in the gradient estimation. We call

$$A^{\pi_{\theta}}(s, a) = Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s) \quad (5.44)$$

the advantage function. It quantifies the advantage of taking action a in state s and following π_{θ} afterwards over simply following π_{θ} . If π_{θ} is deterministic and $\pi_{\theta}(s) = a$, then $A^{\pi_{\theta}}(s, a) = 0$.

We will discuss REINFORCE with a baseline and advantages in more detail in chapter 6.

5.6 Policy gradients in the wild

In the literature, you will find several expressions for the policy gradient, which have the form

$$\nabla_{\theta} J(\pi) \simeq \mathbb{E}^{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad , \quad (5.45)$$

where Ψ_t may be one of the following [Schulman et al., 2016]:

- $\sum_{t=1}^{\infty} r_t$: total reward of the trajectory
- $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t
- $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula
- $Q^{\pi_\theta}(s_t, a_t)$: state-action value function
- $A^{\pi_\theta}(s_t, a_t)$: advantage function
- $\hat{A}_k^{\pi_\theta}(s_t, a_t)$: k -step advantage (see section 6.2.1)
- $r_t + V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)$: temporal difference error

A discount factor γ can be added where appropriate. In the discounted case, some approximations of the policy gradient in the literature are not unbiased estimates of the policy gradient (even worse, some can be shown to not be a gradient of any function, see [Nota and Thomas, 2020](#)). They apparently work in practice anyway.

In practice, most deep RL systems are implemented in frameworks that support automatic differentiation. Accordingly, often gradients are not implemented explicitly but are automatically derived from the implementations of models and loss functions. This is more difficult than in standard supervised learning, because it often requires explicit specification of computations that should be excluded in the gradient computations (e.g., the target computation when learning a value function). Incorporating the discount factor properly in the automatic computation of the policy gradient may not be trivial, which may be a reason for not considering it.

While many design choices in deep RL have to be based on heuristics because theory is lacking, I would argue that one should follow the theoretical results as far as possible and, for example, use unbiased estimates.

Chapter 6

Actor-critic methods

Abstract Actor-critic algorithms mark the state of the art in deep RL. They combine value function estimation and policy-gradient methods. The chapter begins with introducing a value-function baseline to the REINFORCE algorithm and progresses to online actor-critic methods using temporal-difference learning. It discusses details of major modern actor-critic algorithms, including Deep Deterministic Policy Gradient (DDPG), Twin-delayed Deep Deterministic (TD3), Soft Actor-Critic (SAC), and Proximal Policy Optimization (PPO). Key concepts such as experience replay, target networks, entropy and KL regularization, clipped double-Q learning, and advantage estimation are introduced and connected to pseudocode of each method.

6.1 Basic actor-critic methods

A model (e.g., a deep neural network) approximating a value function is referred to as a critic, because it evaluates states or state-action pairs, and a function approximator representing a policy is called an actor, which maps states to actions. Deep RL algorithms based on DQN are referred to as critic-only methods, as function approximation is only involved in learning Q -functions. Critic-only methods involve finding the action that maximizes the value function in a given state, which may be prohibitive for large action spaces, for example, in continuous control. This issue can be addressed by having an actor that efficiently maps states to actions instead of, for example, using an ϵ -greedy policy on top of a value function. However, actor-only methods that only use a function approximator to represent a policy that is learned based on policy gradients suffer from a high variance of the gradient estimates. Actor-critic algorithms learn function approximators both for the value function and for the policy. In deep RL the value function is learned in a way similar to deep Q -learning as presented in chapter 4, and the learning of the policy is based on policy gradient methods as introduced

in chapter 5. Representing the policy by a function approximator allows us to deal with complex action spaces, and additionally approximating a value function stabilizes the learning process.

In the following, we present actor-critic approaches. First, we turn the REINFORCE algorithm into an actor-critic method. Then we will show how to enable learning from intermediate rewards by TD learning in an actor-critic architecture. Section 6.3 introduces the Deep Deterministic Policy Gradient (DDPG) algorithm, a fundamental method for deterministic policies. Before we move on to state-of-the-art algorithms for stochastic policies, section 6.5 shows how learning stochastic policies can be regularized and section 6.2.1 discusses learning advantage functions and a basic algorithm based on it. Section 6.6 and section 6.7 then introduce the Soft Actor Critic (SAC) and Proximal Policy Optimization (PPO), respectively.

6.1.1 REINFORCE with value function baseline

In actor-critic methods, the policies as well as the value functions are parametrized. We denote the parameters of policies by θ and the parameters of value function approximators by $\mathbf{w}$. Throughout this chapter, we assume that the actors and critics are differentiable.

The policy gradient theorem can be generalized to include a baseline, and a good choice for the baseline is an estimate $\hat{V}_{\mathbf{w}}$ of the state value function as discussed in section 5.5. If we add a baseline to REINFORCE as introduced in section 5.4 to reduce the variance of the gradient estimate we arrive at algorithm 11.

Algorithm 11: REINFORCE with baseline

Input: policy π_{θ} , state-value function $\hat{V}_{\mathbf{w}}$, learning rates $\alpha_{\mathbf{w}}, \alpha_{\theta} > 0$

```

1 repeat
2    $s_0 \sim p_{\text{start}}$ 
3   generate episode  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$  following  $\pi$ 
4   foreach  $t = 0, \dots, T-1$  do
5      $R_t = \sum_{k=1}^{T-t} \gamma^{k-1} r_{t+k}$ 
6      $\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} (R_t - \hat{V}_{\mathbf{w}}(s_t)) \nabla_{\mathbf{w}} \hat{V}_{\mathbf{w}}(s_t)$  // critic update
7      $\theta \leftarrow \theta + \alpha_{\theta} \gamma^t (R_t - \hat{V}_{\mathbf{w}}(s_t)) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$  // actor update
8 until stopping criterion is met
```

In this algorithm, we learn both a value-function approximation $\hat{V}_{\mathbf{w}}$ and the parameters of the policy. Thus, REINFORCE with a baseline can be viewed as an actor-critic algorithm.

Line 5 in algorithm 11 computes the return, the (discounted) accumulated reward. If the episode ends because s_{T-1} is a terminal state, then in our setting $r_T = 0$. Recomputing the sum in line 5 for each t is not very elegant. This can be implemented more efficiently if we go through the states in the episode in reverse order, see algorithm 13.

6.1.2 Simple actor-critic with TD learning

REINFORCE is not fully online in the sense that episodes need to finish to provide the learning signal. For online updating, we move to temporal difference learning with bootstrapping.

To adapt a value function approximator $\hat{V}_{\mathbf{w}}$, we minimize the halved squared temporal difference (TD) error

$$\frac{1}{2}(\delta_{\text{TD}}(s, r, s'))^2 = \frac{1}{2} \left(\underbrace{r + \gamma \hat{V}_{\mathbf{w}}(s')}_{\text{target } y} - \hat{V}_{\mathbf{w}}(s) \right)^2 \quad (6.1)$$

in the same way as for $\hat{Q}_{\mathbf{w}}$ described in section 4.2. Algorithm 12 shows a simple actor-critic method with TD learning.

Algorithm 12: Simple actor-critic with TD learning

Input: policy π_{θ} , state-value function $\hat{V}_{\mathbf{w}}$, learning rates $\alpha_{\mathbf{w}}, \alpha_{\theta} > 0$, initial θ and $\mathbf{w}$

```

1 repeat
2    $t \leftarrow 0$ 
3    $s_t \sim p_{\text{start}}$ 
4   repeat
5      $a_t \sim \pi_{\theta}(\cdot | s_t)$ 
6     take action  $a_t$  and observe  $r_{t+1}$ ,  $s_{t+1}$ , and  $d_{t+1}$ 
7      $\delta \leftarrow r_{t+1} + (1 - d_{t+1})\gamma \hat{V}_{\mathbf{w}}(s_{t+1}) - \hat{V}_{\mathbf{w}}(s_t)$ 
8      $\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta \nabla_{\mathbf{w}} \hat{V}_{\mathbf{w}}(s_t)$  // critic update
9      $\theta \leftarrow \theta + \alpha_{\theta} \gamma^t \delta \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$  // actor update
10     $t \leftarrow t + 1$ 
11  until terminal state is reached
12 until stopping criterion is met
```

The algorithm uses the indicator variable d , which is one for terminal states and zero otherwise, to deal with terminal states, see section 4.3. In line 8, the parameters of the value function approximator are updated. Note that lines 7 and 8 could be replaced by $y = r_{t+1} + (1 - d_{t+1})\gamma \hat{V}_{\mathbf{w}}(s_{t+1})$ and $\mathbf{w} \leftarrow \mathbf{w} - \alpha_{\mathbf{w}} \nabla_{\mathbf{w}} \ell(\hat{V}_{\mathbf{w}}(s_t), y)$, where ℓ is the halved squared loss and we

assume that y is independent of $\mathbf{w}$ (see section 4.2). In algorithm 12, the parameters of the policy are updated as in REINFORCE, see algorithm 11.

6.1.3 Actor-critic neural networks

In actor-critic algorithms, we need function approximators for value functions and policies. In deep RL, at least one of the function approximators is a neural network. When both actor and critic are represented by deep neural networks, these networks can share weights. This means that the policy and value function are represented by one deep neural network with two different heads: The initial layers process the input s leading to an intermediate feature representation. This feature representation serves as the input for two different streams in the network, one for the actor and one for the critic.

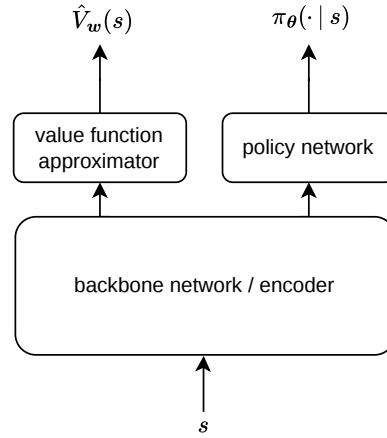


Fig. 6.1 Schema of a deep neural network architecture with two heads, where the value function network and the policy network share weights. Some of the parameters $\mathbf{w}$ and $\boldsymbol{\theta}$, the backbone, are shared.

Figure 6.1 illustrates a network that maps a state s to $(\hat{V}_{\mathbf{w}}(s), \pi_{\boldsymbol{\theta}}(\cdot | s))$, a state value prediction (critic) and a distribution over states (actor), where some of the parameters $\mathbf{w}$ and $\boldsymbol{\theta}$ are shared. Sharing the initial layers, which are also referred to as the backbone or the encoder, increases statistical efficiency. It is reasonable that a learned feature representation that is suitable for predicting the best action in a state is also suitable for predicting the value of a state and vice versa. For example, the architecture in fig. 2.5 could be modified to also provide a value estimate. The convolutional layers can serve as the backbone of two standard neural networks on top – two heads – representing actor and critic, respectively.

Stochastic policies are often represented by tractable distributions. In deep RL, the parameters of these distributions can be given by neural networks.

For example, for a discrete action space, the preferences in the soft-max policy (5.1) can come from a neural network head. For continuous actions spaces, often multi-variate Gaussians are chosen to model the distributions over actions. In this case, the mean and parameters of the covariance matrix can be provided by a neural network head.

6.2 Advantage Actor-Critic

In the REINFORCE algorithm with value function baseline described in section 6.1.1, we introduced $\hat{V}(s)$ to reduce the variance of the policy gradient estimation. That is, instead of optimizing the state-action value function, we rather optimize the advantage $\hat{A}^\pi(s, a) = \hat{Q}^\pi(s, a) - \hat{V}^\pi(s)$. The advantage is positive if the action taken in state s is better than expected by the value function approximation, negative if it is worse, and zero if the action's outcome was as expected. In the following, we study advantages in more detail, and describe a popular actor-critic algorithm called Advantage Actor-Critic (A²C) proposed by Mnih et al. [2016].

6.2.1 Advantage estimation

Our goal is to maximize the expected return $J(\pi) = \mathbb{E}_{s \sim p_{\text{start}}} [V^\pi(s)] = \mathbb{E}^\pi [\sum_{t=1}^{\infty} \gamma^{t-1} r_t]$, where $\mathbb{E}^\pi$ is the expectation over sequences $s_0, a_0, r_1, \dots$ following π and $s_0 \sim p_{\text{start}}$. The advantage of taking action a in state s (and following π afterwards) instead of following π :

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (6.2)$$

In 5.1 we already expressed the expected return of a discounted policy π in terms of its advantage over another reference policy π_{ref} and $J(\pi_{\text{ref}})$ [Kakade and Langford, 2002, Schulman et al., 2015]. For any two policies π_{ref} and π we have

$$J(\pi) = \underbrace{J(\pi_{\text{ref}})}_{\text{independent of } \pi} + \underbrace{\mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\text{ref}}}(s_t, a_t) \right]}_{\text{objective of advantage optimization}}. \quad (6.3)$$

Since $J(\pi_{\text{ref}})$ is independent of π , we can optimize $J(\pi)$ by optimizing the expectation on the RHS of (6.3), which is called advantage optimization.

We can estimate the advantage

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (6.4)$$

from a sample s_t, a_t, r_{t+1} and approximated value function $\hat{V}^\pi$ by:

$$\hat{A}^\pi(s_t, a_t) = r_{t+1} + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t) \quad (6.5)$$

Adding more steps incorporates more information and reduces the influence of poorly estimated $\hat{V}^\pi$ values; see section 2.1.4. Starting from $\hat{A}_1^\pi(s_t, a_t) = r_{t+1} + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$, we define a multi-step advantage

$$\begin{aligned} \hat{A}_k^\pi(s_t, a_t) &= \sum_{l=0}^{k-1} \gamma^l \hat{A}_1^\pi(s_{t+l}, a_{t+l}) = \\ &= -\hat{V}^\pi(s_t) + r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{k-1} r_{t+k} + \gamma^k \hat{V}^\pi(s_{t+k}) = \\ &= G_t^{(k)}(\hat{V}^\pi(s_{t+k})) - \hat{V}^\pi(s_t) \end{aligned} \quad (6.6)$$

where the sequence of states and rewards was obtained by following the policy π . The last equality follows from the definition of the general multi-step return. In the limit, we get the empirical return minus the value function baseline (which plays the role of a control variate):

$$\hat{A}_\infty^\pi(s_t, a_t) = \sum_{l=0}^{\infty} \gamma^l \hat{A}_1^\pi(s_{t+l}, a_{t+l}) = -\hat{V}^\pi(s_t) + \sum_{l=1}^{\infty} \gamma^{l-1} r_{t+l} \quad (6.7)$$

Based on a (sub-)sequence

$$s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, s_T$$

obtained following π we can estimate the advantages:

$$\begin{aligned} \hat{A}_{T-t}^\pi(s_t, a_t) &= -\hat{V}^\pi(s_t) + \\ &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T + \gamma^{T-t} \hat{V}^\pi(s_T) \end{aligned} \quad (6.8)$$

This estimation takes into account more real observations than (6.4), which reduces potential negative effects of bootstrapping. It is a form of multi-step learning, where for each state observed in an (sub-)episode all rewards until the end of the (sub-)episode are considered.

In generalized advantage estimation (GAE), the terms in (6.7) are additionally exponentially weighted

$$\text{GAE}_{\lambda_{\text{TD}}, \infty}^\pi(s_t, a_t) = \sum_{l=0}^{\infty} (\lambda_{\text{TD}} \gamma)^l \hat{A}_1^\pi(s_{t+l}, a_{t+l}) = -\hat{V}^\pi(s_t) + \sum_{l=1}^{\infty} (\lambda_{\text{TD}} \gamma)^{l-1} r_{t+l} \quad (6.9)$$

with parameter $\lambda_{\text{TD}} \in [0, 1]$, and $\text{GAE}_{\lambda_{\text{TD}}, k}^\pi(s_t, a_t)$ is defined accordingly. For $\lambda_{\text{TD}} = 1$ we get the Monte Carlo estimate $\hat{A}_\infty^\pi(s_t, a_t)$ of the advantage, which can be argued to have a high variance due to the large number of terms

[Schulman et al., 2016], $\lambda_{\text{TD}} = 0$ gives (setting $0^0 = 1$) the 1-step TD error $\hat{A}_1^\pi(s_t, a_t)$. Using $0 < \lambda_{\text{TD}} < \gamma$ introduces a bias but can reduce the variance.

It is a common strategy (e.g., in PPO) to break down episodes into sub-episodes of length T for computing the advantage.

6.2.2 Advantage Actor-Critic

Mnih et al. [2016] introduce several RL algorithms based on advantage optimization. They focus on asynchronous algorithms, that is, methods that are designed for parallel threads that update the policy asynchronously. Each thread executes an RL agent on an instances of the environment and provides update information whenever an episode or sub-episode (i.e., a fixed number of steps) is completed.

In the following, we describe a method that resembles the Asynchronous Advantage Actor-Critic method called A³C. We present a single-thread version, which we refer to as A²C. However, the pseudocode in algorithm 13 can be turned into an asynchronous version in a straight-forward way.

In our A²C version, T denotes the maximum number of time steps in a sub-episode and we accumulated over T_{update} sub-episodes, which could run in different threads. The gradient-based update of the policy uses entropy bonus as in (6.16) (with $\alpha_H = 0.01$ in the experiments by Mnih et al., 2016). Line 17 in algorithm 13 could be replaced by $G \leftarrow G_{i-1}^{(t-i+1)}((1-d_t)\hat{V}(s_t))$, that is, the algorithm computes the general corrected multi-step truncated return using the recursive relation (2.32).

The actor-critic algorithm proposed by Mnih et al. [2016], which is the basis of algorithm 13, does not use an experience replay buffer. It relies on a large number of (sub-)episodes (potentially run in parallel threads) to stabilize the gradient update. The A²C algorithm is regarded as an on-policy method; all evaluations and updates are based on the current policy.

The indexing in algorithm 13 may need some explanation. We break down the interaction with the environment into sub-episodes. A sub-episode ends if the episode ends (in which case $t - t_{\text{start}}$ can be smaller than T in the update loop afterwards) or after T interactions. The index t is reset to zero only if a full episode is completed. Therefore, we distinguish two initializations of s_0 . In a new episode starts, we sample a new start state. If the new sub-episode continues from a previous sub-episode that did not reach a goal state, we continue from the last state of the previous sub-episode.

Mnih et al. [2016] suggested adding an entropy bonus to improve exploration as will be discussed in section 6.5.1.

Note that the algorithm is very similar to REINFORCE with a value function baseline, see algorithm 11. Breaking down an episode into sub-episodes of length T can be regarded as a middle ground between waiting until the end of an episode in algorithm 11 and doing an update after every step in

Algorithm 13: Advantage Actor-Critic (A²C)

Input: policy π_θ and value function approximator $\hat{V}_w$, parameters of gradient-based learning algorithm (e.g., a learning rate), sub-episode length T , steps between updates T_{update}

```

1 repeat
2    $\delta_w \leftarrow 0; \delta_\theta \leftarrow 0$  // reset accumulated gradients
3    $t \leftarrow 0$  // counter
4    $d_0 \leftarrow 1$  // sample start state in the beginning
5   for  $e = 1, \dots, T_{\text{update}}$  do
6     if  $d_t = 1$  then // new start after end of episode
7        $s_0 \sim p_{\text{start}}$ 
8        $t \leftarrow 0$  // reset counter
9      $t_{\text{start}} \leftarrow t$  // beginning of sub-episode
10    repeat // interact with environment
11      perform action  $a_t$  according to  $\pi(s_t, \cdot)$ 
12      observe  $r_{t+1}, s_{t+1}, d_{t+1}$ 
13       $t \leftarrow t + 1$ 
14    until  $d_t = 1$  or  $t - t_{\text{start}} = T$ 
15    // bootstrap from last state if not terminal
16     $G \leftarrow (1 - d_t)\hat{V}_w(s_t)$ 
17    for  $i = t, \dots, t - t_{\text{start}} + 1$  do //  $i$  counts backwards
18       $G \leftarrow r_i + \gamma G$ 
19      // REINFORCE style actor update
20       $\delta_\theta \leftarrow \delta_\theta + \nabla_\theta \gamma^{i-1} \log \pi_\theta(a_{i-1} | s_{i-1})(G - \hat{V}_w(s_{i-1}))$ 
21      // critic update
22       $\delta_w \leftarrow \delta_w + \nabla_w \ell(G, \hat{V}_w(s_{i-1}))$ 
23    gradient-based update of  $w$  and  $\theta$  using  $\delta_w$  and  $\delta_\theta$ 
24  until stopping criterion is met

```

algorithm 12. The only other difference from REINFORCE is the gradient accumulation and delayed update after T_{update} sub-episodes, which improves training stability.

6.3 Deep Deterministic Policy Gradient

The Deep Deterministic Policy Gradient (DDPG) algorithm was proposed by Lillicrap et al. [2016]. It is a model-free actor-critic algorithm. It was designed for continuous action spaces. The algorithm is an off-policy approach. It maintains a deterministic policy π_θ , and exploration is performed by randomly modifying the output of π_θ , for example, by adding zero mean Gaussian noise. A version of DDPG is shown in algorithm 14.

Algorithm 14: Deep Deterministic Policy Gradient (DDPG)

Input: policy π_{θ} and value function approximator $\hat{Q}_{\mathbf{w}}$, experience buffer M , parameters of gradient-based learning algorithm, batch size n_{batch} , number of gradient-update steps $n_{\text{gradient-steps}}$, variance σ_a^2 of exploration distribution, feasible region $[a_{\text{low}}, a_{\text{high}}]$, target network blending parameter ρ

```

1  $\mathbf{w}_{\text{target}} \leftarrow \mathbf{w}$ 
2  $\theta_{\text{target}} \leftarrow \theta$ 
3  $s \sim p_{\text{start}}$ 
4  $t \leftarrow 0$ 
5 repeat
    // choose action with added noise for exploration
6   sample  $a = \text{clip}(\pi_{\theta}(s) + \epsilon, a_{\text{low}}, a_{\text{high}})$  with  $\epsilon \sim \mathcal{N}(0, \sigma_a^2)$ 
7   take action  $a$ , observe  $r, s', d$ 
8   store  $\langle s, a, r, s', d \rangle$  in  $M$ 
9   if  $d = 0$  then
10     $s \leftarrow s'$ 
11     $t \leftarrow t + 1$ 
12  else
13     $s \sim p_{\text{start}}$  // start new episode
14     $t \leftarrow 0$ 
15  if  $t \bmod T_{\text{update}} = 0$  then
16    for  $n_{\text{gradient-steps}}$  steps do
17      sample mini-batch  $\{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
18      // compute targets
19      foreach  $i$  do  $y_i = r_i + \gamma(1 - d_i)\hat{Q}_{\mathbf{w}_{\text{target}}}(s'_i, \pi_{\theta_{\text{target}}}(s'_i))$ 
20      // update critic
21      update  $\mathbf{w}$  based on gradient  $\nabla_{\mathbf{w}} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \ell(y_i, \hat{Q}_{\mathbf{w}}(s_i, a_i))$ 
22      // update actor
23      update  $\theta$  based on gradient  $\nabla_{\theta} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \hat{Q}_{\mathbf{w}}(s_i, \pi_{\theta}(s_i))$ 
24      // update target networks
25       $\mathbf{w}_{\text{target}} \leftarrow \rho \mathbf{w}_{\text{target}} + (1 - \rho) \mathbf{w}$ 
26       $\theta_{\text{target}} \leftarrow \rho \theta_{\text{target}} + (1 - \rho) \theta$ 
27 until until stopping criterion is met

```

To stabilize the neural network learning, the algorithm uses mini-batches (see section 4.4) and an experience/replay buffer (see section 4.5), and delayed updates for both critic and actor (see section 4.6). That is, we have an (online) value function approximator $\hat{V}_{\mathbf{w}}$ and a policy π_{θ} with parameters $\mathbf{w}$ and θ , respectively, and corresponding target function approximators with parameters $\mathbf{w}_{\text{target}}$ and θ_{target} .

The action selection requires some attention. We assume $A = [a_{\text{low}}, a_{\text{high}}] \subset \mathbb{R}$. The actor is deterministic, it maps a state s to an action $a' = \pi_{\theta}(s)$. To ensure exploration, DDPG adds random noise to the action, here zero-mean

Gaussian noise with variance σ_a^2 . However, we cannot guarantee that a sample $a' \sim \mathcal{N}(\pi_\theta(s), \sigma_a^2)$ lies inside $[a_{\text{low}}, a_{\text{high}}]$, either because the policy network already suggests an infeasible action or the noise makes a feasible action infeasible. Thus, we have to clip the sample to the feasible domain and consider

$$a = \text{clip}(\pi_\theta(s) + \epsilon, a_{\text{low}}, a_{\text{high}}) \quad (6.10)$$

with $\epsilon \sim \mathcal{N}(0, \sigma_a^2)$.

Let us have a look at the actor update. Where does the expression $\nabla_\theta \hat{Q}_w(s_i, \pi_\theta(s_i))$ in line 20 come from and how do we compute it? Because π_θ is deterministic, we can apply theorem 5.2 and the update rule follows from (5.36). Equation (5.35) shows that

$$\nabla_w \hat{Q}_w(s, \pi_\theta(s)) = \gamma^k \nabla_\theta \pi_\theta(s) \nabla_a \hat{Q}_w(s, \pi_\theta(s)) \Big|_{a=\pi_\theta(s)} . \quad (6.11)$$

As there is a dependence on the iteration t , we also need to keep track of t , for example, by storing tuples $\langle s, a, r, s', t \rangle$ in the experience replay buffer. Thus, for the batch update we have

$$\begin{aligned} \nabla_w \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \gamma^{t_i} \hat{Q}_w(s_i, \pi_\theta(s_i)) \\ = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \nabla_\theta \pi_\theta(s_i) \nabla_a \hat{Q}_w(s_i, a) \Big|_{a=\pi_\theta(s_i)} . \end{aligned} \quad (6.12)$$

However, in general you do not find the time step added in the experience replay buffer, and γ^t is dropped from the actor update. This approximation was already applied in the original DDPG: “In practice, as in commonly done in policy gradient implementations, we ignored the discount in the state visitation distribution” [Lillicrap et al., 2016].

Let us move on to the critic update. For a transition $\langle s, a, r, s', d \rangle$, we compute the targets

$$y = r + \gamma(1 - d) \hat{Q}_{w_{\text{target}}}(s', \pi_{\text{target}}(s')). \quad (6.13)$$

Both target networks are used on the RHS. There is no maximization, the action is given by the target actor. Now we optimize the critic by performing gradient-based minimization of the loss $\ell(\hat{Q}_w(s, a), y)$, typically the halved squared error.

6.4 Twin Delayed Deep Deterministic Policy Gradients (TD3)

The Twin-delayed Deep Deterministic Policy Gradient (TD3) algorithm by [Fujimoto et al. \[2018\]](#) is a common baseline method. It is an off-policy RL approach based on DDPG, see algorithm 14.

The TD3 algorithm uses the “clipped double- Q trick” to avoid overestimation bias, see section 4.7. The algorithm keeps track of two state-action value function approximators $\hat{Q}_{\mathbf{w}_1}$ and $\hat{Q}_{\mathbf{w}_2}$. Because delayed updates are used, there are also two target networks, $\hat{Q}_{\mathbf{w}_{\text{target},1}}$ and $\hat{Q}_{\mathbf{w}_{\text{target},2}}$ with parameters $\mathbf{w}_{\text{target},1}$ and $\mathbf{w}_{\text{target},2}$, respectively. The minimum of the two value estimates, $\min_{j=1,2} \hat{Q}_{\mathbf{w}_{\text{target},j}}(s, a)$ is used for computing the targets for the value functions. This may lead to an underestimation, which has been empirically shown to be preferable to an overestimation. Only the first function approximators $\hat{Q}_{\mathbf{w}_1}$ is considered when updating the policy parameters (in contrast to algorithm 16).

Algorithm 15 shows TD3 in pseudocode for an action space $A = [a_{\text{low}}, a_{\text{high}}] \subset \mathbb{R}$. The algorithm generates trajectories from a deterministic policy. For exploration, it adds Gaussian noise to the sampled actions during learning. As in algorithm 14, the perturbed actions are clipped to the feasible region $[a_{\text{low}}, a_{\text{high}}]$.

An experience replay buffer stabilizes the learning. In every T_{update} steps or when an episode ends, TD3 optimizes the critic networks. A number of $n_{\text{gradient-steps}}$ gradient steps are performed. Given $\langle s, a, r, a', d \rangle$ sampled from the experience replay buffer, the state value $V(s')$ is approximated by $\min_{j=1,2} \hat{Q}_{\mathbf{w}_{\text{target},j}}(s', \bar{a})$ when computing the targets for the critics for a state s . Here, $\bar{a}$ is sampled and perturbed anew and is not taken from the replay buffer.

The actor update in line 21 basically follows DDPG. It just considers the first value function approximator.

In algorithm 15, two noise distributions are used to perturb actions. In line 5, the noise is added to ensure exploration. But why is noise also added in line 14 when computing the targets for the critic updates? The reason is to smooth out peaks in the value function approximators. The operation can be thought of as a Monte Carlo approximation of applying a Gaussian smoothing filter to the value function before computing a value for an action. This is called target policy smoothing [[Achiam, 2018](#)]. The noise distributions in lines 5 and 14 can be different; they can be constant [[Achiam, 2018](#)] or be reduced over time.

In accordance with the literature, algorithm 15 does not keep track of the time within an episode that an experience is made (i.e., no time index is stored in the replay buffer). That is, for $\gamma < 1$ we cannot take the γ^t scaling factor into account when computing the policy gradient following theorem 5.2. In algorithm 14, we therefore added the time steps to the transitions in the

buffer. This was done to demonstrate how one could handle this issue – however, it is not part of the original DDPG.

To get a better idea about the hyperparameters, we look at their defaults in the implementation by Achiam [2018]. The size of the experience replay buffer M is set to 10^6 . Of course, we need to fill M a bit before we can start updating. By default, learning starts after M has 1000 entries. The update interval is set to $T_{\text{update}} = 50$ and the number of gradient steps is tied to this number $n_{\text{gradient-steps}} = T_{\text{update}}$. The batch size is set to $n_{\text{batch}} = 100$. The target policy and target value functions are updated every second gradient step ($T_{\text{update-target}} = 2$) with smoothing $\rho = 0.995$.

6.5 Regularization of stochastic actors

Learning explicit stochastic policies has conceptual advantages over learning deterministic policies as discussed on page 22. Among others, the exploration is directly part of the policy.

If no countermeasures are taken, policy gradient methods applied to stochastic policies tend to suffer from two problems. As discussed earlier, too large parameter updates can make learning unstable and may lead to catastrophic forgetting of previously learned behavior. This holds for both stochastic and deterministic policies. Additionally, when learning stochastic policies, there is also a high risk of premature convergence to a suboptimal deterministic policy.

Regularization of policy updates can address these issues. In the following, we discuss two related popular regularization strategies, entropy and KL regularization.

6.5.1 Entropy bonus

Adding an entropy bonus is a common way to preserve exploration. The Shannon entropy $H(p) = \mathbb{E}_{x \sim p}[-\ln p(x)]$ of a discrete distribution p is a measure of “randomness” or “surprise”. The uniform distribution has maximum entropy; if p corresponds to a deterministic function, then $H(p)$ is the lowest. In RL, incentivizing higher entropy of a probabilistic policy supports exploration and prevents collapse to a deterministic policy. For continuous distributions p , the concept is extended to differential entropy $H(p) = -\int p(x) \ln p(x) dx$. While the Shannon entropy of a discrete distribution is always non-negative, the differential entropy can be negative.

This regularization approach implements the maximum entropy principle in RL [Ziebart et al., 2008], which can be stated as choosing the distribution with the highest entropy that solves a given task. For example, if in a state

Algorithm 15: Twin Delayed DDPG (TD3)

Input: policy π_θ , $\hat{Q}_{w_1}$ and $\hat{Q}_{w_2}$, target smoothing coefficient ρ , feasible region $[a_{\text{low}}, a_{\text{high}}]$, number of gradient steps $n_{\text{gradient-steps}}$, update intervals T_{update} and $T_{\text{update-target}}$, experience replay buffer M , batch size n_{batch} , parameters of gradient-based learning algorithms, parameters σ_a^2 and σ_d^2 of noise distributions

```

1 for  $j \in \{1, 2\}$  do  $w_{\text{target},j} \leftarrow w_j$  // init target critics
2  $\theta_{\text{target}} \leftarrow \theta$  // init target actor
3  $s \sim p_{\text{start}}$ 
4 repeat
    // choose action with added noise for exploration
5   sample  $a = \text{clip}(\pi_\theta(s) + \epsilon, a_{\text{low}}, a_{\text{high}})$  with  $\epsilon \sim \mathcal{N}(0, \sigma_a^2)$ 
6   take action  $a$ , observe  $r, s', d$ 
7   store  $\langle s, a, r, s', d \rangle$  in  $M$ 
8    $s \leftarrow s'$ 
9   if  $d = 1$  then  $s \sim p_{\text{start}}$  // start of new episode
10  if  $T_{\text{update}}$  steps passed since last update then
11    for  $k = 1, \dots, n_{\text{gradient-steps}}$  gradient steps do
12      sample mini-batch  $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
13      // compute targets
14      for  $i = 1, \dots, n_{\text{batch}}$  do
15         $\bar{a}_i = \text{clip}(\pi_{\theta_{\text{target}}}(s_i) + \epsilon_i, a_{\text{low}}, a_{\text{high}})$  with  $\epsilon_i \sim \mathcal{N}(0, \sigma_a^2)$ 
16         $y_i = r_i + \gamma(1 - d_i) \min_{j \in \{1, 2\}} \hat{Q}_{w_{\text{target},j}}(s'_i, \bar{a}_i)$ 
17      // update critics
18      for  $j = 1, 2$  do
19        perform gradient-based minimization using:
20         $\nabla_{w_j} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \frac{1}{2} (y_i - \hat{Q}_{w_j}(s_i, a_i))^2$ 
21      if  $k \bmod T_{\text{update-target}} = 0$  then // time for target update
22        // update actor
23        perform gradient-based maximization using:
24         $\nabla_\theta \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \hat{Q}_{w_1}(s'_i, \pi_\theta(s_i))$ 
25        // update target critic networks
26        for  $j = 1, 2$  do  $w_{\text{target},j} \leftarrow \rho w_{\text{target},j} + (1 - \rho) w_j$ 
27        // update target actor network
28         $\pi_{\theta_{\text{target}}} \leftarrow \rho \theta + (1 - \rho) \pi_{\theta_{\text{target}}}$ 
29  until until stopping criterion is met

```

two different actions lead to optimal behavior, the maximum entropy principle suggests making them equally likely instead of converging to a deterministic optimal policy.

We would like to encourage policies with large entropy, that is, policies that explore a lot. A straight-forward way to do so is to consider the learning problem

$$\pi^* = \arg \max_{\pi} \{J(\pi) + \alpha_H \mathbb{E}^{\pi}(H(\pi(\cdot | s)))\} \quad (6.14)$$

$$= \arg \max_{\pi} \left\{ \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right] + \alpha_H \mathbb{E}^{\pi}(H(\pi(\cdot | s))) \right\} , \quad (6.15)$$

where $\mathbb{E}^{\pi}$ is the expectation w.r.t. sequences following π . The parameter $\alpha_H > 0$ controls the trade-off between maximizing the return and maximizing the entropy and is referred to as temperature (parameter). A popular example using this regularization in deep RL is the work by Mnih et al. [2016], but the idea goes back to Williams and Peng [1991].

This regularization changes the REINFORCE-style actor gradient (see algorithm 11) to

$$\gamma^t \nabla_{\theta} \ln \pi_{\theta}(s_t, a_t) (R_t - \hat{V}_{\mathbf{w}}(s_t)) + \alpha_H \nabla_{\theta} H(\pi_{\theta}(\cdot | s_t)) . \quad (6.16)$$

An additional term in the objective that rewards policies with high entropy is also called an entropy bonus.

An alternative approach (e.g., see Haarnoja et al. [2019b], Achiam [2018]) to incorporate entropy maximization is to consider

$$\pi^* = \arg \max_{\pi} \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha_H H(\pi(\cdot | s_t)) \right) \right] , \quad (6.17)$$

which leads to the definitions

$$V_H^{\pi}(s) = \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha_H H(\pi(\cdot | s_t)) \right) \middle| s_0 = s \right] \quad (6.18)$$

and

$$Q_H^{\pi}(s, a) = \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} + \alpha_H \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot | s_t)) \middle| s_0 = s, a_0 = a \right] , \quad (6.19)$$

where the definition of $Q_H^{\pi}(s, a)$ includes the entropy terms for all time steps except the first [Achiam, 2018]. These expressions are related via

$$V_H^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q_H^{\pi}(s, a)] + \alpha_H H(\pi(\cdot | s)) , \quad (6.20)$$

which brings in the missing entropy term related to the initial time step, and

$$\begin{aligned} Q_H^{\pi}(s, a) &= \mathbb{E}^{\pi} [r_{t+1} + \gamma (Q_H^{\pi}(s_{t+1}, a_{t+1}) + \alpha_H H(\pi(\cdot | s_{t+1}))) \mid s_t = s, a_t = a] \\ & \quad (6.21) \end{aligned}$$

$$= \mathbb{E}^{\pi} [r_{t+1} + \gamma V_H^{\pi}(s_{t+1}) \mid s_t = s, a_t = a] . \quad (6.22)$$

By the definition of the entropy, we have

$$\begin{aligned} Q_H^\pi(s, a) &= \mathbb{E}^\pi [r_{t+1} + \gamma (Q_H^\pi(s_{t+1}, a_{t+1}) + \alpha_H H(\pi(\cdot | s_{t+1}))) | s_t = s, a_t = a] \end{aligned} \quad (6.23)$$

$$= \mathbb{E}^\pi [r_{t+1} + \gamma (Q_H^\pi(s_{t+1}, a_{t+1}) - \alpha_H \log \pi s_{t+1} a_{t+1}) | s_t = s, a_t = a] \quad (6.24)$$

$$= \mathbb{E}^\pi [r_{t+1} + \gamma Q_H^\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a] - \alpha_H \gamma \mathbb{E}^\pi [\log \pi(\cdot | s_{t+1}) | s_t = s] . \quad (6.25)$$

In practice, we make use of a Monte Carlo estimation of the last expression. Typically, in the update computation $s_t, a_t, r_{t+1}, s_{t+1}$ are taken from an experience replay buffer, but the action in state s_{t+1} is sampled anew from the current policy π . Thus, the Monte Carlo approximation reads

$$Q_H^\pi(s_t, a_t) \approx r_{t+1} + \gamma (Q_H^\pi(s_{t+1}, \bar{a}_{t+1}) - \alpha_H \log \pi(\bar{a}_{t+1} | s_{t+1})) \quad (6.26)$$

with $\bar{a}_{t+1} \sim \pi(\cdot | s_{t+1})$.

Choosing the temperature

Choosing the temperature parameter α_H is not straight-forward. The scaling of the entropy depends on the dimensionality. For example, for a discrete distribution with n outcomes the maximum entropy is $\ln n$, and the entropy of an n -dimensional Gaussian distribution scales linearly in n (and the logarithm of the determinant of the covariance matrix). The latter case shows that rescaling a continuous action space also changes the magnitude of the entropy.

To increase robustness, it has been suggested to adapt α_H during learning [Haarnoja et al., 2019b]. The idea is to fix a target entropy H_{target} and try to achieve $\mathbb{E}^\pi(H(\pi(\cdot | s))) \approx H_{\text{target}}$. To address this soft constraint, the temperature can be updated as follows [Haarnoja et al., 2019b]:

$$\alpha_H \leftarrow \alpha_H - \lambda_H (\mathbb{E}^\pi(H(\pi(\cdot | s))) - H_{\text{target}}) \quad (6.27)$$

with a positive learning rate λ_H . That is, if the entropy is larger than H_{target} , the temperature (i.e., the entropy bonus) is reduced, and if the entropy is smaller than H_{target} vice versa. The changes are proportional to the constraint violation. You can view α_H as a Lagrange multiplier for the hard constraint $\mathbb{E}^\pi(H(\pi(\cdot | s))) - H_{\text{target}} = 0$. In practice, $\mathbb{E}^\pi(H(\pi(\cdot | s)))$ is replaced by a Monte Carlo estimation, see algorithm 16.

6.5.2 KL regularization

To prevent too large updates in a gradient step that can make learning unstable (e.g., lead to divergence, catastrophic forgetting, loss of exploration), we can force the policy not to deviate too strongly from some reference policy. Let both the current policy and the reference policy be stochastic. Thus, we need to compare distributions (over actions), and we use the Kullback–Leibler (KL) divergence to do so. The KL divergence

$$\text{KL}[p \parallel q] = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] = \mathbb{E}_{x \sim p} \left[-\log \frac{q(x)}{p(x)} \right] \quad (6.28)$$

measures the difference between two distributions p and q .

To avoid too large updates of the policy parameters, we monitor the KL divergence

$$\text{KL}[\pi(\cdot | s) \parallel \pi_{\text{ref}}(\cdot | s)] \quad (6.29)$$

between the current policy π and a reference policy π_{ref} , which can be estimated over the observed states in a batch. The idea of KL regularization is to keep π and π_{ref} sufficiently close. An upper bound on the divergence can serve as a constraint [Schulman et al., 2015], or the estimated divergence can be added as a weighted penalty (to be minimized) to the learning objective.

We can rewrite the KL definition as

$$\text{KL}[p \parallel q] = \mathbb{E}_{x \sim p} [-\log q(x)] - H(p) \quad (6.30)$$

The first term is the cross-entropy (giving (2.57) its name), the second term is simply the entropy of p . If the reference policy π_{ref} is uniform, then minimizing (6.29) reduces to maximizing the entropy, and we arrive at the entropy bonus as a special case of KL regularization (for uniform q , the $\mathbb{E}_{x \sim p} [-\log q(x)]$ term is independent of p).

The estimate

$$\text{KL}[p \parallel q] = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] = \mathbb{E}_{x \sim p} \left[-\log \frac{q(x)}{p(x)} \right] \quad (6.31)$$

can have a high variance. Instead of the KL divergence, we could optimize another f -divergence, for example

$$\mathbb{E}_{x \sim p} \left[\frac{1}{2} \left(\log \frac{p(x)}{q(x)} \right)^2 \right] \quad (6.32)$$

An alternative way to reduce the variance is to introduce a (negatively correlated) control variate (see section 2.3):

$$\mathbb{E}_{x \sim p} \left[\frac{q(x)}{p(x)} - 1 \right] = 0 \quad (6.33)$$

Adding the control variate to the $\text{KL}[p \parallel q]$ term gives:

$$\mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} + \frac{q(x)}{p(x)} - 1 \right] = \mathbb{E}_{x \sim p} \left[\frac{q(x)}{p(x)} - \log \frac{q(x)}{p(x)} - 1 \right] \quad (6.34)$$

This is the KL regularization term used in Group Relative Policy Optimization (GRPO), see section 6.7.4.

6.6 Soft Actor Critic

The Soft Actor Critic (SAC) is a state-of-the-art off-policy algorithm (e.g., Haarnoja et al., 2019a). Several variants exist, we basically follow Haarnoja et al. [2019b] and Baykal et al. [2025]. SAC adds an entropy bonus to ensure exploration, see section 6.5.1, and the “clipped double- Q -trick” to address overestimation bias, see section 4.7 and section 6.4. The SAC pseudocode is shown in algorithm 16. The approach is similar to TD3 (see algorithm 15), the main difference is that TD3 relies on a deterministic policy perturbed by noise, while SAC uses a stochastic policy and an entropy bonus.

SAC actor update

In section 4.7 we discussed that overestimation can be a problem in off-policy methods, and that the problem is amplified if function approximators are used. Fujimoto et al. [2018] argue that actor-critic methods also suffer from overestimation bias, even if no explicit max operation is performed.

In SAC, the “clipped double- Q trick” is used to avoid overestimation bias [Fujimoto et al., 2018], see section 4.7. However, it is applied slightly differently compared to TD3. As in TD3, two state-action value function approximators $\hat{Q}_{\mathbf{w}_1}$ and $\hat{Q}_{\mathbf{w}_2}$ with corresponding $\hat{Q}_{\mathbf{w}_{\text{target},1}}$ and $\hat{Q}_{\mathbf{w}_{\text{target},2}}$ are considered, and their minimum is used when computing the target for the value function updates and also – and this is different from TD3 – in the actor update. That is, $\hat{Q}_{\mathbf{w}_{\text{target},1}}$ has no special role compared to $\hat{Q}_{\mathbf{w}_{\text{target},2}}$.

The TD3 method uses delayed updates for actor as well as critics. In contrast, SAC only uses delayed updating for the value functions.

The SAC algorithm uses a stochastic policy, for example, draws actions from a multi-variate Gaussian, the parameters of which are given by a neural network. To optimize the parameters of the policy, the “reparameterization trick” is used, see section 2.3.3.2. The stochastic policy π_{θ} is replaced by a deterministic policy $\bar{\pi}_{\theta}$, where for all states and actions the probability $\pi(a \mid s)$ is equal to the probability of $a = \bar{\pi}_{\theta}(s, \xi)$ given ξ is drawn from some fixed tractable distribution q .

To support exploration and to prevent too large policy updates, the SAC algorithm adds an entropy bonus as in (6.17), see section 6.5.1. That is, the

SAC critics learn value functions as described by (6.18), however, we drop the H subscript in the description of SAC to keep the notation uncluttered.

Example 6.1 Consider a policy π_θ where the continuous action in state s is the realization of a Gaussian random variable with mean $\mu_\theta(s)$ and variance $\sigma_\theta(s)$ squashed by a tanh function. The squashing avoids too extreme action values and plays the same role as the clipping in (6.10). Then we can use the reparameterization

$$\bar{\pi}_\theta(s, \xi) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \cdot \xi) \quad , \quad \xi \sim \mathcal{N}(0, 1) \quad (6.35)$$

and, for example, compute

$$\nabla_\theta \mathbb{E}_{a \sim \pi_\theta(s, \cdot)} [Q^{\pi_\theta}(s, a)] = \nabla_\theta \mathbb{E}_{a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))} [Q^{\pi_\theta}(s, a)] \quad (6.36)$$

$$= \mathbb{E}_{\xi \sim \mathcal{N}(0, 1)} [\nabla_\theta Q^{\pi_\theta}(s, \bar{a}_\theta(s, \xi))] \quad , \quad (6.37)$$

where $\bar{a}_\theta(\cdot, \cdot)$ is a deterministic function.

The SAC uses an experience replay buffer and the actor update involves an entropy bonus, see section 6.5.1.

Putting everything together, the actor update performs $n_{\text{gradient-steps}}$ gradient-based maximization steps using the gradient

$$\nabla_\theta \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \underbrace{\left(\underbrace{\min_{j \in \{1, 2\}} \hat{Q}_{\mathbf{w}_{\text{target}, j}}(s_i, \bar{\pi}_\theta(s_i, \xi_i))}_{\text{clipped double-}Q} - \underbrace{\alpha_H \log \bar{\pi}_\theta(s_i, \xi_i)}_{\text{entropy bonus}} \right)}_{\substack{\text{reparameterization} \\ \text{trick} \quad \quad \text{reparameterization} \\ \text{trick}}} \quad (6.38)$$

replaces $\hat{Q}_{\mathbf{w}}(s_i, a_i)$ in DDPG

for a mini-batch $\mathcal{D} = \{(s_i, a_i, r_i, s'_i, d_i, t_i)\}_{i=1}^{n_{\text{batch}}}$. As $\bar{\pi}_\theta$ is a deterministic function, we can apply the policy gradient theorems for deterministic policies to compute the terms $\nabla_\theta \hat{Q}_{\mathbf{w}_j}(s_i, \bar{\pi}_\theta(s_i, \xi_i))$. The update step can be viewed as the DDPG update with clipped double- Q trick, entropy bonus, and reparameterization trick added.

SAC critic update

The SAC algorithm considers an entropy bonus as in (6.18). Following (6.26), given a single sample $\langle s_i, a_i, r_i, s'_i, d_i, t_i \rangle$, SAC computes the target

$$y_i = r_i + \gamma(1 - d_i) \left(\min_{j \in \{1, 2\}} \hat{Q}_{\mathbf{w}_{\text{target}, j}}(s_i, \bar{a}_i) - \alpha \log \pi_\theta(s_i, \bar{a}_i) \right) \quad , \quad (6.39)$$

where $\bar{a}_i \sim \pi_\theta(\cdot | s_i)$ is drawn anew and is not taken from the experience replay buffer. The two state-value function approximators are updated by performing gradient-based minimization based on

$$\nabla_{\mathbf{w}_j} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \frac{1}{2} \left(y_i - \hat{Q}_{\mathbf{w}_j}(s_i, a_i) \right)^2 \quad (6.40)$$

for $j \in \{1, 2\}$.

Algorithm 16: Soft Actor Critic (SAC)

Input: policy π_θ , $\hat{Q}_{\mathbf{w}_1}$ and $\hat{Q}_{\mathbf{w}_2}$, target smoothing coefficient ρ , target entropy H_{target} , initial entropy bonus weight α_H , number of gradient steps $n_{\text{gradient-steps}}$, update interval T_{update} , experience replay buffer M , batch size n_{batch} , parameters of gradient-based learning algorithms (including λ_H)

```

1 for  $j \in \{1, 2\}$  do  $\mathbf{w}_j^{\text{target}} \leftarrow \mathbf{w}_j$  // init target networks
2  $s \sim p_{\text{start}}$ 
3 repeat
4   select  $a \sim \pi_\theta(\cdot | s)$  and execute  $a$ 
5   observe  $r, s', d$ 
6   store  $\langle s, a, r, s', d \rangle$  in  $M$ 
7    $s \leftarrow s'$ 
8   if  $d_t = 1$  then  $s_t \sim p_{\text{start}}$  // start of new episode
9   if  $d = 1$  or  $T_{\text{update}}$  steps passed since last update then
10    for  $n_{\text{gradient-steps}}$  gradient steps do
11      sample mini-batch  $\mathcal{D} = \{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
12      // compute targets
13      for  $i = 1, \dots, n_{\text{batch}}$  do
14        draw  $\bar{a}_i \sim \pi_\theta(\cdot | s_i)$ 
15         $y_i = r_i + \gamma(1 - d_i) \left( \min_{j=1,2} \hat{Q}_{\mathbf{w}_{\text{target},j}}(s_i, \bar{a}_i) - \alpha \log \pi_\theta(s_i, \bar{a}_i) \right)$ 
16      // update critic
17      for  $j = 1, 2$  do
18        perform gradient-based minimization using:
19         $\nabla_{\mathbf{w}_j} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \frac{1}{2} \left( y_i - \hat{Q}_{\mathbf{w}_j}(s_i, a_i) \right)^2$ 
20      // update actor
21      for  $i = 1, \dots, n_{\text{batch}}$  do  $\xi_i \sim q$  // reparameterization
22      perform gradient-based maximization using:
23       $\nabla_\theta \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \left( \min_{j=1,2} \hat{Q}_{\mathbf{w}_j}(s_i, \pi_\theta(s_i, \xi_i)) - \alpha_H \log \pi_\theta(s_i, \xi_i) \right)$ 
24      // update temperature
25       $\alpha_H \leftarrow \alpha_H - \lambda_H \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (-\log \pi_\theta(s_i, \xi_i) - H_{\text{target}})$ 
26      // update target networks
27      for  $j = 1, 2$  do  $\mathbf{w}_j^{\text{target}} \leftarrow \rho \mathbf{w}_j^{\text{target}} + (1 - \rho) \mathbf{w}_j$ 
28 until until stopping criterion is met
```

The SAC agent is described in algorithm 16. As in algorithm 15 we do not keep track of the time within an episode when collecting experiences.

Haarnoja et al. [2019b] set the target smoothing coefficient $\rho = 0.005$, the number of gradient update steps $n_{\text{gradient-steps}} = 1$ and the experience replay buffer size to 10^6 . The policies considered by Haarnoja et al. [2019b] are multi-variate Gaussian distributions. The mean and parameters of the covariance matrix are provided by a neural network head. For this scenario, the target (differential) entropy H_{target} is set to the negative dimensionality of the action space. The default setting for the initial entropy bonus weight in the implementation by Baykal et al. [2025] is $\alpha_H = 1$. Eimer et al. [2023] conducted a hyperparameter study for deep RL algorithms, including SAC. In their experiments, the learning rate was important but in particular also the target smoothing coefficient ρ and the experience replay buffer size. The code by Raffin et al. [2021] can be regarded as a reference implementation of SAC; details of the algorithm matter, and some general implementation details will be discussed in section 6.7.3 in the context of PPO.

6.7 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a popular deep RL method [Schulman et al., 2017]. It uses importance weighting (see section 2.3.1) to become “more” on-policy. It works for discrete and continuous action spaces and is known to be rather robust. The fact that PPO was used in fine-tuning ChatGPT contributed to its popularity [Ouyang et al., 2022]. The main ingredients of PPO are a surrogate loss function known as CPI loss and a particular way of clipping (see section 4.8).

6.7.1 CPI loss function

Assume that we are given a reference policy π_{ref} . From lemma 5.1 we know that

$$J(\pi) = J(\pi_{\text{ref}}) + \sum_s \eta_{\gamma}^{\pi}(s) \sum_a \pi(a | s) A^{\pi_{\text{ref}}}(s, a) . \quad (6.41)$$

The dependence on η_{γ}^{π} on the RHS makes (5.15) difficult to optimize. Thus, an approximation is introduced:

$$J_{\pi_{\text{ref}}}^{\text{CPI}}(\pi) = \underbrace{J(\pi_{\text{ref}})}_{\substack{\text{can be dropped} \\ \text{when optimizing } \pi}} + \sum_s \eta_{\gamma}^{\pi_{\text{ref}}}(s) \sum_a \pi(a | s) A^{\pi_{\text{ref}}}(s, a) \quad (6.42)$$

When optimizing $J_{\pi_{\text{ref}}}^{\text{CPI}}(\pi)$ w.r.t. to the parameters of π , the $J(\pi_{\text{ref}})$ term can be dropped as it is independent of π .

Let the policy be parameterized by θ . So far, our optimization task reads:

$$\arg \max_{\theta} \sum_s \eta_{\gamma}^{\pi_{\text{ref}}}(s) \sum_a \pi_{\theta}(a | s) A^{\pi_{\text{ref}}}(s, a) \quad (6.43)$$

Applying proposition 5.1, using $\sum_a \pi(a | s) A^{\pi_{\text{ref}}}(s, a) = \mathbb{E}^{\pi}[A^{\pi_{\text{ref}}}(s, a) | s]$, and dropping the γ^t term (i.e., assuming $\gamma = 1$, which is in general an approximation) gives:

$$\arg \max_{\theta} \mathbb{E}^{\pi_{\text{ref}}} \left[\sum_{t=0}^{\infty} \mathbb{E}^{\pi_{\theta}} [A^{\pi_{\text{ref}}}(s_t, a) | s_t] \right] \quad (6.44)$$

We could approximate this expression if we have sampled states following π_{ref} and have samples of actions in these states drawn from π_{θ} . Let us assume that we have access to trajectories following π_{ref} to estimate (6.43). The actions in these trajectories are drawn from π_{ref} and not from π_{θ} . We use importance weighting to correct for this. From

$$\mathbb{E}^{\pi_{\theta}} [A^{\pi_{\text{ref}}}(s, a) | s] = \mathbb{E}^{\pi_{\text{ref}}} \left[\frac{\pi_{\theta}(a | s)}{\pi_{\text{ref}}(a | s)} A^{\pi_{\text{ref}}}(s, a) \mid s \right] . \quad (6.45)$$

we get

$$\arg \max_{\theta} \mathbb{E}^{\pi_{\text{ref}}} \left[\sum_{t=0}^{\infty} \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} A^{\pi_{\text{ref}}}(s_t, a_t) \right] . \quad (6.46)$$

This gives the CPI (named after conservative policy iteration, [Kakade and Langford, 2002](#)) objective, namely to maximize

$$\sum_{t=0}^{T-1} \underbrace{\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}}_{\substack{\text{importance weighting} \\ \rightarrow \text{"more" on-policy}}} \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) \quad (6.47)$$

based on one or several (sub-)episodes

$$s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, s_T \quad (6.48)$$

following π_{ref} , see also section 6.2.1.

6.7.2 PPO clipping

In the iterative optimization of a policy, it is a problem if a single step leads to a too large update, because a single step may be a very noisy approximation of the proper gradient direction; and too large a step may take the parameters

outside the region where the first-order approximation underlying a gradient update is reasonable. To constrain the change of the policy, the update step (typically the approximated gradient) can be constrained, for example, by limiting the length of the update vector or clipping the components of the update vector as discussed in section 4.8.

The PPO uses a very particular way of clipping. The reweighted advantages in (6.47) are clipped as follows:

$$\min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} \hat{A}^{\pi_{\text{ref}}}(s_t, a_t), \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) \right) \quad (6.49)$$

That is, the reweighted advantage term is considered in the objective if $\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} \in [1 - \epsilon, 1 + \epsilon]$ or if the gradient direction does not point away from that interval.

To understand the second condition, assume $\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} \notin [1 - \epsilon, 1 + \epsilon]$. If the clipping is active, the gradient of the clipping term w.r.t. θ is zero:

$$\begin{aligned} \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} \notin [1 - \epsilon, 1 + \epsilon] \Rightarrow \\ \nabla_{\theta} \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) = \mathbf{0} \end{aligned} \quad (6.50)$$

Now consider two cases. First, assume that $\hat{A}^{\pi_{\text{ref}}}(s_t, a_t) > 0$. That is, the action a_t has performed better than expected, and therefore optimization wants to increase $\pi_{\theta}(a_t | s_t)$. In this case, (6.49) is equal to

$$\begin{aligned} \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) = \\ \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, 1 + \epsilon \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) \end{aligned} \quad (6.51)$$

If in this case $\pi_{\theta}(a_t | s_t)$ is already more likely compared to the reference policy by a factor larger than $1 + \epsilon$, then it is not further updated because the clipping is active and the gradient w.r.t. the policy parameters is zero. If $\hat{A}^{\pi_{\text{ref}}}(s_t, a_t) < 0$ the optimization wants to decrease $\pi_{\theta}(a_t | s_t)$ and (6.49) becomes $\max \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, 1 - \epsilon \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t)$, the gradient of which is zero if already $\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)} < 1 - \epsilon$. Thus, the PPO objective wants to confine $\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}$ in $[1 - \epsilon, 1 + \epsilon]$.

Algorithm 17 shows PPO with clipping in pseudocode. Several PPO variants were already proposed by Schulman et al. [2017]. They compared PPO using the clipping objective as shown in algorithm 17 with PPO using KL regularization instead (see section 6.5.2), with clipping giving better results. The clipping is enough to ensure that the policy does not change too much,

Algorithm 17: Proximal Policy Optimization (PPO)

Input: policy π , value function $\hat{V}_{\mathbf{w}}$, sub-episode length T , number of sub-episodes T_{update} , number of gradient steps $n_{\text{gradient-steps}}$, parameters of gradient-based learning algorithms, mini-batch size

```

1 repeat
2    $M = \emptyset$  // reset experience replay buffer
3    $d_0 \leftarrow 1$ 
4   for  $e = 1, \dots, T_{\text{update}}$  do // gather experience
5     if  $d_t = 1$  then
6        $t \leftarrow 0$ 
7        $s_t \sim p_{\text{start}}$ 
8     repeat
9       perform action  $a_t \sim \pi_{\theta}(\cdot | s_t)$ 
10      observe  $r_{t+1}, s_{t+1}, d_{t+1}$ 
11       $p_t = \pi_{\theta}(a_t | s_t)$  // 'old' probabilities ( $\pi_{\text{ref}}(\cdot | s_t)$ )
12       $t \leftarrow t + 1$ 
13    until  $t = T$  or  $d_t = 1$ 
14    for  $i = 0, \dots, t - 1$  do
15       $\hat{R}_i(s_i, a_i) = r_{i+1} + \gamma r_{i+2} + \dots + \gamma^{t-i-1} r_t + \gamma^{t-i} \hat{V}_{\mathbf{w}}(s_t)$ 
16       $\hat{A}_i(s_i, a_i) = \hat{R}_i(s_i, a_i) - \hat{V}_{\mathbf{w}}(s_i)$ 
17      store  $\langle s_i, a_i, p_i, \hat{R}_i, \hat{A}_i \rangle$  in  $M$ 
18    for  $i = 1, \dots, n_{\text{gradient-steps}}$  do
19      sample mini-batch  $\mathcal{D}$  from  $M$ 
20      // actor update using PPO-Clip objective
21      do gradient-based maximization following
          
$$\nabla_{\theta} \frac{1}{|\mathcal{D}|} \sum_{\langle s, a, p, \hat{R}, \hat{A} \rangle \in \mathcal{D}} \min \left( \frac{\pi_{\theta}(a | s)}{p} \hat{A}, \text{clip} \left( \frac{\pi_{\theta}(a | s)}{p}, 1 - \epsilon, 1 + \epsilon \right) \hat{A} \right)$$

22      // update critic
23      do gradient-based minimization following
          
$$\nabla_{\mathbf{w}} \frac{1}{|\mathcal{D}|} \sum_{\langle s, a, p, \hat{R}, \hat{A} \rangle \in \mathcal{D}} \left( \hat{R} - \hat{V}_{\mathbf{w}}(s) \right)^2$$

24 until stopping criterion is met

```

making additional KL regularization unnecessary, which is supported by the empirical study by [Andrychowicz et al. \[2021\]](#).

The PPO algorithm maintains an experience replay buffer (see section 4.5), which is, however, flushed after each update round, where each update round comprises $n_{\text{gradient-steps}}$ iteration of a mini-batch gradient-based optimization algorithm such as Adam. All elements in the buffer come from T_{update} sub-episodes following the same policy π_{ref} .

Emptying the buffer M and ensuring that the policy does not change too much in one update round has the effect that the distribution of states in M , which is based on π_{ref} , does not deviate too much from a distribution following π_{θ} . This justifies the approximation leading to (6.42) and referring to PPO as being an on-policy method, although a replay buffer is used and the importance weighting only addresses changes in the probability of taking an action in a state but not changes in probability of visiting a state.

Schulman et al. [2017] added an entropy bonus to the objective (see section 6.5.1). However, in experiments on continuous control tasks by Andrychowicz et al. [2021], it was found that an entropy bonus is not necessary if the initialization of the policy guarantees sufficient exploration.

The critic $\hat{V}_{\mathbf{w}}$ and the actor π_{θ} typically share parameters (parts of the network), and the corresponding objectives can be weighted (with signs adjusted so that both need to be either minimized or maximized), added, and optimized jointly. If an entropy bonus is used, it is also simply added to the weighted sum [Schulman et al., 2017]. Thus, in practice the algorithm performs gradient-based optimization of a single multi-term loss function with the gradient being determined using automatic differentiation.

6.7.3 PPO bells and whistles

The choice of hyperparameters matters in deep RL, and small implementation details can lead to significant differences in performance.

Engstrom et al. [2020] discusses important details in the PPO reference implementation [Raffin et al., 2021], and Andrychowicz et al. [2021] and Eimer et al. [2023] demonstrate the importance of tuning the hyperparameters.

Eimer et al. also report which hyperparameters were particularly important in their PPO experiments: the learning rate (of course), the clip parameters ϵ and ϵ_V (see below), the parameter bounding the gradient norm (see below), the coefficient weighting the entropy bonus (set to zero by default), and the GAE parameter λ_{TD} (see below).

Andrychowicz et al. [2021] studied on-policy RL algorithms for continuous control problems. For learning the policy, the PPO objective gave the best results in their comparison; they recommend $\epsilon = 0.25$ as a starting point.

In the following, we will discuss the details of the implementation of PPO beyond algorithm 17 based on the aforementioned studies.

Generalized advantage estimation

The PPO algorithm can use generalized advantage estimation (GAE), see section 6.2.1. This introduces a new hyperparameter λ_{TD} . Eimer et al. [2023] show the influence of this hyperparameter, however, they observe a trend in their experiments, which suggest the starting point $\lambda_{\text{TD}} = 0.85$. Andrychow-

icz et al. [2021] also recommend GAE, suggesting $\lambda_{\text{TD}} = 0.9$ as a starting point.

Value function clipping

The PPO clipping (6.49) prevents too large changes in the policy. The same idea can be applied to the value function update. Instead of the squared error $\nabla_{\mathbf{w}} \frac{1}{|\mathcal{D}|} \sum_{\langle s, a, p, \hat{R}, \hat{A} \rangle \in \mathcal{D}} \left(\hat{R} - \hat{V}_{\mathbf{w}}(s) \right)^2$ in line 21 of algorithm 17, the default implementation considers a PPO-style clipped version:

$$\begin{aligned} \ell_{\text{PPO}}(\hat{R}, \hat{V}_{\mathbf{w}}(s)) = \\ \max \left((\hat{R} - \hat{V}_{\mathbf{w}}(s))^2, (\hat{R} - \text{clip}(\hat{V}_{\mathbf{w}}(s), \hat{V}_{\mathbf{w}}(s) - \epsilon_V, \hat{V}_{\mathbf{w}}(s) + \epsilon_V))^2 \right) \end{aligned} \quad (6.52)$$

This introduces a new hyperparameter ϵ_V , which is set to $\epsilon_V = 0.5$ by default.

Reward scaling and clipping

Large rewards can lead to large gradients, which can destabilize training. Thus, rewards are often transformed. The PPO reference implementation supports reward scaling. The reward scaling updates running statistics of the accumulated discounted rewards $\hat{R}$ (see line 15) and divides the rewards r_t by the standard deviation of the accumulated discounted rewards [Engstrom et al., 2020]. Furthermore, simply clipping the reward to a prespecified interval is also supported.

Gradient clipping

As discussed above, PPO performs gradient-based optimization of a single multi-term loss function with the gradient being determined using automatic differentiation. The reference implementation supports clipping of the gradient of this combined loss function, limiting the norm as in (4.25), which introduces the hyperparameter g_{max} .

Miscellaneous deep learning tricks of the trade

The reference PPO implementation [Raffin et al., 2021] applies some standard techniques to improve deep neural network training. We discuss them based on Engstrom et al. [2020].

The inputs to the network are standardized, that is, normalized to mean-zero and unit variance. Similarly to clipping the rewards, the state observations can also be clipped.

According to Engstrom et al. [2020], the neural networks use orthogonal initialization [Saxe et al., 2014] and hyperbolic tangent activation functions. The Adam optimizer, see section 2.2.3, is employed and optionally uses learn-

ing rate annealing. [Andrychowicz et al. \[2021\]](#) also recommend Adam with weight decay (see algorithm 3).

6.7.4 Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) has been used to train/tune large language models (LLMs) in DeepSeekMath/DeepSeek [\[Shao et al., 2024\]](#).

Large language models are generative models, they sample a response given an input (prompt). Repeated sampling creates a group of responses. The GRPO approach to tuning LLMs does not use a learned value function, the target is computed by evaluating a group of responses returned to the same question. Each response in the group is evaluated. The advantages of the individual responses are computed relative to the group mean normalized by the standard deviation. Thus, although derived from PPO, GRPO is no actor-critic method but actor-only. The algorithm uses KL regularization as in (6.34) added as an additional loss term.

Chapter 7

Summary

Deep neural networks have led to tremendous progress in applied RL. They

- can learn highly non-linear functions, which allows them to model strong nonlinear interactions, represent multimodal functions, and approximate discontinuities;
- have proven to be applicable to large input spaces;
- can learn high-level representations from low-level sensory input;
- allow incorporation of prior knowledge in the network architecture (CNNs, RNNs, ...);
- allow iterative optimization using stochastic gradient descent.

However, training deep RL systems

- is resource intensive, needing a lot of data and compute;
- can be unstable;
- bears the risk of premature convergence to a suboptimal policy;
- involves many hyperparameters, which are difficult to choose.

Modern deep RL algorithms try to address these issues. Algorithmic improvements make learning more efficient; for example, using a prioritized experience replay buffer increases data efficiency. However, the success of supervised deep learning is partially based on the increasing availability of resources, on more training data, and more efficient compute to use it – and the same holds for RL: the big success stories are often based on big compute resources.

Training a deep RL agent is, in general, more difficult compared to training a deep neural network for a supervised learning task. Simply replacing a look-up table by a deep neural network in general does not work well, even if one applies the standard techniques from supervised learning, such as mini-batch learning, gradient clipping, and weight decay, to stabilize the learning process. Thus, many of the techniques discussed in this text address training stability. KL-regularization has been introduced to avoid too large changes in the policy. Delayed updates of (target) value functions and policies increase stability; sampling mini-batches from large experience replay buffers

reduces the bias resulting from collecting data from subsequent steps of the same episode. Advantage learning aims to reduce variance by introducing a control variate; and using different Q -functions for target value computation mitigates overestimation bias.

Premature convergence is prevented by properly addressing the exploration-exploitation tradeoff, one of the fundamental difficulties in RL in general. We discussed adding an entropy bonus to encourage exploration, and special neural network layers can be used to add noise for exploration.

The biggest challenge remains hyperparameter tuning. Supervised deep learning already involves many hyperparameters for defining the network architecture (number of layers, layer types, number of neurons per layer, etc.), the loss function, and the learning algorithm, so many, that it is almost impossible to explore and tune them systematically. Now, in deep RL the hyperparameters of the RL algorithm come on top, and their choice matters [Eimer et al., 2023]. Choosing the right architecture and other hyperparameters is always difficult in deep learning, and the common practice is to start from an architecture that has worked well on some related task. Direct search methods can be used to search the hyperparameter space, for example, one could use the methods discussed in chapter 3 to optimize the hyperparameters of RL agents. However, solving challenging RL problems, including the typical benchmark problems used to evaluate deep RL systems these days, is resource intensive. This renders extensive hyperparameter optimization infeasible, and this is a big problem in RL research. Because the right hyperparameter setting is algorithm and problem dependent, it is difficult to empirically compare methods, which typically requires that even a single configuration is evaluated on a broad set of tasks and repeatedly because of the stochastic nature of the learning process. How can we conclude that algorithm A is better than algorithm B and that empirical differences are not due to implementation details and hyperparameter settings?

Algorithms in this introduction

This text derived several deep RL approaches, see table 7.1 for an overview. Some were selected for didactic reasons, others for their practical relevance. They can be broadly distinguished on the basis of the role of the neural networks. In deep RL networks can be used as critics to represent value functions. However, critic-only methods in the style of Q -learning as described in chapter 4 need to evaluate the expected value of possible actions in each given state, which is difficult if the action space is large, for example, in continuous control tasks. Deep neural networks can also be used to model policies directly, that is, to map states to actions, which allows dealing with continuous action spaces. However, optimizing actors by direct search, as presented in chapter 3, cannot be expected to scale well (but see, e.g., Qiu et al. [2026], who successfully applied evolution strategies to tuning LLMs). The scaling can be addressed by using gradient-descent as discussed in chapter 5.

However, without value function baselines, actor-only policy gradient methods suffer from a high variance of the gradient estimate. Thus, actor-critic methods mark the state of the art in deep RL, allowing for gradient-based learning and large state and action spaces.

Table 7.1 Properties of algorithms presented in the previous chapters.

Algorithm	Actor	Critic	On-policy	Stochastic policy
CE / CMA-ES	✓	–	✓	–
DQN / DDQN	–	✓	–	–
REINFORCE	✓	–	✓	✓
REINFORCE w/ baseline	✓	✓	✓	✓
A ² C	✓	✓	✓	✓
DDPG	✓	✓	–	–
SAC	✓	✓	–	✓
TD3	✓	✓	–	–
PPO	✓	✓	(✓)	✓
GPPO	✓	–	(✓)	✓

The algorithmic details and descriptions of the methods discussed in this introductory text vary across publications and implementations. There is, for example, not single definite SAC or PPO algorithm, the methods have evolved over time. The descriptions in the pseudo-code were chosen for didactic reasons, but should still represent canonical basic versions of the RL approaches. Implementation details matter and can explain significant performance differences [Engstrom et al., 2020]. Anyway, the algorithms presented in this text are better viewed as examples of deep RL concepts. These concepts are a toolbox and can be mixed and matched. For example, an experience replay buffer can be added to algorithm 13, Polyak averaging can be interchanged with interval-based updates, and most of the implementation details discussed in the context of PPO in section 6.7.3 are useful in general.

There are several (mainly historically) relevant algorithms that this text does not cover. The Natural Actor Critic (NAC) proposed by Peters and Schaal [2005] is widely regarded as being outperformed by newer methods such as SAC and PPO in terms of sample efficiency and training stability. To appreciate the algorithmic beauty of the NAC, basic knowledge in information geometry is needed, and introducing this concept is beyond the scope of this text.

Another influential algorithm that is mathematically a bit more involved is Trust Region Policy Optimization (TRPO) [Schulman et al., 2015]. However, for most applications, TRPO has been superseded by SAC and especially PPO, which can also be regarded as an on-policy method and can be seen as a direct replacement for TRPO. The update in TRPO is regularized by using

the KL divergence as a constraint (see section 6.5.2), which may be beneficial in scenarios with safety-constraints.

Which algorithm should I now choose for my task?

The best deep RL algorithm for a task is highly dependent on the task and the available resources. The performance crucially depends on the choice of hyperparameters. How well the hyperparameters are chosen depends partly on the experience of the humans responsible for the hyperparameter selection process, and their expertise may be different across methods. For specialized domains, specialized algorithms may exist that can be expected to outperform general approaches.

Against this background, without domain knowledge, the author would recommend starting with PPO, which is rather robust and has proven to work well in practice for a broad range of applications; SAC is highly recommended for continuous control. For discrete action spaces, variants of DDQN, in particular Rainbow, are not outdated and give good results. For learning policies with only a few parameters, the CMA-ES is easy to use and a viable option.

Where to go next?

The review of deep RL by François-Lavet et al. [2018] gives a broader overview of the field, including links to advanced topics not discussed in this text.

We focused on model-free RL agents. However, learning world models is a big topic in AI. “A model is a form of reversible access to the MDP dynamics (known or learned)” [Moerland et al., 2023]. Model-based RL brings together learning and planning. In this setting, algorithms presented in this text can learn a policy using a model of the MDP. Having such a model enables the agent to plan ahead by simulating the MDP dynamics and predicting the consequences of actions. It allows for a form of generalization we have not discussed in this text, namely to generalize to new tasks, where the rewards have changed but the (learned) transition probabilities $P_{ss'}^a$ are still valid.

An area that also has recently gained a lot of attention recently is RL from human feedback (RLHF), which deals with the incorporation of human evaluations into the learning process. Human feedback is, for example, used in the tuning of LLMs (e.g., when humans evaluate the responses for GRPO), see Kaufmann et al. [2025] for a review.

Offline RL studies how an RL agent can learn best from historic data from previous interactions, which increases data efficiency and may give access to richer training data [Prudencio et al., 2023]. This is particularly relevant in real-world scenarios where it is not possible, too expensive, or too dangerous for the agent to interact with the environment.

Learning complex tasks can profit from breaking the task down into sub-tasks. This approach is followed in hierarchical RL. A high-level policy may select a subtask, which is then executed by a lower-level policy, see [Pateria et al. \[2021\]](#) for a review.

This text should have enabled the reader to apply model-free deep RL in an informed way. Thus, the next step would be to put the conceptual understanding into practice. There are many open source libraries that implement state-of-the-art deep RL algorithms, for example, Stable Baselines3 (SB3) [[Raffin et al., 2021](#)], which provides clean PyTorch implementations and is a good recommended starting point.

Index

- A²C
 - see* Asynchronous Advantage Actor-Critic 95
- A³C
 - see* Asynchronous Advantage Actor-Critic 95
- action, 8
- activation function, *see* transfer function, 38
- actor, 78, 89
- actor-critic, 21, 89
- actor-only, 21
- Adam, 32, 66
- AdamW, 32, 69
- advantage, 87, 93
 - optimization, 93
- advantage estimation, 95
 - generalized, 95
- advantage function, *see* advantage
- agent, 9
- almost-sure termination, 13
- asynchronous, 95
- Asynchronous Advantage Actor-Critic, 95
- baseline, 87
- baseline function, *see* baseline
- Bellman equation, 11
- Bellman error, 16, 62
- Bellman optimality equation, 13, 19
- bootstrapping, 18, 91, 94
- catastrophic forgetting, 100
- CE, *see* cross-entropy method
- classification, 29
 - binary, 30
 - multi-class, 30
- clipped double- Q trick, 71, 105
- clipping, 73
- CMA-ES, *see* covariance matrix adaption evolution strategy
- CNN, *see* convolutional neural network
- conservative policy iteration, 109
- control variate, 43, 87
- convolutional neural network, 33
- covariance matrix adaption evolution strategy, 51
- CPI, 108
 - see* conservative policy iteration 109
- critic, 89
- critic-only, 21, 89
- cross-entropy, 30, 104
- cross-entropy method, 48
- DDQN, *see* double deep Q -learning network
- deep Q -network, 69
- deep learning, 23, 25
- deep neural network, 25
- deep neural networks, 33
- direct policy search, *see* direct search
- direct search, 47
- discount factor, 10
- distributional Q -learning, 74
- double Q , 71
- double Q -learning, 71
- double deep Q -learning network, 71
- DQN
 - see* deep Q -network 69
- dueling network, 75
- dynamic programming, 3
- empirical risk minimization, 29
- encoder, 92

- entropy, 100
 - bonus, 95, 100, 104, 106
 - differential, 100
 - regularization, 100, 104
- entropy bonus, 102
- episode, 13
- ϵ -greedy
 - see* policy
 - ϵ -greedy 17
- error function, 29, 31
- η_γ^π , 81
- evolution path, 52
 - conjugate, 53
- experience buffer, *see* memory buffer
- experience replay, 65
- exploitation, 15, 17
- exploration, 15, 17

- forget bias, 40
- forget gate, 40

- GAE
 - see* advantage estimation
 - generalized 95
- γ , *see* discount factor
- gated recurrent units, 41
- gates, 40
- general multi-step return
 - see* return
 - multi-step 21
- generalization, 28
- goal state
 - see* terminal state 13
- gradient, 31
- gradient-clipping, 73
- Group Relative Policy Optimization, 105, 114
- GRPO
 - see* Group Relative Policy Optimization 114

- Huber loss, 74
- hypothesis, 28
- hypothesis class, 28

- importance weighting, 42
- input gate, 40
- internal state, 40

- KL divergence
 - see* Kullback–Leibler divergence 104
- KL regularization, 100, 104
- Kullback–Leibler divergence, 104

- large language model, 114
- layer, 25
 - convolutional, 33
 - fully-connected, 26
 - pooling, 33
- learning, 27
- likelihood, 29
- LLM
 - see* large language model 114
- log-derivative trick, 44
- long short-term memory, 39
- long-term cumulative reward, *see* return
- loss function, 28
 - robust, 74
- LSTM, *see* long short-term memory

- Markov decision process, 8
 - episodic, *see* episodic task
 - infinite-horizon, *see* infinite-horizon task
 - partially observable, 10, 55
- Markov property, 8
- maximization bias, 70, 105
- maximum entropy principle, 100
- MDP, *see* Markov decision process
- mean-squared error, 28
- memory buffer, 65
- mini-batch, 31, 65
- Monte Carlo method, 42, 47
- MSE, *see* mean-squared error
- multi-step learning
 - see* temporal difference learning
 - multi-step 21

- NAC,
 - see* natural Actor-Critic 117
- Natural Actor-Critic, 117
- neural network, 23
 - backbone, 92
 - feedforward, 23
 - head, 92
 - recurrent, 37
- neurodynamic programming, 3
- neuron, 24
 - hidden, 25
 - input, 25
 - linear, 29
 - output, 25
- NN, *see* neural network
- noisy networks, 74

- objective function, 47
- off-policy, 22
- on-policy, 22, 95

- on-policy distribution, 81
- one-hot encoding, 30
- online, 18
- online network, 67
- output gate, 40
- output state, 38
- overestimation bias, 105
 - see* maximization bias 70
- peephole connections, 40
- PER
 - see* prioritized experience replay 67
- performance difference lemma, 82
- PGT, *see* policy gradient theorem
- policy, 9
 - differentiable, 78
 - ϵ -greedy, 17
 - greedy, 14, 17, 20
 - optimal, 13
 - soft, 17
 - stochastic, 22, 79
- policy gradient method, 77
- policy gradient theorem, 80
 - deterministic, 85
- policy improvement theorem, 14
- policy iteration, 15
 - generalized, 17
- policy network, 48
- Polyak averaging, 68, 70
- POMDP, *see* partially observable Markov decision process, 22, 37
- pooling, 37
- PPO
 - see* Proximal Policy Optimization 108
- pre-activation, 24
- preferences, 78, 93
- premature convergence, 100
- prioritized experience replay, 67
- Proximal Policy Optimization, 108
- Q -function, 12
- Q -learning, 18, 63
 - tabular, 18
- Rainbow, 74
- recurrent neural network, 55
- reference policy, 93
- regression, 28
- REINFORCE, 86, 90
- reinforcement learning, 7
 - model-based, 22
 - model-free, 22
- reparameterization trick, 45, 105
- reset gates, 41
- return, 10
 - n -step, 21
 - expected, 10
 - multi-step, 21
- reward, 8
 - cumulative, *see* return
 - immediate, 8
- risk, 28
 - empirical, 28
- RL, *see* reinforcement learning
- RNN, *see* recurrent neural network
- SAC
 - see* Soft Actor Critic 105
- sample complexity, 22
- SARSA, 18, 63
 - tabular, 18
- score function, 44, 86
- shortcut connections, 25
- skip connections
 - see* shortcut connections 25
- Soft Actor Critic, 72, 105
- soft-max, 30
 - policy, 78, 93
- state, 8
 - terminal, 13
- state distribution
 - discounted, 81
 - improper, 81
- state-value function, 11
- steepest descent, 31
- step-size adaptation, 53
- stride, 37
- supervised learning, 27
- target network, 67
- target policy
 - smoothing, 99
- task, 8
 - discounted, 10
 - episodic, 13, 80
 - infinite-horizon, 13
 - undiscounted, 10
- TD error, *see* temporal difference error
- TD3
 - see* Twin-delayed Deep Deterministic Policy Gradient 99
- temperature, 102
- temporal difference error, 16, 91
- temporal difference learning, 15, 16, 91
 - multi-step, 21
- transfer function, 24

- TRPO,
 - see Trust Region Policy Optimization [117](#)
- Trust Region Policy Optimization, [117](#)
- Twin-delayed Deep Deterministic Policy Gradient, [99](#)
- update gates, [41](#)
- weight sharing, [92](#)
- weight-decay, [31](#), [74](#)

References

- J. Achiam. Spinning Up in Deep Reinforcement Learning, 2018. URL <https://spinningup.openai.com>. 99, 100, 102
- M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations (ICLR)*, 2021. 111, 112, 114
- H. Bai, R. Cheng, and Y. Jin. Evolutionary reinforcement learning: A survey. *Intelligent Computing*, 2:0025, 2023. 51
- B. Bakker. Reinforcement learning with long short-term memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1475–1482, 2001. 39
- A. G. Barto, R. S. Sutton, and C. W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983. 3, 55
- G. Baykal, A. Akgül, M. Haussmann, B. Tasdighi, N. Werge, Y.-S. Wu, and M. Kandemir. ObjectRL: An object-oriented reinforcement learning codebase. *arXiv preprint arXiv:2507.03487*, 2025. 105, 108
- M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013. 4, 74
- M. G. Bellemare, W. Dabney, and R. Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 449–458. PMLR, 2017. 74
- Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2): 157–166, 1994. 39
- C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, and R. Józefowicz. Dota 2 with

- large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019. [4](#), [58](#)
- D. Bertsekas and J. N. Tsitsiklis. *Neuro-dynamic Programming*. Athena Scientific, 1996. [3](#), [20](#)
- C. M. Bishop and H. Bishop. *Deep learning: Foundations and concepts*. Springer, 2023. [6](#), [23](#)
- J. A. Boyan. Technical update: Least-squares temporal difference learning. *Machine Learning*, 49(2):233–246, 2002. [63](#)
- S. J. Bradtke and A. G. Barto. Linear least-squares algorithms for temporal difference learning. *Machine Learning*, 22(1–3):33–57, 1996. [63](#)
- K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. [41](#)
- T. Dao and A. Gu. Transformers are SSMS: generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, pages 10041–10071, 2024. [41](#)
- K. Doya. Recurrent networks: Supervised learning. In M. A. Arbib, editor, *The Handbook of Brain Theory and Neural Networks*, pages 796–800. MIT Press, 1995. [37](#)
- A. E. Eiben and J. E. Smith. From evolutionary computation to the evolution of things. *Nature*, 521:476–482, 2015. [51](#)
- T. Eimer, M. Lindauer, and R. Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *International Conference on Machine Learning (ICML)*, pages 9104–9149. PMLR, 2023. [108](#), [112](#), [116](#)
- J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. [38](#)
- L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. In *International Conference on Learning Representations (ICLR)*, 2020. [112](#), [113](#), [117](#)
- M. Fortunato, M. G. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, C. Blundell, and S. Legg. Noisy networks for exploration. In *International Conference on Learning Representations (ICLR)*, 2018. [74](#)
- V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau. An introduction to deep reinforcement learning. *Foundations and Trends® in Machine Learning*, 11(3–4):219–354, Dec. 2018. [118](#)
- S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018. [71](#), [72](#), [99](#), [105](#)
- F. A. Gers and J. Schmidhuber. Recurrent nets that time and count. In *International Joint Conference on Neural Networks (IJCNN 2000)*, volume 3, pages 189–194. IEEE Press, 2000. [40](#)

- A. Graves and J. Schmidhuber. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks*, 18(5-6):602–610, 2005. 39
- K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber. LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232, 2017. 39, 40
- T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine. Learning to walk via deep reinforcement learning. In *Robotics: Science and Systems (RSS)*, 2019a. 105
- T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications, 2019b. URL <https://arxiv.org/abs/1812.05905>. 102, 103, 105, 108
- R. W. Hamming. On the distribution of numbers. *The Bell System Technical Journal*, 49(8):1609–1625, 1970. 10
- N. Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2023. 51, 54, 56
- N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001. 51
- N. Hansen, S. D. Müller, and P. Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003. 52
- M. J. Hausknecht and P. Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposia*, volume 45, 2015. 39
- V. Heidrich-Meisner and C. Igel. Neuroevolution strategies for episodic reinforcement learning. *Journal of Algorithms*, 64(4):152–168, 2009. 51, 55
- V. Heidrich-Meisner, M. Lauer, C. Igel, and M. Riedmiller. Reinforcement learning in a nutshell. In *European Symposium on Artificial Neural Networks (ESANN)*, pages 277–288. Evere, Belgium: d-side publications, 2007. 8
- M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver. Rainbow: combining improvements in deep reinforcement learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2018. 74
- S. Hochreiter. Untersuchungen zu dynamischen neuronalen Netzen. Diploma thesis, TU München, 1991. 39
- S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. 39
- S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. In S. C. Kremer and J. F. Kolen, editors, *A Field Guide to Dynamical Recurrent Neural Networks*. IEEE, 2001. 39
- L. Kaelbling, M. Littman, and A. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998. 10

- S. Kakade and J. Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2002. [82](#), [93](#), [109](#)
- T. Kaufmann, P. Weng, V. Bengs, and E. Hüllermeier. A survey of reinforcement learning from human feedback. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. [118](#)
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015. [32](#)
- Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015. [4](#), [23](#)
- Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Chintala, T. Lillicrap, O. Vinyals, F. Hutter, D. Silver, and D. Hassabis. Competition-level code generation with alphacode. *Science*, 378(6624): 1092–1097, 2022. [4](#)
- T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016. [96](#), [98](#)
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. [32](#), [33](#)
- S. Mannor, R. Y. Rubinstein, and Y. Gat. The cross entropy method for fast policy search. In *International Conference on Machine Learning (ICML)*, pages 512–519, 2003. [48](#)
- S. Mannor, Y. Mansour, and A. Tamar. *Reinforcement Learning: Foundations*. Cambridge University Press, 2026. [6](#), [12](#), [80](#), [82](#), [83](#), [87](#)
- A. Merke and R. Schoknecht. A necessary condition of convergence for reinforcement learning with function approximation. In *Proceedings of the 19th International Conference on Machine Learning*, pages 411–418, 2002. [64](#)
- M. Minsky. *A neural-analogue calculator based upon a probability model of reinforcement*. PhD thesis, Harvard University Psychological Laboratories, Cambridge, Massachusetts, 1952. Cited after [Sutton and Barto \[2018\]](#). [3](#)
- A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and J. Dean. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021. [4](#)
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. [4](#), [34](#), [38](#), [65](#), [68](#)

- V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, pages 1928–1937, 2016. [65](#), [68](#), [74](#), [93](#), [95](#), [102](#)
- T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 16(1):1–118, 2023. [118](#)
- C. Nota and P. S. Thomas. Is the policy gradient a gradient? In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 939–947, 2020. [80](#), [85](#), [88](#)
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:27730–27744, 2022. [4](#), [108](#)
- E. Parisotto, F. Song, J. Rae, R. Pascanu, C. Gulcehre, S. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, S. Noury, M. M. Botvinick, N. Heess, and R. Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 7487–7498. PMLR, 2020. [41](#)
- S. Pateria, B. Subagdja, A.-h. Tan, and C. Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys*, 54(5):1–35, 2021. [119](#)
- J. Peters. Policy gradient methods. *Scholarpedia*, 5(11):3698, 2010. [77](#)
- J. Peters and S. Schaal. Natural actor-critic. In *European Conference on Machine Learning (ECML)*, pages 280–291. Springer, 2005. [117](#)
- R. F. Prudencio, M. R. Maximo, and E. L. Colombini. A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE Transactions on Neural Networks and Learning Systems*, 35(8):10237–10257, 2023. [118](#)
- M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2005. [13](#)
- X. Qiu, Y. Gan, C. F. Hayes, Q. Liang, Y. Xu, R. Dailey, E. Meyerson, B. Hodjat, and R. Miikkulainen. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. *arXiv preprint arXiv:2509.24372*, 2026. [116](#)
- A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. [108](#), [112](#), [113](#), [119](#)
- M. Riedmiller. Neural fitted q iteration - first experiences with a data efficient neural reinforcement learning method. In *European Conference on Machine Learning (ECML)*, volume 3720 of *LNAI*, pages 317–328, 2005. [66](#)
- S. M. Ross. *Introduction to stochastic dynamic programming*. Academic Press, 1983. Cited after [Singh et al. \[1994\]](#). [22](#)

- R. Y. Rubinstein and D. P. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*. Springer, New York, 2004. [48](#)
- A. Saxe, J. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014. [113](#)
- T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016. [67](#), [74](#), [75](#)
- J. Schmidhuber. Deep learning in neural networks: an overview. *Neural Networks*, 61:85–117, 2015. [23](#)
- J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. [4](#)
- J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015. [82](#), [93](#), [104](#), [117](#)
- J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. [87](#), [95](#)
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. [108](#), [110](#), [112](#)
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. [4](#), [114](#)
- D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning (ICML)*, 2014. [80](#), [81](#), [85](#)
- D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. [4](#)
- D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017. [4](#), [58](#)
- S. P. Singh, T. Jaakkola, and M. I. Jordan. Learning without state-estimation in partially observable Markovian decision processes. In *International Con-*

- ference on Machine Learning (ICML)*, pages 284–292. Elsevier, 1994. [22](#), [129](#)
- I. Smirnov and S. Gu. RL BenchNet: The right network for the right reinforcement learning task. *arXiv preprint arXiv:2505.15040*, 2025. [41](#)
- R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3(1):9–44, 1988. [16](#), [18](#)
- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2. edition, 2018. [3](#), [7](#), [9](#), [14](#), [15](#), [17](#), [21](#), [58](#), [80](#), [86](#), [128](#)
- R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems (NeurIPS)*, 12, 1999. [78](#), [80](#)
- C. Szepesvári. *Algorithms for reinforcement learning*. Springer, 2022. [7](#)
- I. Szita and A. Lőrincz. Learning Tetris using the noisy cross-entropy method. *Neural Computation*, 18(12):2936–2941, 2006. ISSN 0899-7667. [48](#)
- C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone. Deep reinforcement learning for robotics: A survey of real-world successes. *Annual Review Control, Robotics, and Autonomous Systems*, 8: 153–188, 2025. [4](#)
- G. Tesauro. Practical issues in temporal difference learning. *Advances in neural information processing systems*, 4, 1991. [3](#)
- G. Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995. [3](#)
- S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 Connectionist Models Summer School*, pages 255–263. Psychology Press, 1994. [70](#)
- M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. [26](#), [55](#)
- J. N. Tsitsiklis and B. Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 9, pages 1075–1081, 1997. [64](#)
- H. van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010. [70](#), [71](#)
- H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, pages 2094–2100, 2016. [71](#), [74](#)
- O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Donato, J.-B. Martens, D. McIlroy, K. Kavukcuoglu, D. Silver, and D. Hassabis. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. [4](#)

- Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1995–2003. PMLR, 2016. [75](#)
- C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine learning*, 8:279–292, 1992. [18](#)
- R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. [86](#)
- R. J. Williams and J. Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991. [102](#)
- W. Zaremba, I. Sutskever, and O. Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014. [39](#), [40](#)
- B. D. Ziebart, J. A. Bagnell, A. K. Dey, et al. Maximum entropy inverse reinforcement learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2008. [100](#)